INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

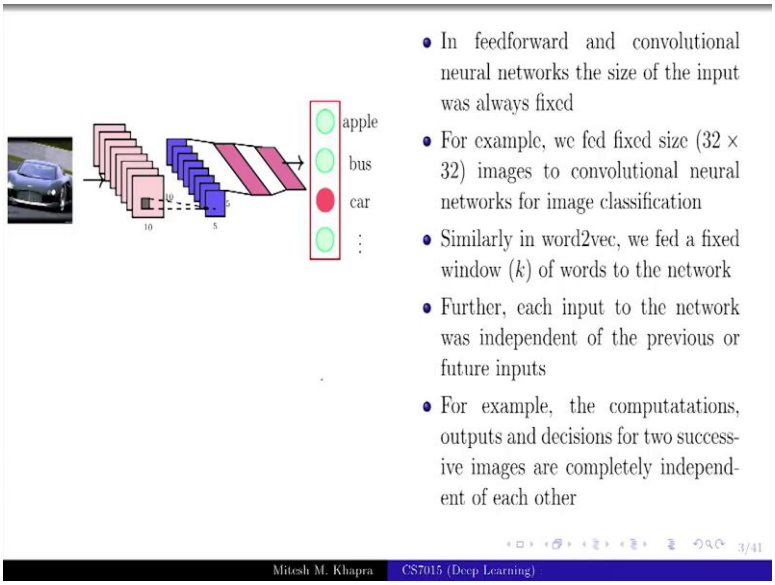
112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 13
Sequence Learning Problems, Recurrent Neural Networks, Backpropagation Through Time (BPTT), Vanishing and Exploding Gradients, Truncated BPTT

In this lecture we will talk about Sequence Learning Problems and in particular some neural network architectures which deal with sequences, so recurrent neural networks is what we are going to see. So, we will start with the first module which is on Sequence Learning Problem.

(Refer Slide Time: 00:30)

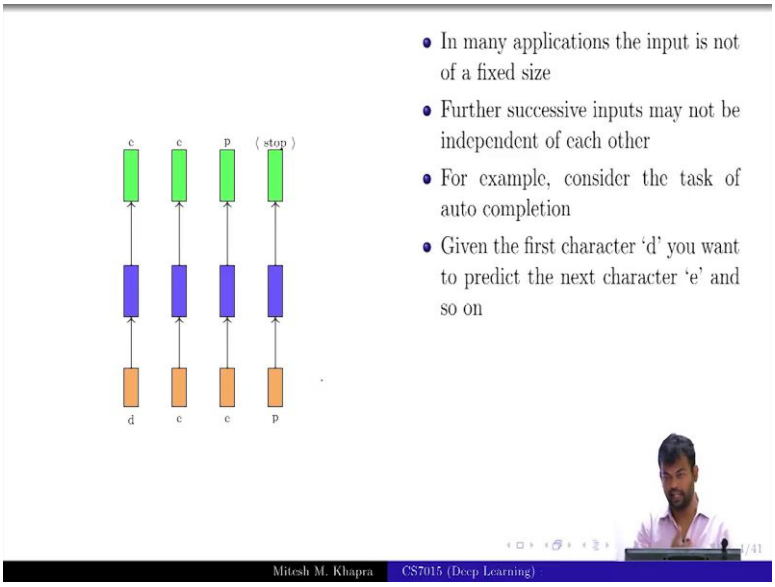


So, what are Sequence Learning Problem? So, so far we have dealt with two types of networks one is feedforward neural networks and the other is convolution neural networks and both these networks the input was always of a fixed size.

So, what do I mean by that is, if you take a convolution neural network you are feeding 32×32 images to it or 227×227 images to it and this size will always fixed. All your training images, all your test images were always scaled or cropped to this particular size, ok. Similarly when we used feedforward neural networks, so one example was word2vec, the size of the input was always fixed we had this input of size 2 v, right or k v in general; if you are looking at the k word window, right.

So, this input was not varying from one training instance to another training instance or one training instance to the test instance or anything. And secondly, each input to the network was independent to the previous or future inputs. So, I pass an image of an apple I get the prediction apple, then I pass some other image to the network and I get a different prediction. It does not matter whether my previous image was a apple or a car or a mango or whatever it just reads each of these inputs independently. There is no dependence between the inputs and the size of the inputs is fixed.

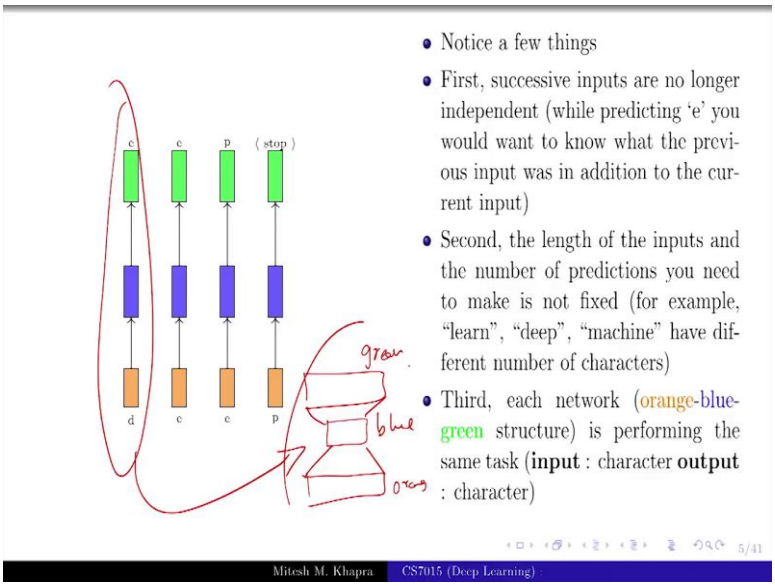
(Refer Slide Time: 01:46)



But in many applications the input is not of a fixed size. So, and also successive inputs may not be independent of each other. So, let us understand this with the example of auto completion that all of us are used to while typing SMSs or Whatsapp or other things.

So, given the first character d, I want to predict the next character which is e then once I have predicted e, I want to predict the next character again and so on till I get the full word, ok. This is what my task is.

(Refer Slide Time: 02:21)



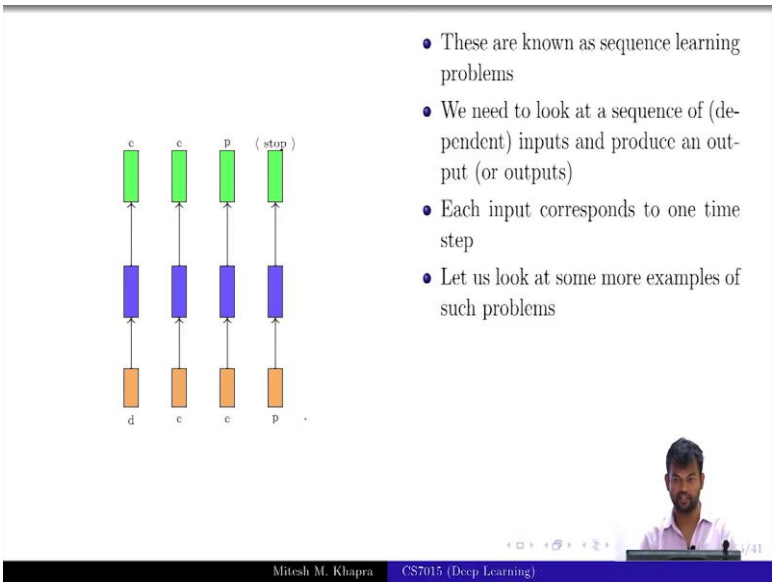
So, let us notice a few things. First successive inputs are no longer independent. If I know that the previous input was d and the correct input is e then I know that only a few things are possible, right. In particular if you know that the previous input was a z and the correct input is a e then most likely the next is going to be a b, right. But if you ignore the previous input which is z, then after e there are many things which can appear, right. So, the inputs are no longer independent of each other.

And the second thing is, the length of the input is not fixed because words could be of arbitrary sizes I am trying to type the word deep that is 4 letters or if I am trying to type the learn which is 5 letters machine which is 7 letters and so on, right. So, the input size is no longer fixed and the inputs are now dependent on each other, right; there is some dependence between. So, now, this is very different from what we saw in convolutional neural networks and feedforward neural networks. So, how do we deal with this?

And the third thing here is that, each network now I am calling this as a network and I will just clarify some notations also soon. Each network is actually performing the same task. It is taking as input a character and it is producing as output one character. And now just remember that these networks I have drawn them vertically. You are used to seeing them as this. So, this is input, this is your hidden layer, and this is your output. So, this is the green part, this is the blue part, and this is the orange input and this is the fully connected layer, right.

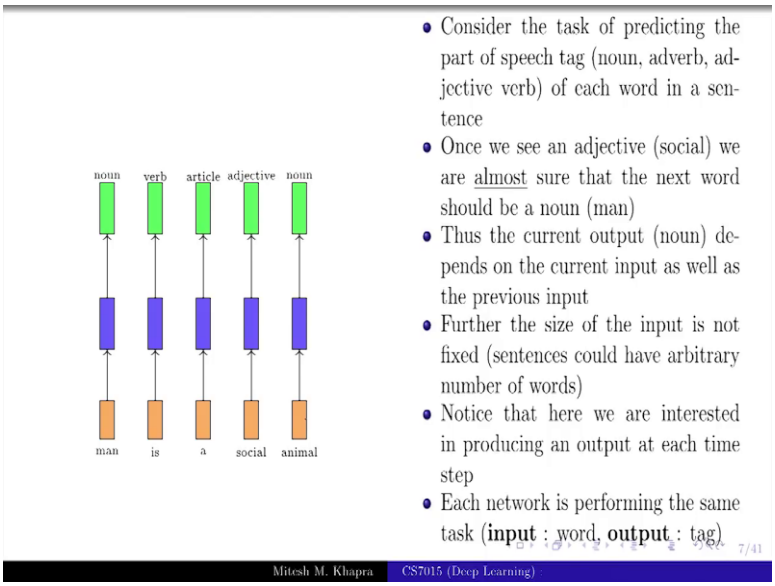
So, each of these boxes is actually this network. Ok I have just drawn it more concisely because I need to draw many such networks. So, everyone gets that? Just remember this mind that each of these orange, blue, green, structures is a fully connected network like this.

(Refer Slide Time: 04:09)



So, these problems are known as sequence learning problems where you have a sequence of inputs and then you need to produce some outputs. And each input actually corresponds to one time step, so this is the input at time step 1, time step 2, time step 3, time step 4 and so on. So, let us at some more examples of such sequence learning problems.

(Refer Slide Time: 04:29)



So, one classic example is the task of predicting the part of speech tag of every word in a sentence, right. So, I am given a sentence man is a social animal and for every word I want to predict whether it is a noun or an adverb or an adjective or a verb or any other part of speech type, right and this is how it happens.

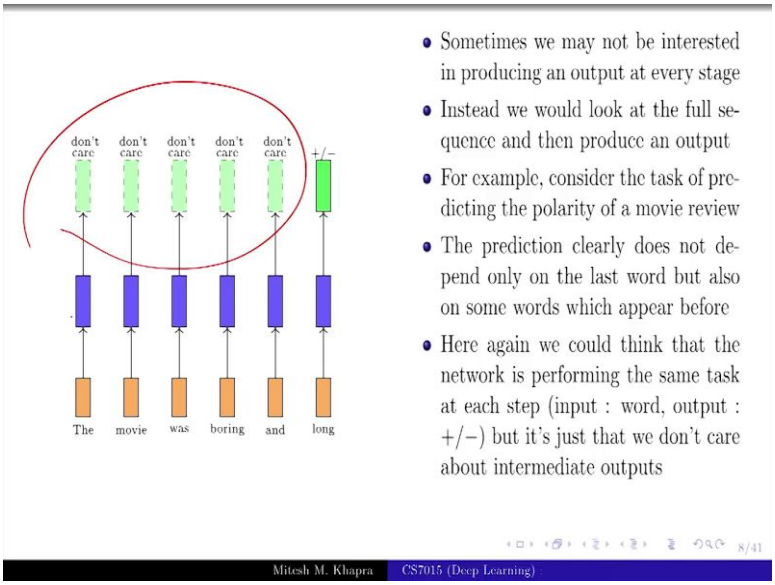
Now, notice that once we see an adjective in this case social we are almost sure that the next what is going to be a noun or at least we are sure that the next word cannot be an article or most likely it will not be a verb, right. There is a very high prior that the next word is going to be a noun. So, that is why these inputs are actually dependent on each other.

So, the current output not only depends on the current input it is also actually depends on the previous input, right. Unlike the case of convolutional neural networks where I feeded an apple it is no dependence on whether the previous input that I pass to the network was an apple or a car or what not. And the size of the input is not fixed because these sentences could be of arbitrary lengths I could have sentences as small as 3 to 4 words or as long as 25 to 30 words, right. So, average Wikipedia sentence for example, is 25 words, roughly 25 words.

And notice that and this case we are interested in producing an output at every time step, because for every input I want an output. And each network again this orange, blue,

green structure is performing the same task, its taking as input a word and its producing an output. What is producing as an output? Part of speech tag.

(Refer Slide Time: 06:01)



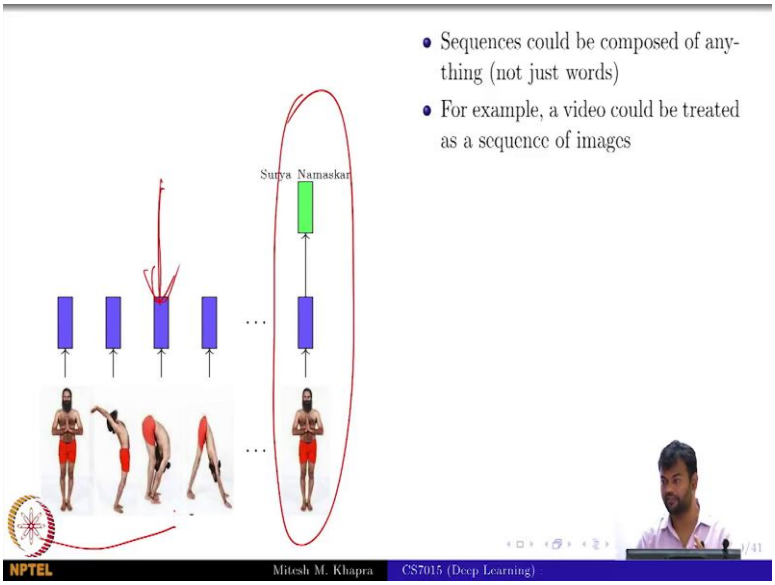
So, here the two examples that we saw we were having an input and every time step and an output at every time step. But there could also be cases we are interested in producing the output only at sometime steps or at the final time step. So, let us consider task of predicting the polarity of a movie review, right sentiment analysis.

So, I am given a movie review and after I have read the entire review I should give a prediction, right otherwise it would be incomplete. I cannot actually look at only this word and give a prediction does not make sense. I does not also make sense to make a prediction here at this point, because there could have been the movie was boring, but I still loved it or but the action was amazing or something like that it because it could have already flipped after that. So, I need to look at the entire sentence and make a prediction. But you are not interested and prediction as these intermediate time steps.

Even in this case you can actually assume that every network is performing the same task its taking a word as an input and it is producing some output it just that till the end you do not care about you outputs you care about the output only produced at the final step. You do not care about what the outputs are at this time step, right that is one way of looking at it.

So, again at every time step we have the same network, but you are only interested in some time steps of the network, ok.

(Refer Slide Time: 07:14)



Finally, it is not always necessary that sequences are composed of only words. What other kinds of sequences are you familiar, popular sequences? Speech is one, video is another.

So, a video could be treated as a sequence of images and now you could have a video where some, someone is performing Surya Namaskar. And as you can understand that I need to look at the entire sequence and only then be able to make a prediction, right. If I stop at this point if I only consider this, this is only Namaskar, no Surya Namaskar, right. So, you have to look at the entire sequence and then decide what the output is and you do not care about the intermediate outputs. I do not care what is the prediction till this point. This of course, is again some aasan but I do not care about that, I care about the full sequence that I am dealing with. It is just to motivate that sequences can be of all types.

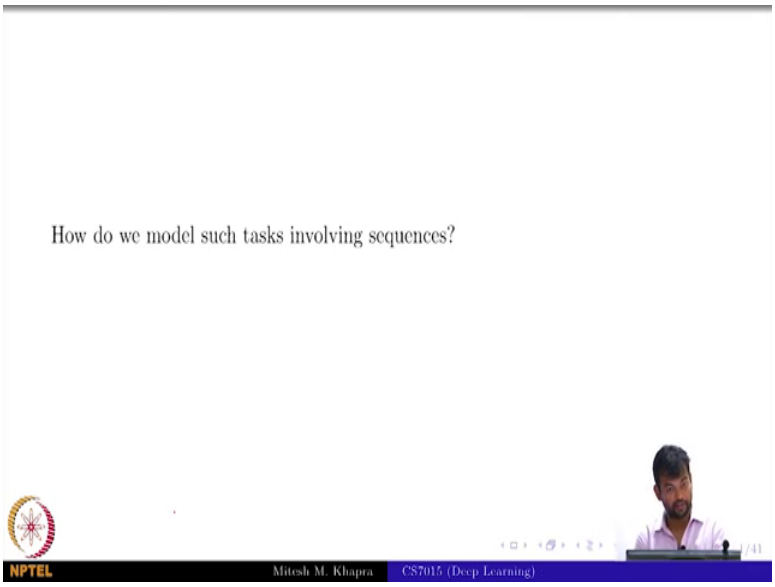
And I apologize to the speech people, I do not really understand much of speech processing, so I never give speech examples. But video is something I understand, so I can give examples on that.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 104
Recurrent Neural Networks

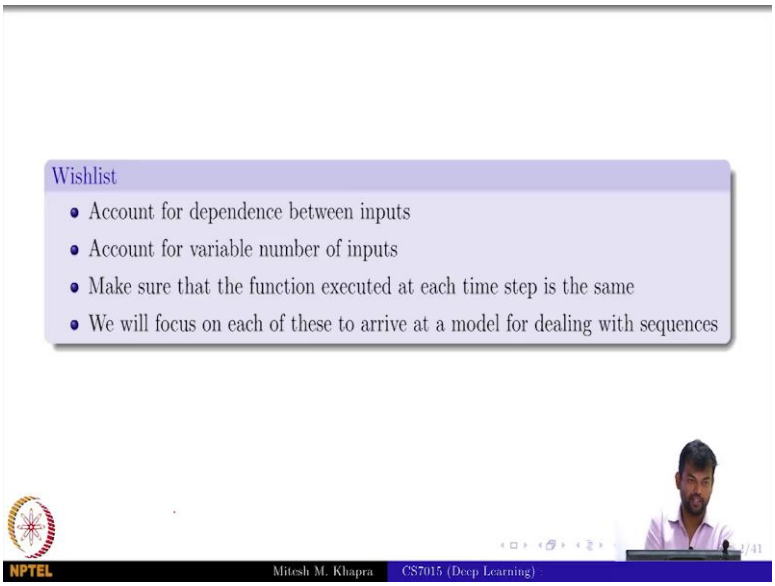
So, we have seen Sequence Learning Problems. Now, we are interested in the question of how to model these, right. So, we look at something known as recurrent neural networks.

(Refer Slide Time: 00:23)



And our question that we are interested is how do you model tasks which involves such sequences, ok.

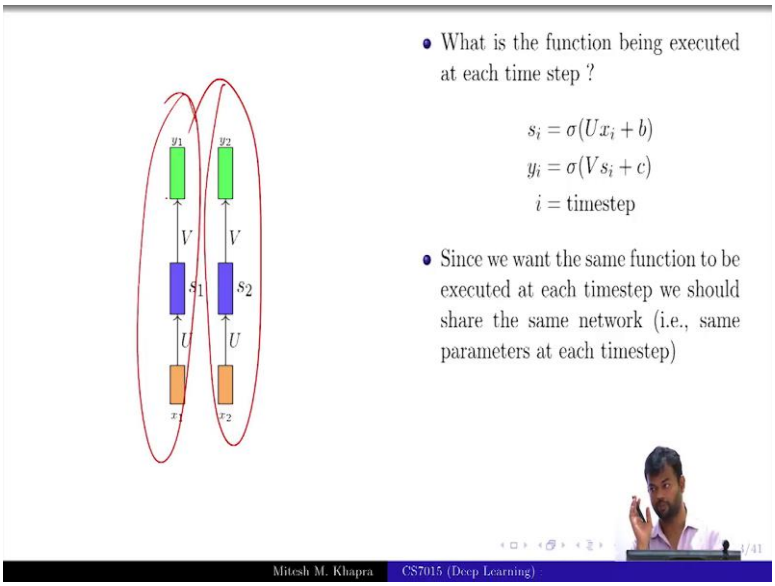
(Refer Slide Time: 00:27)



So, here is the wishlist that we have. What will model will come up with should account for the dependence between inputs? Because that is the strong case that we have made, that the output actually depends on multiple inputs and not just a single input. You should also account for variable number of inputs because a video could be 300 seconds, it could be 20 seconds, 25 seconds; a sentence could be of arbitrary lines and so on.

And it also makes sure that the function at each time step is the same, right but every time step they are trying to do the same activity, ok. So, we will focus on each of these items from our wishlist and then try to arrive at a model for dealing with such problems.

(Refer Slide Time: 01:05)



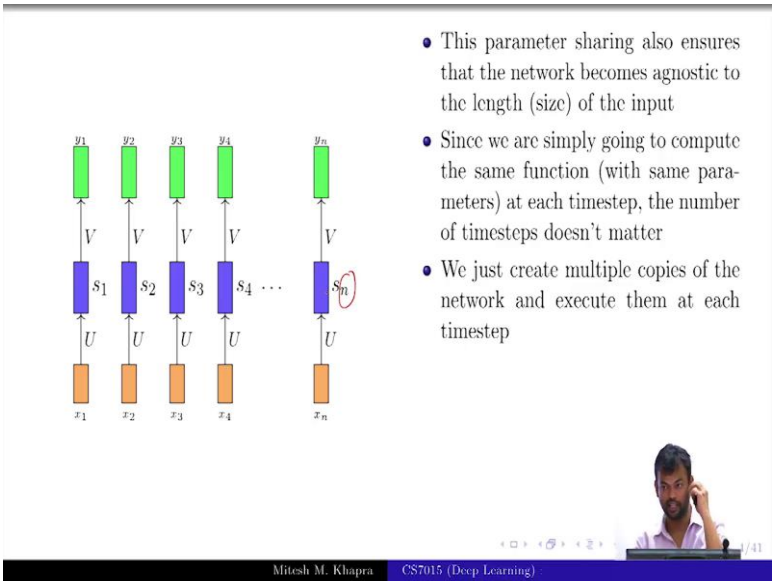
So, first let us ask this question what is the function being executed at each time step? What is the function being executed at each time step? Either should come after dealing. You have an input your ability to go blank on me is just amazing actually. You have a hidden representation and then you have an output. The first time we are seeing this situation in the entire course where we have an input a hidden representation and an output.

What is the function being executed? Remember the output is always a function of the input. Lecture 2 or 3, I do not know; definitely not lecture 14. So, what is the function? Can you write y i as a function of x? For my sake if not for god sake, you can, ok. What is it? Ok, first tell me what is s 1? U s 1, no nonlinearity, no bias which all that plus bias then no nonlinearity; who cares about nonlinearities, ok. And then what is y_1? Ok, some output function, ok. For some reason I have written sigma here. I will just call the output function as O always maybe in this case sigma would work but O is what I will call it just to make the this thing clear, ok; is that fine.

So, this is the function being executed at this every time step we can just write it using these two equations which are seeing for a first time, and i is a time step. Since we want the same function to be executed at each time step we should share the same network at every time step. That means, what do I mean by share the same network? Share the same parameters good. So, this is the same as this because U, V and b and c are the same, ok.

So, that is an easy way of taking care of the requirement that I want the same function to be executed at every time step.

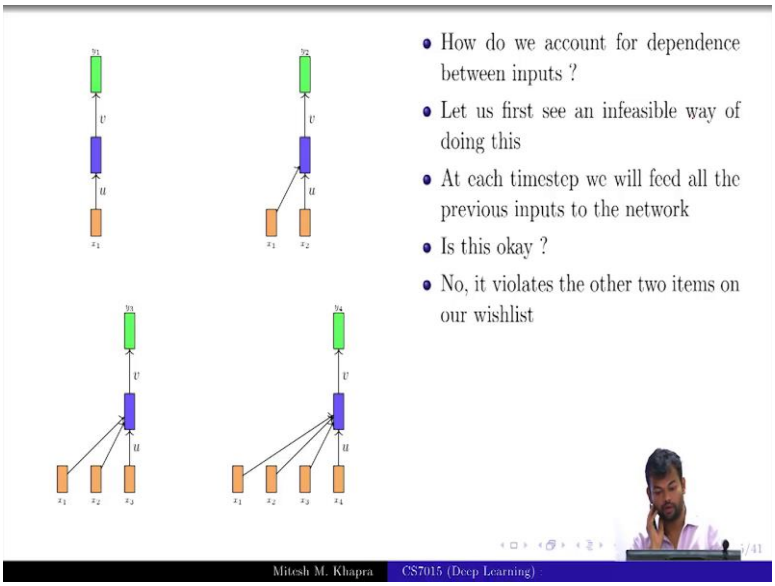
(Refer Slide Time: 02:51)



And this parameter also sharing also ensures that the network becomes agnostic to the length of the input. Because now whether I have a word which has 10 characters or 20 characters it does not matter, because at every time step I am going to execute the same function. That is why it is important that at every time step we have the same function.

So, since we are going to complete the same function the number of times it does not matter and we can just create multiple copies of this network that we have and for any arbitrary length n we can still compute the output, ok. Still not quite there, we still need to take care of a lot of things, but we are just slowly addressing each item from our wishlist.

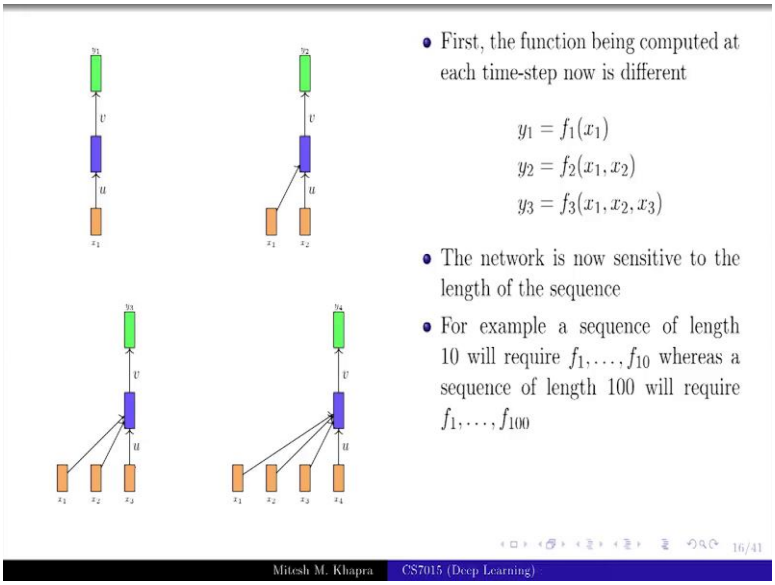
(Refer Slide Time: 03:30)



Now, how do we account for dependence between inputs? Or rather actually the, right way of asking this is how do we account for the case that the output actually depends on multiple inputs and not just the current input, ok? How do we account for that? Feed in the ok, good.

So, let us first see an infeasible way of doing this, ok. So, you are given the first time step x_1, you have a network which predicts 1 from x_1. You know at the same second time step you also want to look at the previous inputs, so why not just feed it x_1 and y x_2 both and then try to predict y 2. At the third time step feed in x $_1$, x $_2$, x_3 and predict y_3 and so on forever. Is this fine? probably the word infeasible is there. So, y is; so what is the problem with this? Yeah good. So, I am looking in terms of the conditions that we have on the wishlist; which condition does is violet. What is the function being executed at each time step, ok? So, let us see.

(Refer Slide Time: 04:31)

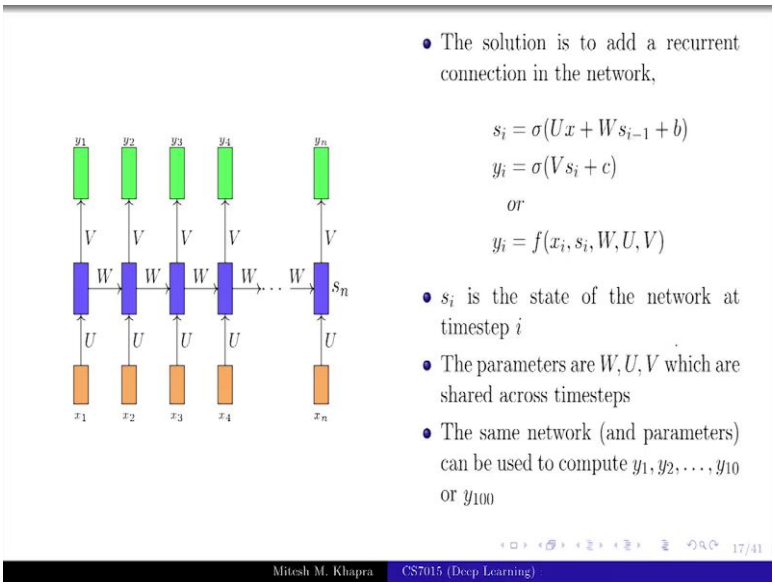


The function being executed at every time step is different. So, y_1 is function of x_1, y_2 is a function of x_1, x_2. Remember that this is not just saying that you are passing to inputs everything changes, because you now you need to have u_1 and u_2 here you need to have u_1, u_2, u_3, right. So, everything changes; it is not the same function how many of you get this it is a different function being executed at every time step.

So, now, if I have a sequence of length 100 what happens? I need y 100 which takes f x_1 to x_{100} as inputs. And has how many parameters? u_1 to u_{100}, right. You could you could share u_1 to u 99, for y 99 and y 100 but we still need those many of that, right. So, that network is now sensitive to the length of the input and on the length of the input goes you will have to construct more and more functions, right.

And imagine that if the training time the maximum sequence length that you had seen was 25, and suddenly a test time you get a sentence which is of length 30 you do not even know how to compute that because you have not train any parameters for doing that.

(Refer Slide Time: 05:36)



So, then the final solution is actually to add a recurrent connection in the network. Why does this work? Ok, before that, now can you tell me what is the function being executed at every time step? Assume there is a s 0 here; these are a s $_1$, s $_2$, s $_3$ up to s $_n$ and there is a s $_0$.

Now, what is the function being executed at every time step? Can you write it down? If you? It would help if you think in terms of y $_2$ and not in terms of y $_1$, y $_1$ is the boundary case was special case the thing in terms of y $_2$ or any other of the y's. And first think of what s $_2$ is from s $_2$ y is straight forward. How many of you can write the function? So, s i in general s i is U into x i plus W into s i minus 1 plus b. How many of you get this? And then what is y i? Again this has to be output functions, ok. But how does this solve our problem? Does it take care of everything on the wishlist? One the way we have written it in terms of i, which is the time step definitely the same function is getting executed at every time step. There is no doubt about that, right. Modulo this boundary case of s 1 where will assume that there is an s 0, ok.

So, same function being executed at every time step. Can you deal with inputs of arbitrary length? Yes, as long as you ensure that the same function is executed its fine. Does it ensure that the output is actually dependent on the previous inputs? How?

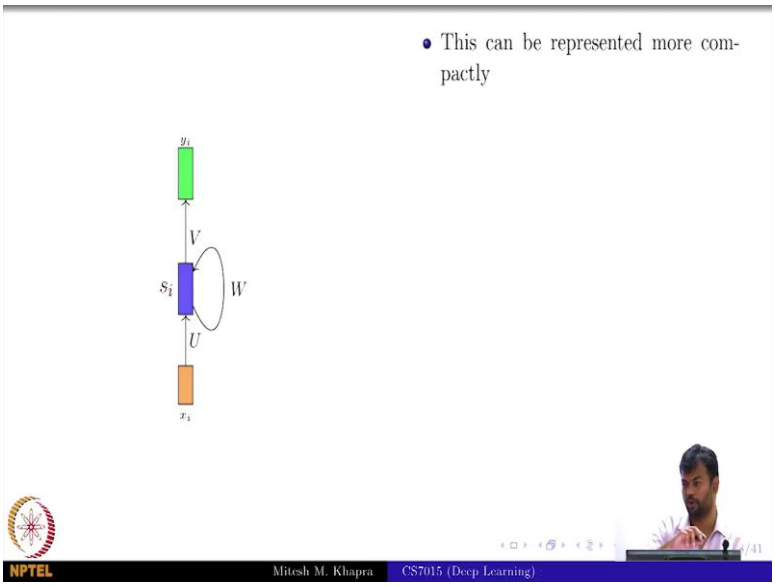
Student: (Refer Time: 07:01), s i.

Through s i minus, right, so that is an interesting thing that this guy actually depends on this guy which depend on the previous input and also on this guy which in turn depends on the provision inputs. So, recursively you can see that you depend on all the previous inputs that you had, ok. That is a very neat way of ensuring that your output depends on all the previous inputs and you do not blow away the parameters blow of the parameters by sharing this recurrent connection, and that is this is a compact way of writing is that your y i is now function of x i s i and has these parameters W, U, V and b and c. So, s i is called the state of the network at times step i.

And what is see here, this is just for the sake of completion this is known as a recurrent neural network because of this recurrent connection and s i is a state of the network at time step i. So, as when you start working in deep learning and you are dealing with sequence problems state of RNN or state of LSTM or state of GRV something that you will be hearing or reading often. So, this is what you mean by the state of the recurrent neural network this is the current state which kind of encode everything that is happened so far, right. It has a encoding of all the inputs that you had seen so.

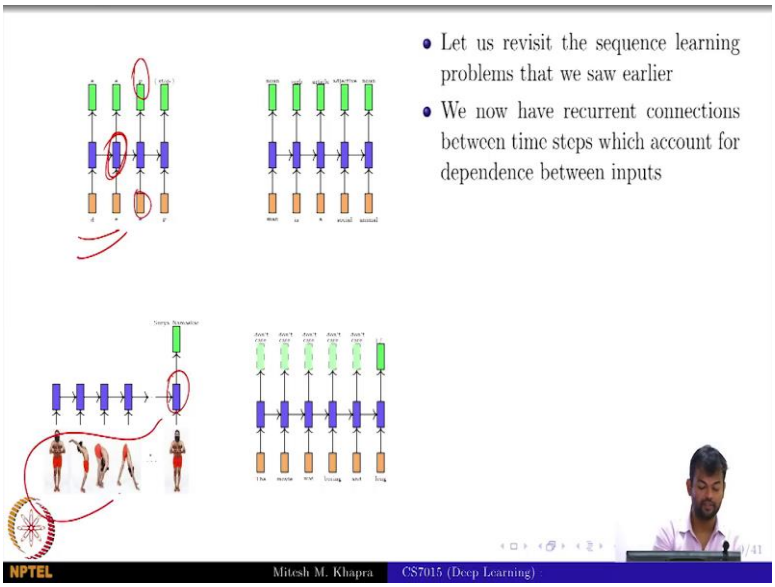
The parameters of the network are W, U and V which I shared across time steps I obvious forget the biases. And the same network is getting executed every time steps I do not need to worry about whether I am computing y_1, y_2, y_3 or y_{100}, right. So, everyone agrees that this solution takes care of all the things that we had on our wishlist. How many of you agree with that?

(Refer Slide Time: 08:31)



And this is a more compact way of representing that that you say that you compute s i and then you are feeding it back. So, this is just more compact way of representing a recurrent neural network.

(Refer Slide Time: 08:41)



So, let us now revisit the sequence learning problems that we have seen. So, now, just correct each of these networks. So, what would happen? Each of these things I was thinking of all the inputs as independent. So now, what will I do? What is the only thing to be done? Just add the recurrent connection, right.

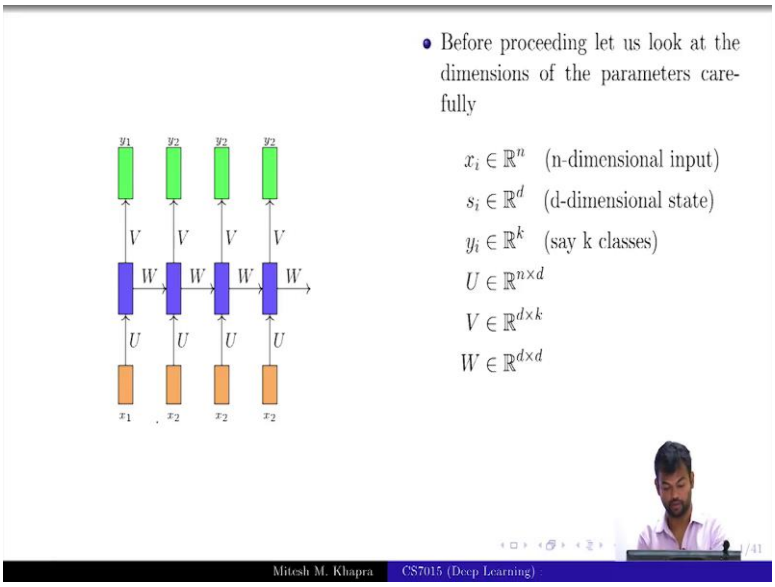
So, once you add the recurrent connection, now you can go back and relate to all these problems that one I am trying to predict the character which appears after e. I also have the information of d and e. And same argument you can make for all the other examples that you have. But I am trying to predict this final state I have the information of all the previous inputs here, ok.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 105
Backpropagation through time

So, that was Recurrent Neural Networks. Now, whenever we propose a network what do we do next? Training, right. So, what we will look at it back propagation through time. This is not the title of a fiction and movie or anything; this is an algorithm that we will see.

(Refer Slide Time: 00:27)



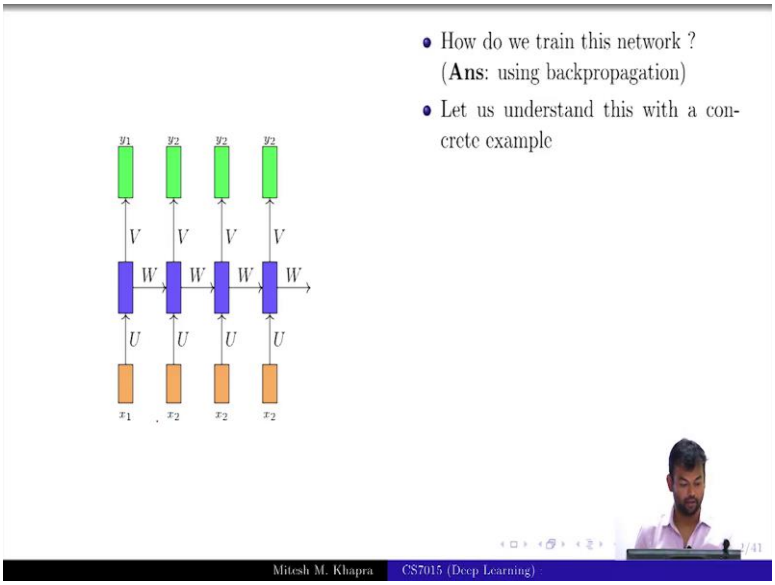
So, before we proceed, right let us look at the dimension of the parameters that we have and I expect you to tell me the dimensions. So, I will define somethings for you which are very hard. So, x_i belongs to R^n, so let us be clear about that; s_i belongs to R^d that means, the s_i is a d dimensional vector and y_i belongs to R^k which has k classes, ok.

So, now what is U? What is V? d × k, is it d × k I am asking Soham? Now I mean we have be written it as d × k. And W is?

Student: (Refer Time: 01:05).

d × t sure? Everyone sure? Ok, right. So, these are the dimensions. Why am I talking about these dimensions? Whenever we talk about gradients what we talk about partial derivatives or gradient or something, we need to know what is the size of the parameter with respect to which we are taking the gradient because that is what the size of the gradient matrix is going to be, right. That is why I am asking you to focus on this.

(Refer Slide Time: 01:31)

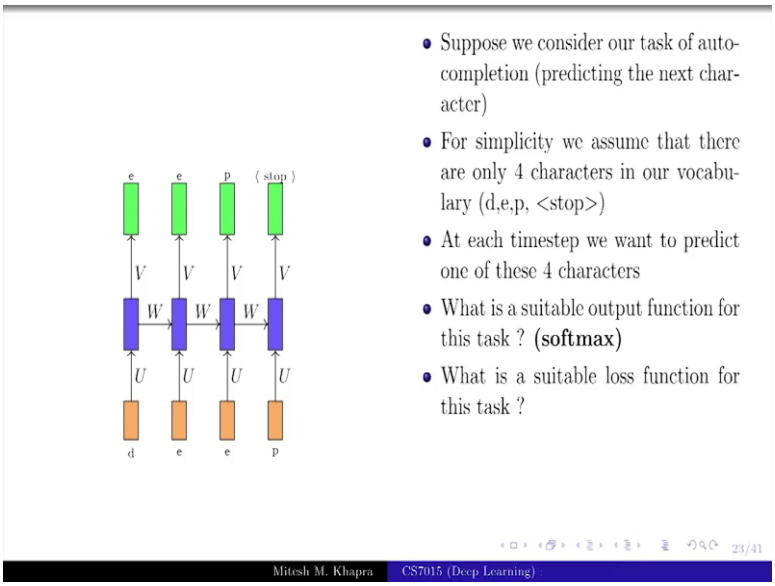


Now, how do we train this network? Title of the module.

Student: Backpropagation.

Backpropagation, ok. How? Why do I have a module if I am only going to tell you about backpropagation? Do you see any problem with this? Why cannot you just apply the standard backpropagation algorithm? Ok, so, we will try to understand this with the help of a concrete example and we will go back to our example of predicting characters, ok.

(Refer Slide Time: 01:51)



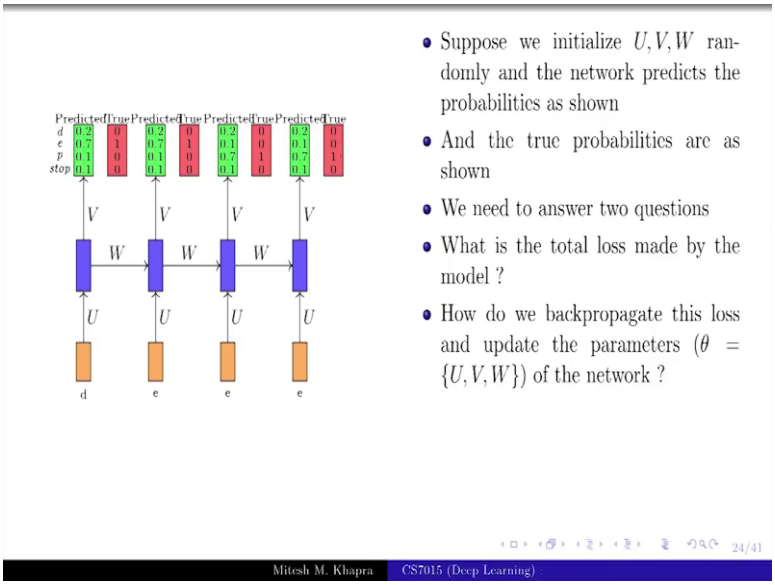
So, this is the auto completion task and for simplicity we will assume that English has only these 3 characters d, e, p and then a stop to indicate that the word has been completed, ok. This is what you are going to consider that my vocabulary size is just 4 that means, I can only predict one of these k 4 classes, k is equal to 4, ok.

And at each time say I want to predict one of these things. What is the suitable output function for this task? Can everyone say with probability 99.9 percent?

Student: Half max.

Half max, ok. What is the suitable loss function for this task? Small pleasures in life that is all I get, ok.

(Refer Slide Time: 02:34)

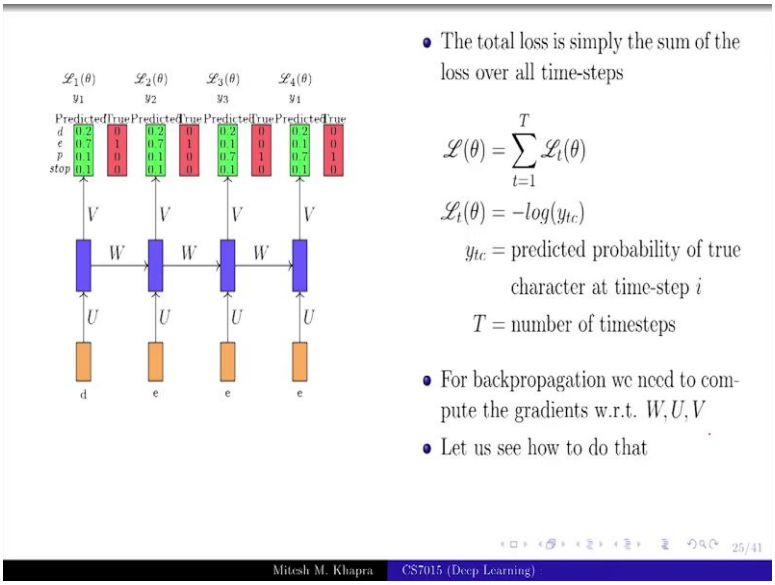


Suppose we initialize U, V, W randomly and networks predicts the following probabilities, ok. So, let us understand what is happening. I fed it d as the input I have just started training. So, my U, W and V are all some randomly initialized weight matrices, right now, and so it has predicted this as my probability distribution, this is the predictions that I have got from the network.

And I also know what is the true probability distribution. What is the true probability distribution, for the first time step? 0 1 0 0 and so on, right you can see it. Second times that is also 0 1 0 0; third is 0 0 1 0 and the last one should have been 0 0. So, given the situation and before I talk about learning algorithms, what is the first thing that I need to define? Objective function, right. So, what is the objective function here? How many errors do I have? I mean I can make my errors at 4 places, whether I making an error or not is the separate case but I can have 4 loss functions.

So, then these are the two questions that I am interested in. What is the total loss made by the model and how do we back propagate this loss and update the parameters of the model as usual I am ignoring the biases which is W, U and V. So, we can answer these two questions then we are done, right. If you can do this then we are done.

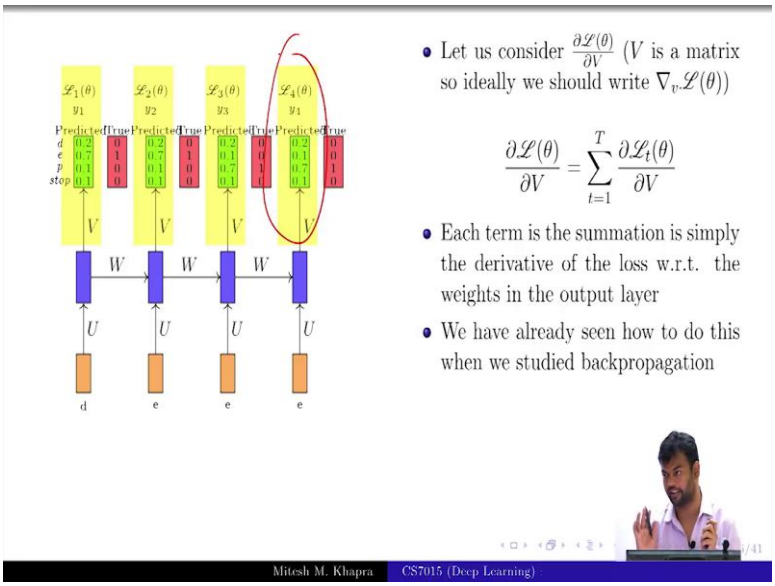
(Refer Slide Time: 03:53)



So, the total loss, what is the total loss actually? Take a guess, sum of all the loss, right good. So, just going to be the sum of the loss over the times steps that you are I mean very logical and what else would it be. And we know that the loss at every time step is, so this is the loss at time step t hence y t. And what is c actually? The true class at time step t, right. So, it is would be e at first time step, e at second time step, then p and then stop, ok, so that what c is.

So, this is we all comfortable with is this is the cross into p loss and I am going to sum at over all the t time setup that I have. Now, for back propagation what we need is we need to be able to compute the gradient of this loss function with respect to W, U, V.

(Refer Slide Time: 04:48)



If I give you your formula for the gradient the rest is straight forward you will just apply gradient as well, ok. So, let us look at each of these parameters. We will look at the easy one first which is V. So, what is the derivative of the lost function with respect to V? Have you ever done this in life?

Student: Yes.

Yes.

When?

Student: (Refer Time: 04:55).

Now, I am asking the date, ok. So, you have done this when you doing backpropagation. This is the gradient of the loss function with respect to the weights in the output layer and we know how to do that, right. That is very straightforward and there is no complication there. And you will see what I mean by complication later on.

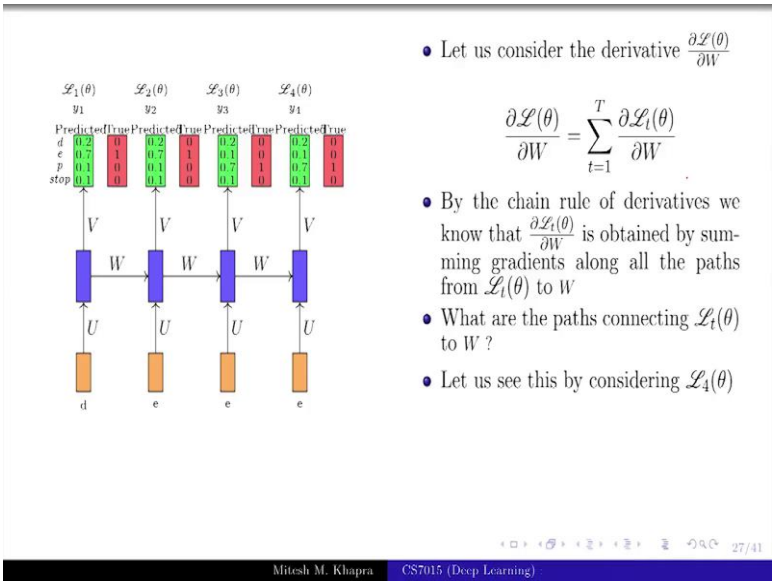
So, all I need to do is take this loss function and compute its gradient with respect to V, it is very simple chain rule which I can update there, apply there and I can compute it separately for all these guys and I can just sum it up, right. So, this is the easy part. This is very straight forward. So, where one parameter we are all set, we know how to do that, right. We can just add up all these gradients, the some lose notation here this is actually

an addition of 4 matrices, right. Each of this I hope is a matrix. Is that a matrix or a scalar or a vector or a tensor?

Student: Matrix.

Matrix, ok? So, we have already seen how to do this back propagation. And this is a smallest chain possible in the back propagation and we have enough confidence in doing this.

(Refer Slide Time: 05:56)



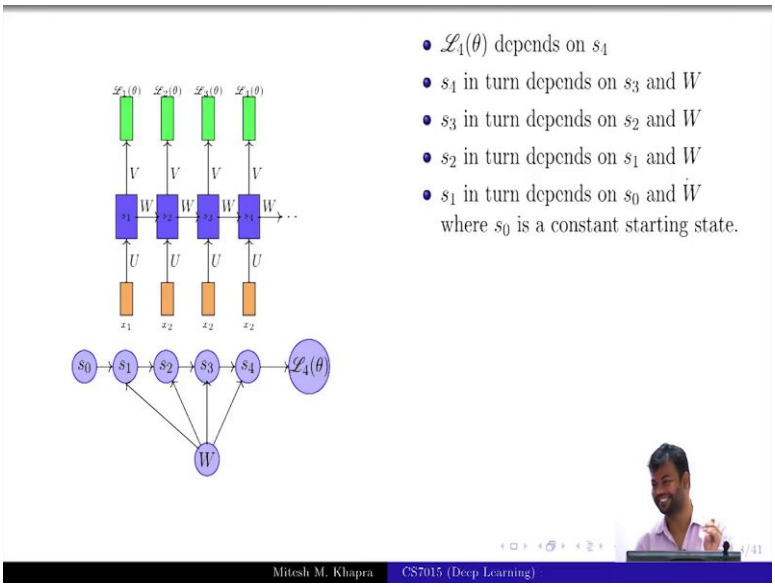
Now, let us considered the derivative of the loss function with respect to W. Just take a minute and see if it is complicated or if it is straight forward to see a lot of W's in the figure, ok. So, let us see how to do that, right.

So, again the loss with respect to W or the derivative with respect to the loss derivative of the loss with respect to W is going to just be the sum of these 4 or t derivatives. And by changed of derivatives we can just sum the derivative across all the paths which lead from the loss function to W, is that fine, right. Whenever, you want to compute the derivative of the loss function with respect to any parameter a recipes to look at all the paths which go from the loss function to that parameter and some of the gradients across those paths. How many if have fine with this? What are the paths which are actually connecting the loss function to W?

Student: (Refer Time: 06:51).

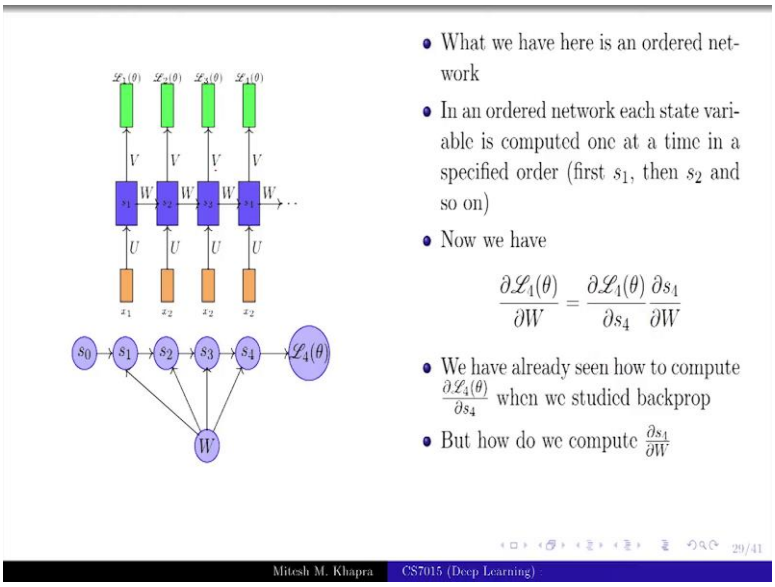
There will be t paths, good. So, let us see we will consider $L_4(\theta)$, this is the last time step.

(Refer Slide Time: 06:57)



So, $L_4(\theta)$ actually depends on s_4, s_4 depends on what? W and s_3. s_3 depends on what? W and s_2, s_2 depends on what? An s_1 depends on W and s_0, always assume there is s 0. What kind of a network is this? What kind of a function is this? What did I ask to revise? This is not an order derivative. What kind of function is this?

(Refer Slide Time: 07:19)



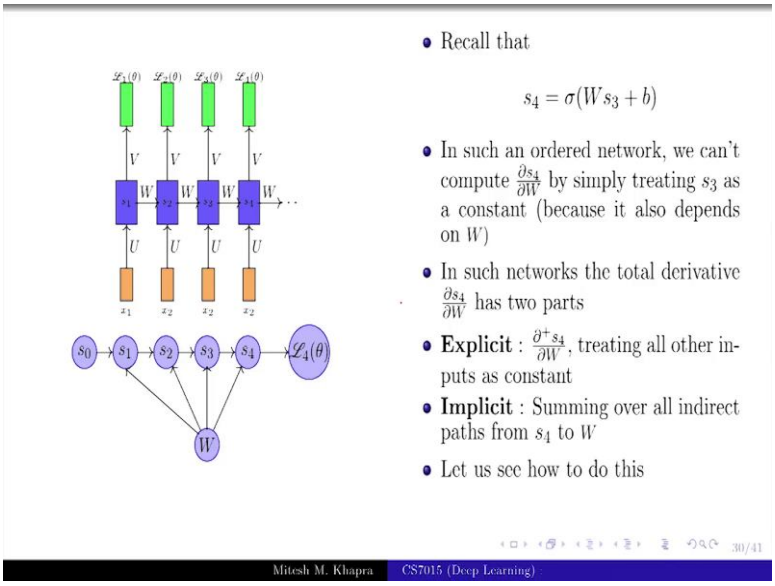
So, we have an ordered network with I will give it to you and it is not be to. Say in an ordered network each state is computed one at a time, right. So, we will first compute s 1, then we will compute s_2 because s 2 depend on s_1 there is no other way we can compute s_2, then s_3, s 4 and then finally, the loss function.

So, now, we have the following situation that the derivative of $L_4(\theta)$ with respect to W can be written using this chain rule which is the derivative with respect to s 4, and then the derivative of s 4 with respect to W. And that is that looks manageable there is nothing fancy here or is it I see a lot if people that looks manageable, right; everyone is not (Refer Time: 08:01).

Student: (Refer Time: 08:01).

Even though you have done the assignment everyone is not; even though you have revise the assignment everyone is not (Refer Time: 08:06). So, this part we have already seen. This is not the tricky part. $L_4(\theta)$/ s_4 is straight forward because it only depends on this V and so its fine that part we have seen. This is same as computing the gradient of the loss function with respect to the hidden layer. But now let look at the derivative of s_4 with respect to W.

(Refer Slide Time: 08:28)



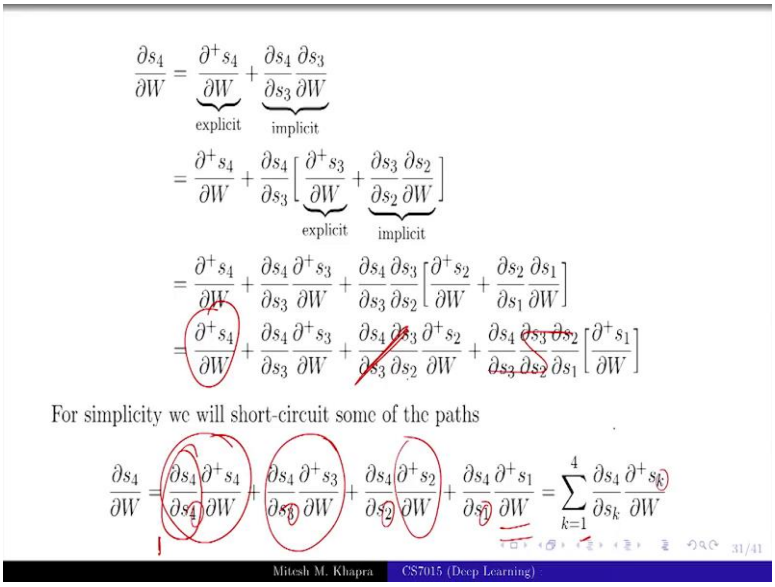
What is s_4 actually? $\sigma(Ws_4 + b)$. So, now, if I want to compute ∂s_4 by let me just remove the sigma, right I mean we can always get back the nonlinearity. So, I want to compute $\partial s_4 / \partial W$. So, it will just be s_3, s_3 is again.

Student: Depend on W.

Depend on W, right. So, that is the problem with an ordered network. In such an ordered network you cannot compute the gradient of a s_4 with respect to W assuming that s_3 is a constant, s_3 is not a constant its again a function of W and W is the parameter with respect to a computing the derivative, right. That is the problem here.

So, in such networks the total derivative has two parts, what are these two parts? Okay, how many if you have revise this? What are the two parts called? Explicit and I mean at least your language model should be fine at. Explicit, and what else can it be? think on at least have that much smartness, either you do not read its fine. So, that is going to be explicit and implicit. What do we do in the explicit case? If you can read the slide, we treat all the other inputs as constant, right. An implicit is summing over all the indirect paths from s_4 to W. So, let us actually try to derive this whole thing, right.

(Refer Slide Time: 09:38)



So, this is what the total derivative looks like. All of you are comfortable with this, right. I mean this is all we have done this in the assignments I will not go into the theory and

all that. You should be comfortable if you have not revised, you have to be blamed sorry for that, but I cannot go into the details of that but I still derive the whole thing.

So, this is what it looks like. The plus here indicates that we are going to treat everything else as a constant and just take the derivative with respect to W. And then the implicit part would be this, we are going to sum across all the paths. So, this is a path, ok.

Now, here again we have a total derivative $\partial s_3 / \partial W$. So, what am I going to do for that? Again explicit and implicit; again I have this ∂s_2 by 2 by ∂W which is again explicit plus implicit, again $\partial s_1 / \partial W$ is that fine and then this is finite because s 1 does depends on s 0 which has no connection to W. So, this is what your entire formula looks like. Now, this, sum slide abuse of notation here because what is each of these actually? Scalar? Vector? Matrix?

Student: (Refer Time: 10:41).

s_4 is?

Student: s_4 is vector.

Vector. W is?

Student: Matrix.

Matrix. The derivative of a vector with respect to a matrix is?

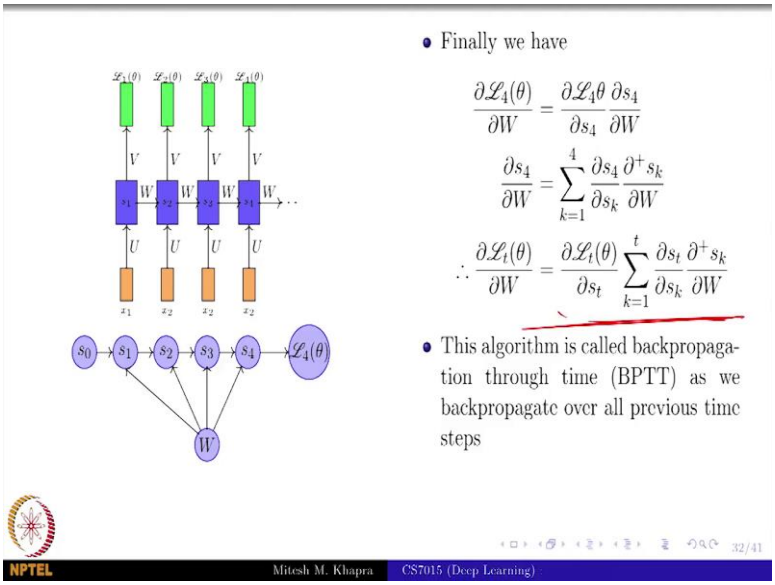
Student: Tensor.

Tensor. You cannot do this in your head is it; these 3 sentences is one after the other, ok. So, for simplicity what I am going to do is I am going to short circuit some of these paths, right. So, let us I will just tell you what I am going to short circuit. So, I am going to write just for ease of coming up with the generic formula. The first term I am going to write as this. And this is fair because this is just one, right. The second term also is fine. The third term I am going to short circuit this path I am just going to write as $\partial s_4 / \partial s_2$ and then $\partial s_2 / \partial W$, and again I am going to short circuit these paths and just write it as $\partial s_4 / \partial s_1$ and then this.

The reason I am doing this then I can write it as a very simple summation, where I have s_4 by s_k, where k goes from 1, 2, 3, 4 and then I just have the explicit derivative of s_k with respect to W. Just stare at this for the minute and not a minute actually just 10 second or something. If you have any problems with this let me know, I will use my standard trick. If you do not understand this you will not understand anything afterwards, no one is falling for that, ok. Everyone is comfortable with this? Ok.

So, we have a formula for $\partial s_4 / \partial W$, and we have dealt with the tricky situation where we have these multiple paths in an ordered network and hence we are to split into explicit and implicit derivatives. So, we have done all that (Refer Time: 12:20) math and you have come up with the simplified formula for this, ok. So, finally, this is what we have.

(Refer Slide Time: 12:23)



You noting it down, right?

Student: (Refer Time: 12:28).

[laugher] I do not see you noting it down, ok. So, now, let us look at $\partial s_4 / \partial W$, that is exactly what we have derived on the previous slide and that was a summation of t terms, and for us t is equal to 4, ok. And in general L_t by, the this was for L_4 so in general if I want to do L_t then it is going to be this which I am replaced by t, and this which I have replaced by this formula. Everyone is fine with this? What were this means? Everyone is

fine with this formula, right. This is generic formula with respect to any time step. The only thing is that on the previous slide we are derived with respect to s_4, now I have just come up with the generic formula, ok.

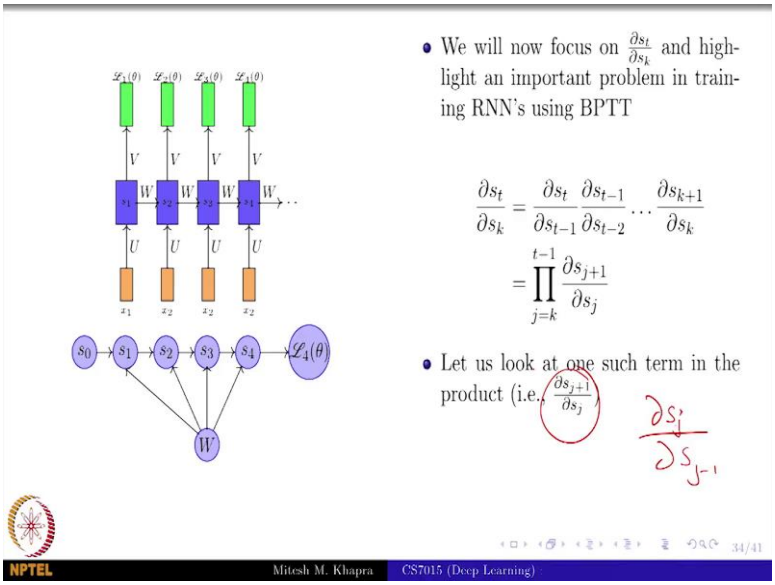
So, this algorithm is called backpropagation through time because now we have taken care of this ordered network and you have a way of computing this gradient, once you have this gradient your life is simple because now we can just supply the gradient descent update, ok. So, we have dealt with V, we have dealt with W and as the name suggest who will deal with U, you ok, fine. So, you will to find out what it is for U, ok. By its going to be something very similar, and I do not want to do it because that is not I mean going to be something very similar you can do it on your own. But I want to focus on something which is important.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 105
The problem of Exploding and Vanishing Gradients

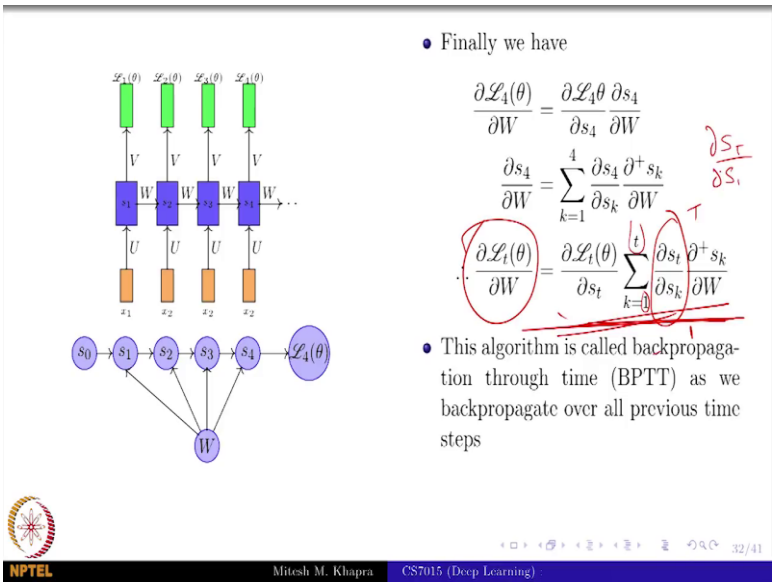
And that takes us to the Problem of Vanishing and Exploding Gradients ok. So, you want to see what is a problem with this back propagation through time, which could lead to certain interesting situations.

(Refer Slide Time: 00:24)



So, we will focus on this $\partial s_t / \partial s_k$ and let me just go back. So, remember that this formula had this $\partial s_t / \partial s_k$, right, where s_t could be the last time step and s_k could also be the first time step because you are summing over all the time steps, right.

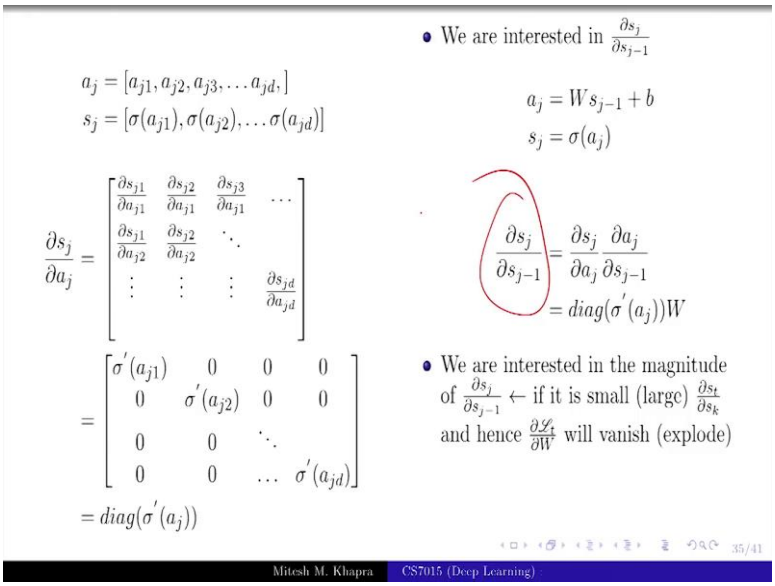
(Refer Slide Time: 00:27)



So, you could have a term which is s_T capital T which is the last time step the first time step and the derivative of the last time step with respect to the first time step, right. So, that is a situation that we are dealing with. So, we will consider one such generic element which is $\partial s_t / \partial s_k$ and we will just try to expand it. So, remember I have done this short circuiting, so I am now just going to expand it again. So, this is going to be t / t - 1, t 1/ t-2 and so on up to k+ 1/ s_k. Ok and I can write it as this generic formula. Everyone find with this, I have just replace this as a product and written it more compactly.

Now, let us look at one such term here $\partial s_j / \partial s_{j+1}$. Now, just to confuse you guys from next slide I will go over to $\partial s_j / \partial s_{j-1}$ or not confuse you I just did not pay attention to this. So, instead of s plus 1 and j and I am going to do j and j minus 1, right, it remains the same does not matter.

(Refer Slide Time: 01:33)



So, we are interested in this particular quantity. So, let us see what this derivative is. And remember that in the final formula we have a product of these quantities. So, I am looking at one such term in my final product. So, just to jog a memory a j is the pre-activation which is given by this and then s j is the hidden representation after activation after the nonlinearity which is given by. So, let me just write it down as s j by s j minus 1 can be written as this chain rule which is first compute s j with respect to a j and then a j with respect to s j minus 1. Everyone is fine that so far at this point? Please raise your hands if you find ok.

Now, let me just write down a j and s j explicitly. So, remember that a j is this d dimensional vector which are the entries a j 1, a j 2, up to a j d and s j is the corresponding activation applied vector which has these entries sigma a j 1, a j 2 and so on ok. Now, first question. What is this quantity? Scalar? Vector? Matrix? Tensor? Numerator is a.

Student: (Refer Time: 02:44).

Denomitor is a.

Student: (Refer Time: 25:45).

That is why it is a matrix ok. So, that is the matrix that I am interested in. If I can give you that matrix and we are kind of done so, it help me filling in this matrix.

Tell me what this matrix is going to look like even before we start filling it ok. You are right, but it does not matter because you will have U x and then you are taking the derivative with respect to s j minus 1, right so, this does not matter ok. So, everyone gets that? You will have a U x j here, right but that does not matter because you are taking a derivative with respect to s j so, that is a constant.

So, $\partial s_j / \partial a_j$ is what? What does this matrix look like? How many of you see a diagonal matrix? Ok good. So, it is straightforward, right. What is the first entry it is going to be $\partial s_{j1} / \partial a_{j1}$, what is that going to be? It will be something. But let us look at the second entry $\partial s_{j1} / \partial a_{j1}$ what is this going to be? What this going to be?

Student: 0.

0, because it does not depend on that, right. So, now, you can see how the full matrix will look like all the of-diagonal elements are going to be 0s and diagonal elements are going to be sigma primes. Everyone fine with this? Ok. So, this matrix I am going to just call it as diagonal sigma prime a j. This is a diagonal matrix which I have. And what is dou a j by dou s j minus 1? Scalar? Vector? Matrix? Scalar.

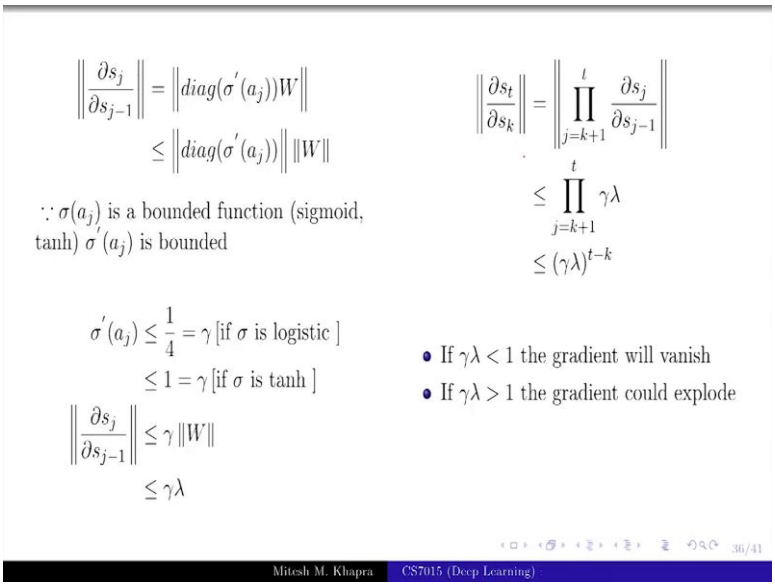
Student: (Refer Time: 04:17).

Matrix. Which matrix?

Student: W (Refer Time: 04:18).

W, right ok. So, now, for some reason I am interested in the magnitude of this. Why I am interested in the magnitude of this? For some reason I am interested, let us see. Why we will become clear that for some reason I am interested in.

(Refer Slide Time: 04:40)



And here I will write how I will write the magnitude of this, right. So, this is the norm that I am interested in. So, I have already said that this is actually equal to whatever is inside this norm. So, I can just write it as this norm so, I have norm of c is equal to norm of a b which is less than equal to.

Student: Norm a, norm b.

Norm a norm b. Ok? This is fine ok. Now, let us look at the norm of this. Now, going to say that sigma a j is actually a bounded function because, we are using sigmoid or tan h or something so, it is a bounded function ok. So, that mean sigma dash a j is also going to be bounded. Actually, can you tell me what is the bound for the logistic function for sigma dash a j.

If sigma is logistic function what sigma dash what is the bound for sigma dash. If I say 1 by 4 how many of you will agree with that? How many of you have a problem with that? If you do not understand this you not understand anything after that ok, still do not have a problem. So, for the logistic function the bound is actually 1/4, the maximum derivative that you can get if you have this curve so, then that would be 1/4, ok.

What about the tan h function? And that actually happens at this point, right 0.5, so 0.5 into 0.5 is 1/4. What about the tan h function? The bound is 1, right. So, this is, this clearly an upper bond on these things the derivative is going to be an upper bounded

thing that means, this magnitude is actually going to be upper bounded by something and I will just call it as lambda sorry as gamma. So, this quantity is bounded and I am going to call that bound as gamma.

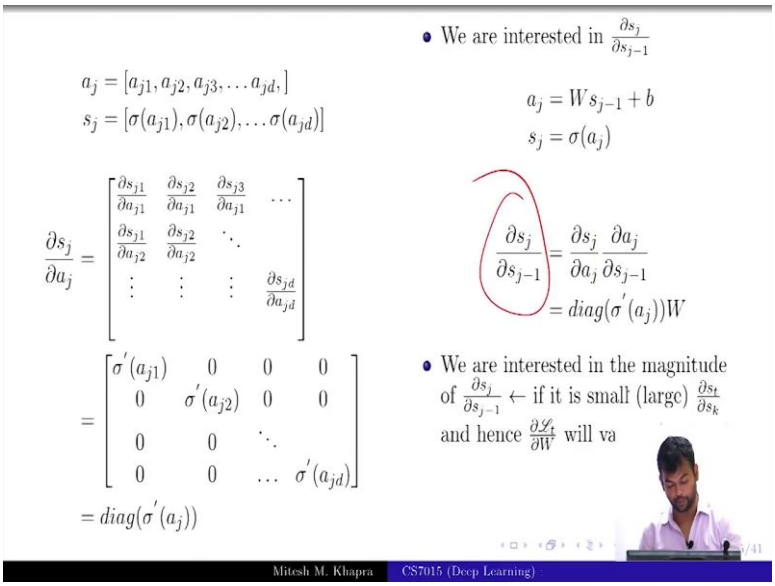
What about our weight matrix? It is again bounded, right we have real weights we do not have like blowing we do not have very large weights it is all bounded. So, it is still going to be some upper bound on this and I will call this magnitude as gamma, right. So, this quantity on the left hand side, I can say that it is less than equal to some gamma into lambda.

Now, let us look at the product. So, this is a quantity that I was interested in and this is actually a product of various such quantities. So, what is it going to be now? Can you go to the next step? It will be gamma into lambda raise to t minus t minus k, right, t minus. It basically as t minus this product as t minus k terms, right. So, it will be gamma lambda raise to t minus k. Now, if gamma or lambda, or rather gamma into lambda if it is greater than 1, what will happen? What will happen to the series? Explore. If it is less than 1?

Student: (Refer Time: 07:20).

It will vanish, right so, you get that. So, that is why you have this vanishing an exploding gradients problem ok. But why what if this vanishes what vanishes? Let us go back. So, I have shown you that this quantity could vanish right if this vanishes the entire gradient could vanish. And if the gradient vanishes what would happen?

(Refer Slide Time: 07:39)

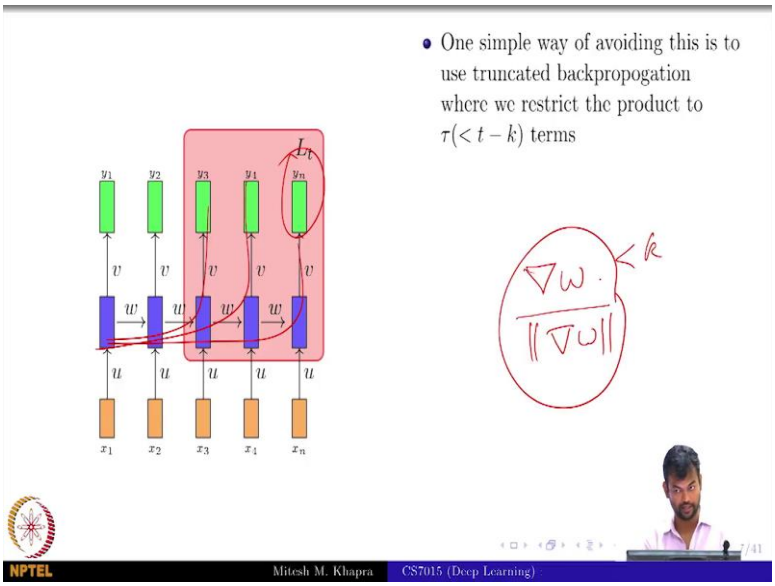


Student: No updates.

No updates and you just stuck where you are. If the gradient explodes what happens? Think in terms of the WB plane, you suddenly have a very large gradient what will happen is just gone way far from where you are right. Now, because your update is W is equal to W minus eta into this gradient and this you have got a very large value now.

It just going to move somewhere very far from where you are and that is never go where your suddenly jump to a different universe ok. So, that is the problem in training recurring neural networks. You could have this problem of exploding or vanishing gradients, and we have done a mathematical derivation of why you have this problem, ok.

(Refer Slide Time: 08:16)



So, one trick to do that is to avoid this is remember these are t minus k terms and the problem appears when your t minus k is or rather you are t is close to capital T, and k is closed to 1, right. In those cases you will have many terms in the product you will have as many as T terms in the product, so even if your product is even if this product is slightly less than 1, if you raise it to capital T it is going to vanish, right. So, can you think of solution for this?

And the last module in the title of this lecture was truncated back propagation. Can you think of a solution for this? So, you do not back propagate through all the time steps. Yes use an approximation that if you are at time step n. We are just going to look at n minus k time steps and we are not going to look all the way back, right. That is the common trick used to avoid exploding and vanishing gradients.

What is the other thing that you could do to avoid exploding gradients? So, remember that you have some gradient, right. To think in terms of vectors we have some gradient vector W whose magnitude is very large. What will you do to avoid exploding gradients? In gradient descent your always interested in the direction so, what can I do?

Student: (Refer Time: 09:35).

Just normalize it, right. So, you can just do this so, typically what is done is that you can it is a normalizing it you can just say that you will clip the gradient so that it is

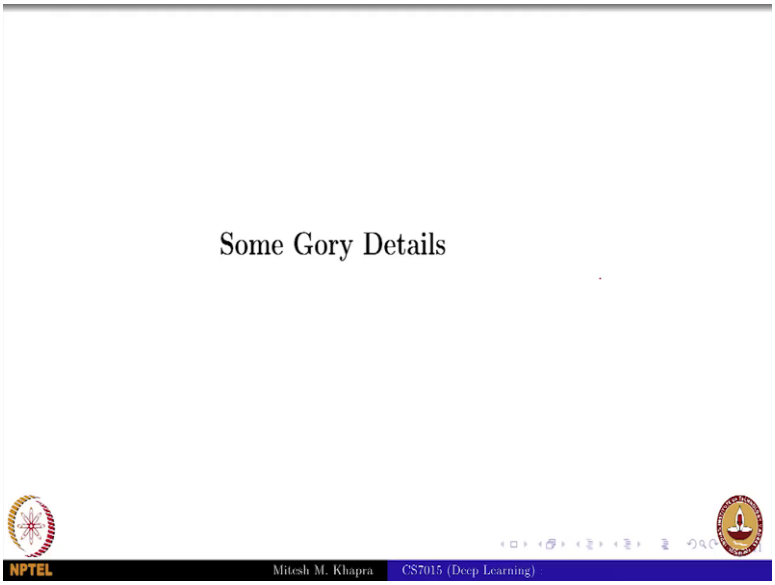
magnitude is less than a certain k, right. So, normalize it in such a way that it is grade it is magnitude becomes k. So, this is something typical that you will see when you use tensor flow where you have something with says clip the gradients to a certain magnitude. And there are different ways of doing this, so I just give you an intuition that this is what is used for magnitude but there are other things that you can use for magnitude. So, just go back and look at that ok.

So, that is a back propagation through time with exploding and vanishing gradients and then the solution for that or a part for that is truncated back propagation ok. We have we have not yet done with this problem, we will again look at other solutions for handling this which will lead us to LSTMs which is Long Short Term Memory cells and gated recurrent units, so that we will do in the next lecture.

Deep Learning
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

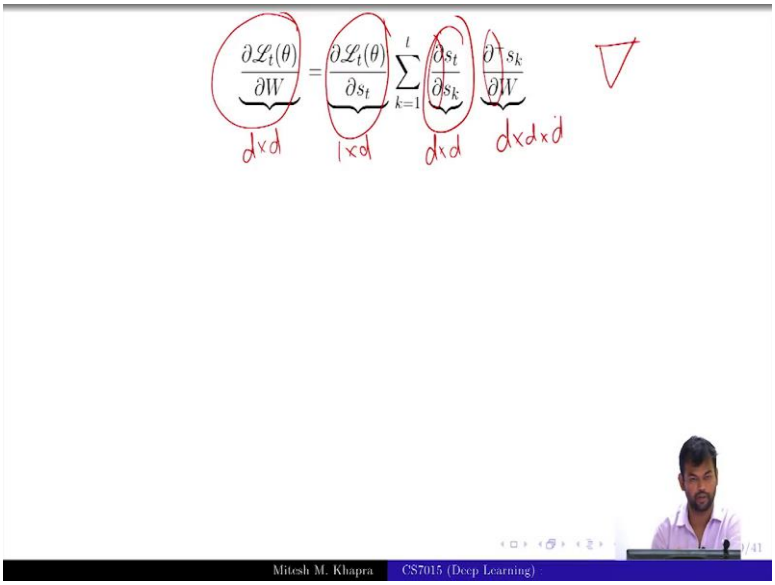
Lecture – 107
Some Gory Details

(Refer Slide Time: 00:11)



Before that I will just go into Some more Gory Details about the math, this is not to scare, but just to make you more comfortable that we can actually deal with something, which is not very straightforward or very neat as compared to what you are seen so far.

(Refer Slide Time: 00:25)



So, I just go back to the formula, which I had for computing the gradient of the loss function with respect to W, and I cannot repeat enough times that, all these notations are actually a bit of abuse of notation, because these are gradients and not partial derivatives. So, I should actually be using this notation, but for ease of explanation, I use I stick to my original notations ok.

So now, let us look at each of these quantities here and tell me the dimensions of these quantities. Let us start with the left hand side, what is the dimension of this? W was what matrix? What is I am talking about the circle entity, what is the magnitude, what is the dimension of that? K × d.

Student: (Refer Time: 01:09).

d × d.

Student: (Refer Time: 01:11).

d × d. someone n × d.

Student: (Refer Time: 01:13).

n × d and n × k are the two options, which are left. W is the recurrent weight. So, W is what dimension?

Student: (Refer Time: 01:20).

d × d.

So, what is this gradient, d × d? Ok what about this? Fast. S_t what the hidden representation, so that was d dimensional. So, what is this d ×1? Ok what about this? Why do you guys still struggle with this?

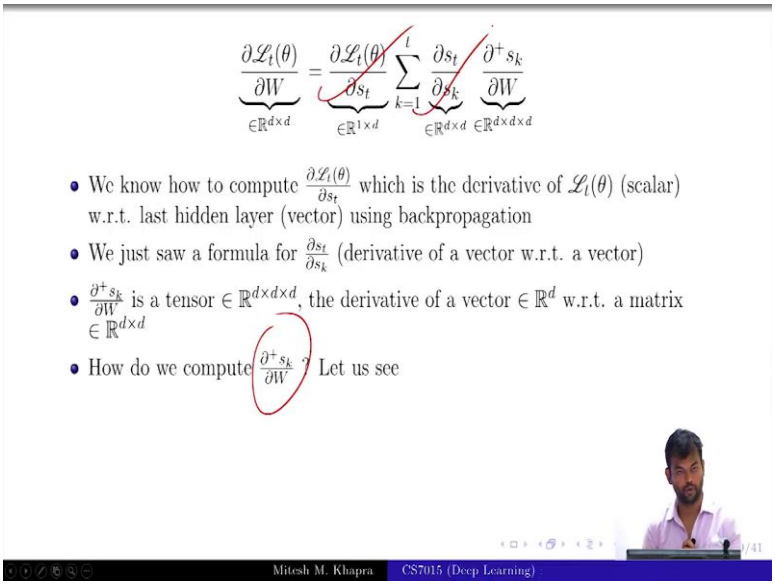
Student: (Refer Time: 01:42).

d × d and this d cross,

Student: (Refer Time: 01:44).

d cross it is very straightforward, right? What is the dimension of numerator? what is the dimension of denominator thats all right.

(Refer Slide Time: 01:53)



So, you see the kind of multiplication that you are doing here. Say of d cross, d1×d, d × d and then d ×d × d ok. Let us look at each of these quantities and see if you are actually comfortable in implementing these. Are you comfortable with this? The loss function with respect to the hidden representation, we have done this enough times in back propagation, what about this? We just saw a formula for this right. So, we know how to compute this quantity, we have seen this in back propagation, this is the derivative of a scalar with respect to a vector and we are very comfortable in computing this.

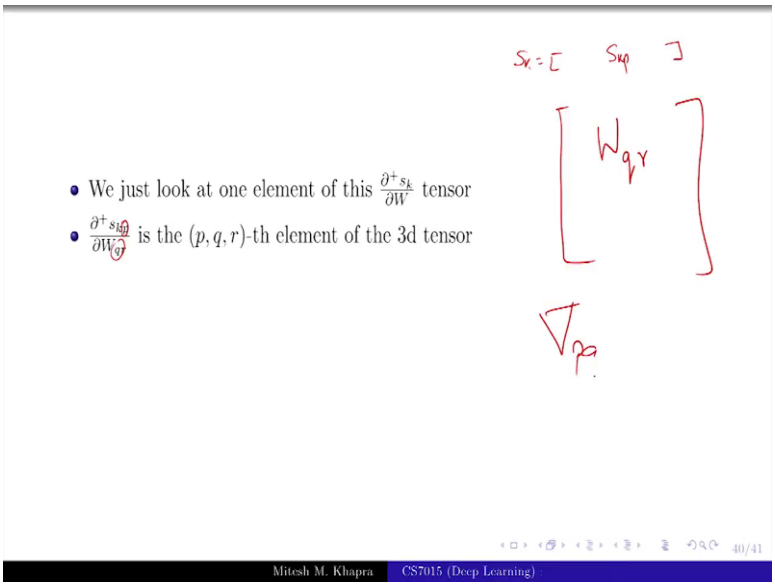
This was slightly tricky, but we just derive this formula on the previous slides, everyone okay with that, what about this? This is a tensor, how do we compute this tensor? what is our standard recipe focus on?

Student: (Refer Time: 02:38).

The little guy, 1 element of this tensor and then you can generalize somewhere right. So, this is the tensor and we will just see that, this just to make you all comfortable is this not like just to intimated you with all these large sized tensors, but I am just trying to show that, this is all easy, this is not hard ok. So how do we compute this? All the other terms are covered.

This is the only one, that we do not know.

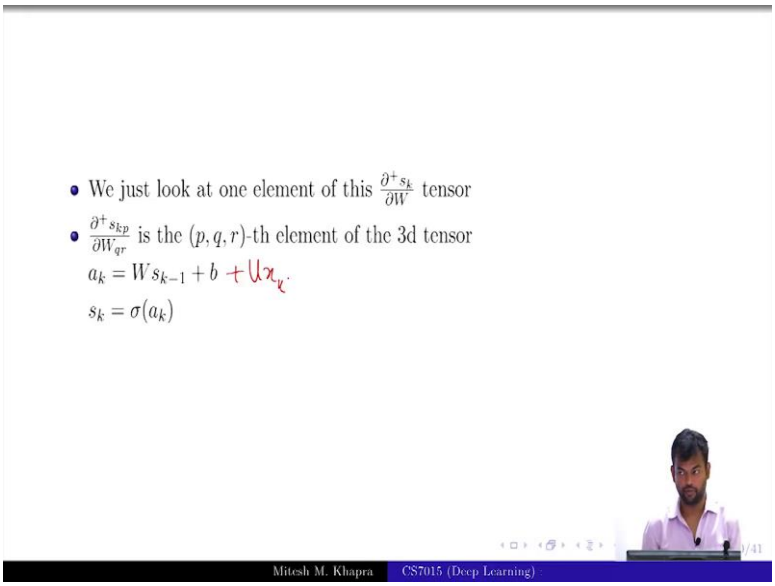
(Refer Slide Time: 03:03)



So, we will just look at 1 element of this tensor and it is going to be $S_{kp}/W_{q\,r}$.

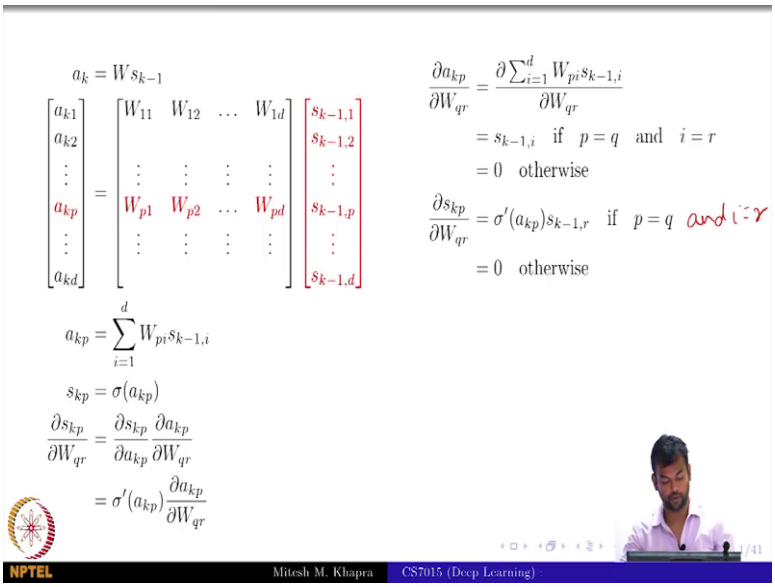
So, let us just see that. You have S k as this vector and you have W as this matrix. So, I am considering one such weight, which is, W p comma q and one such element from here, which is S k sorry. So, q r and I am considering one element from this, which is S k p. So, I am trying to compute the derivative of one element of the vector with respect to one element of the matrix. So, this is going to give me one entry, in my tensor and that entry is going to be what? P q r, how many of you are fine with this? Ok fine.

(Refer Slide Time: 03:53)



So, now recall that a k was equal to W into S k minus 1 plus b and S k was sigmoid of a k. I think again, I have miss that U into x k, but that will not matter because, that is not there in the derivative ok. You are fine with so far?

(Refer Slide Time: 04:11)



So now, let us look at this, because the other 2 terms do not matter. So, I just look at a k is equal to W into S k minus 1. So, this is the matrix way of writing it. Ok. Now I am looking at one of these elements which actually comes, from the multiplication of a row and a column, the highlighted row and the column. Everyone gets this? Ok.

Now so, I can write it as a k p is actually equal to this summation, which is nothing, but the dot product of this row with this column ok. Now S k p is just the sigmoid of that. So now, if I want to compute S k p with respect to W q r, I can just write the chain rule that S k p with respect to a k p which is straightforward and then a k p with respect to W q r and I already have a formula for a k p, how many of you are fine so far? Please raise your hands high up, if you are fine ok. So, what is the first term going to be? Sigma.

Student: (Refer Time: 05:10).

Sigma prime of a k p and what is the second term going to be? This is what, the second term is. Now, what this is lot of terms here, which of these terms would actually remain? Only the once where, only the terms where i is equal to.

Student: (Refer Time: 05:28).

R and.

Student: (Refer Time: 05:30).

P is equal to q. So, only that term will remain. In that case, it would be this right and in the other cases going to be 0 right. So, now, you have one element of this tensor and you have it as a very generic formula, you can just fill in all the elements of the tensor right. So, what does this tensor look like? It is a very.

Student: (Refer Time: 05:52).

Sparse tensor right that is all I wanted to convey ok. So, this is again the same thing right, is that fine? So, even though it is a nasty looking tensor, if we just break it down to one element, it is going to be very easy and now from this element, you can just reconstruct the entire tensor. Do not worry, I am not going to ask you to implement this, but if someone were to, maybe at some point, then you should be able to do it right, that is where we will end today.

So, we have finished recurrent neural networks and the next thing that, we are going to look at is LSTMs and gated recurrent (Refer Time: 06:25) ok.

Thank you.

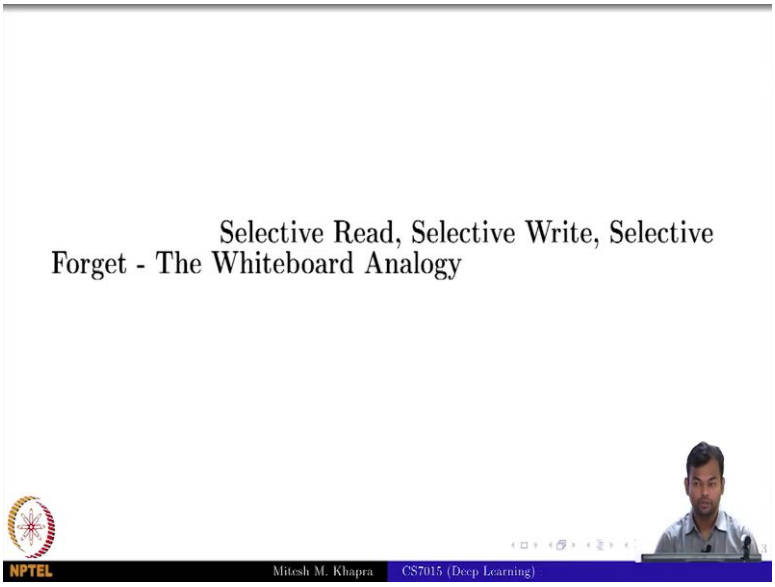
Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 14
Long Short Term Memory Cells (LSTMs), Gated Recurrent Units (GRUs)

So, we have been looking at a new kind of or a different kind of neural network which is recurrent neural network. And last class we spent some time on showing that it is how to train these recurrent neural networks because of the specific problem of exploding and vanishing gradients. In particular we saw that if you want to propose, if you want to back propagate the gradient from time step t, the final time step to an arbitrary time step k then you have this multiplicative term in the back propagation which could explode or vanish.

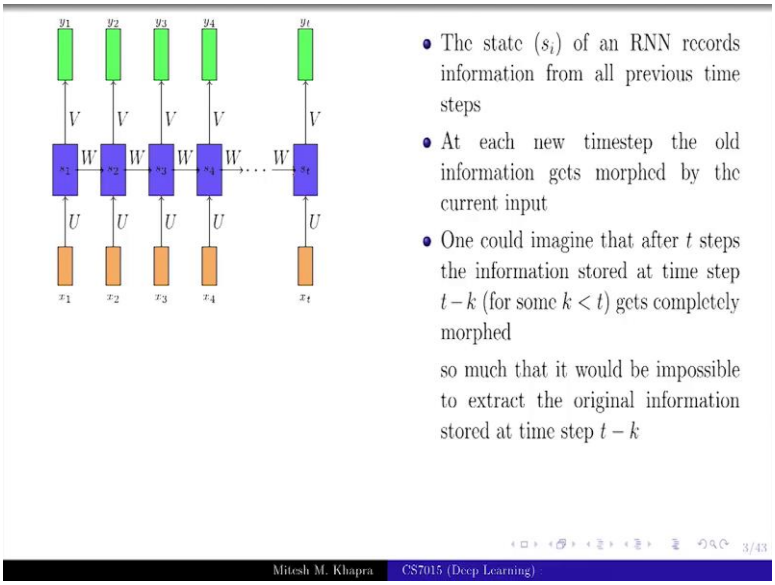
So, today we are going to try to see if we are trying to focus on something which can help us solve this problem to whatever extent, right. So, that is why we will look at LSTMs, a Long Short Term Memory Cells and Gated Recurrent Units, ok. So, let us start with that.

(Refer Slide Time: 01:02)



So, first we will introduce the idea of selective read, selective write and selective forget and then we will try to build on this intuition and see how could you could realise it by using LSTMs and gated recurrent units and whether that help in solving the vanishing and exploding gradient problem.

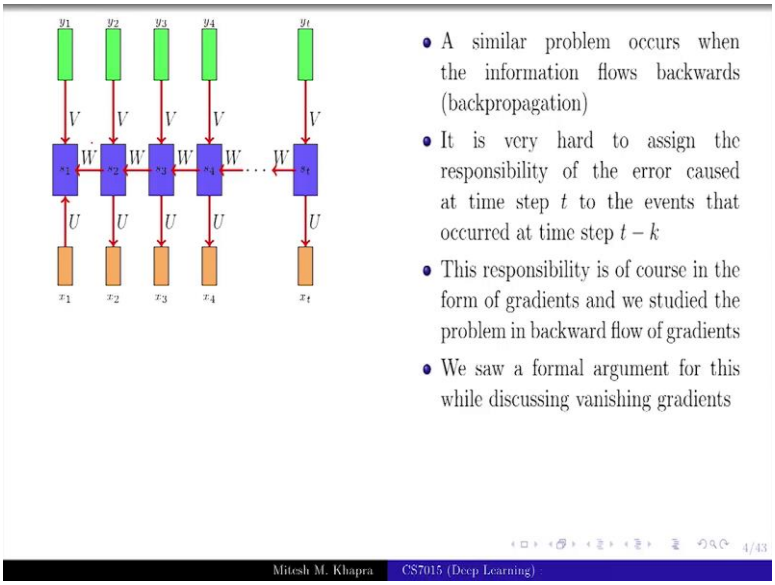
(Refer Slide Time: 01:18)



So, recording the state of an RNN records information from all the previous time steps that was the whole idea that you add this recurrent connections. So, as you keep going on at this time step the cell is not only recording information from the current time step but it also has some kind of an accumulated history from all the previous time steps, right. But now the issue is that this state or this blue coloured vector that you see is going to be of some finite size, right we will say that, it is a 100 dimensional vector or a 1000 dimensional vector but whatever be the size it is going to be some finite dimension.

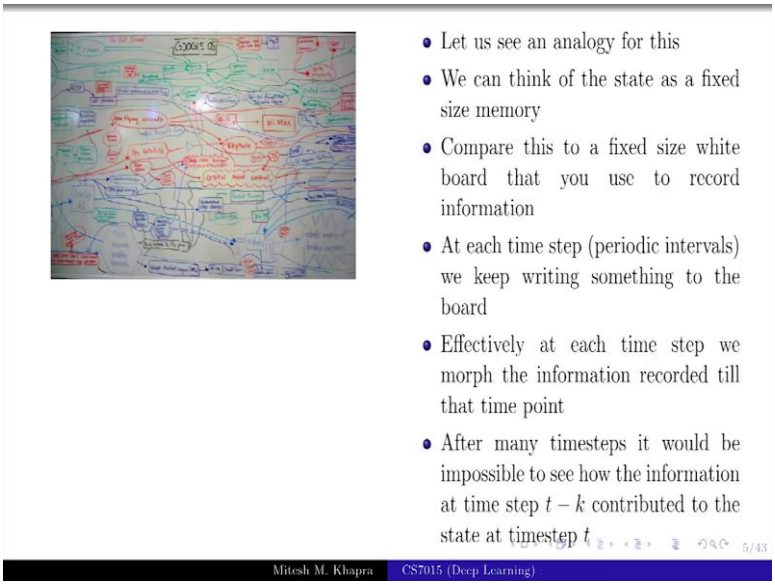
Now, as you keep writing information to that cell you are morphing the information that you had written at the earlier time steps, right do you get that. So, now, that is the problem, right. Because on one hand you are saying that you want to record the information from all the previous time steps, but the at the same you at the other hand you just have a finite amount of memory to deal with. So, it is bound to read over ridden and the information will get morphed so much that it is completely impossible to say what was the original contribution at time step 1 or time step 2 once you have reach some time step 20, 30 or so on, right. So, that is the problem with recurrent neural networks. And we will tie it back to the problem that we had with the vanishing and exploding gradients, right.

(Refer Slide Time: 02:33)



So, in fact, the similar problem occurs when you try to when the information flows backwards during back propagation, right. It is very hard to assign the responsibility of the error caused at time step t to arbitrary time steps before it, right to very far away time steps it is very hard, and that is the vanishing gradient problem, right. Because we have this multiplicative term and the gradients vanish, so that that is very hard to do. So, both during forward propagation the information vanishes and even during backward propagation the information vanishes, right. And we saw a formal argument of this while doing vanishing gradients. So, this is just an illustrative diagram, but we also saw that formally the guardians do vanish under certain conditions, right.

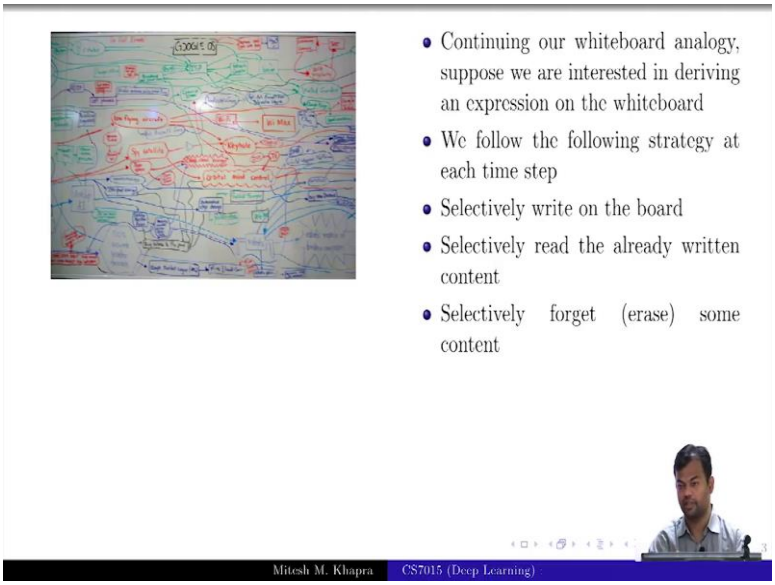
(Refer Slide Time: 03:11)



Now, let us see an analogy for this and from here on we will build on some ideas which will help us arrive at LSTMs and gated recurrent units, right. So, the analogy is a whiteboard. So, you have a whiteboard and it is always of a fixed size, right I mean the whiteboard is not infinite, you just have some size for the whiteboard and you keep writing information on that. So, at every time steps I keep going and writing something on the board and I am trying to derive something or just try to make a story or anything, right. I just keep trying to write information on the board.

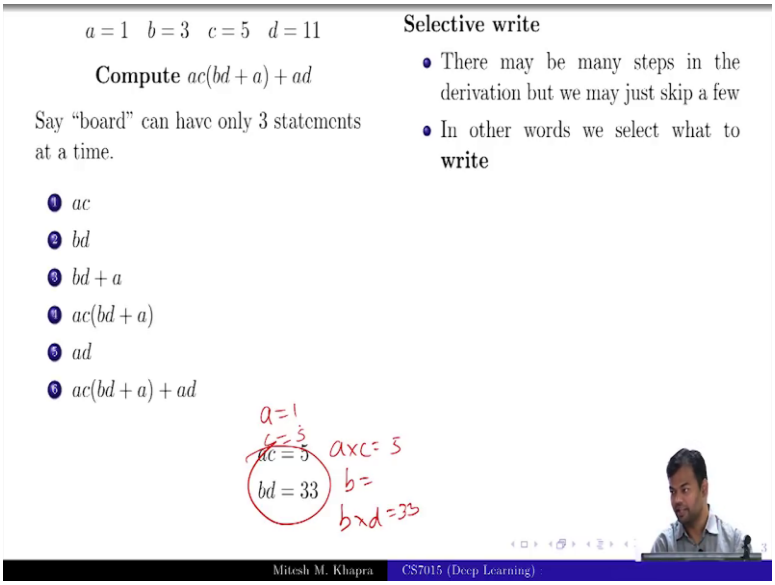
Now, since the whiteboard is fixed size, at every time I am essentially morphing the information which was written at the previous time step and after many time steps it would be impossible to find out that now whatever my state of the board is how did time step one contribute to that state, right. Because, it is written all over the board I do not know where I started what I did and so on and its going to be very hard for me to find and this happens, right. When you do these long derivations on, on the whiteboard it becomes very hard to track, where did I start where was this variable defined and so on, right because it is a fixed size you cannot really I will end of deleting some terms and so on, right.

(Refer Slide Time: 04:17)



So, and we will make this more concrete with the help of an example, right. Suppose I am trying to drive an expression on the board and typically what we do is we use 3 things, we use selectively write on the board, selectively read the already written content and selectively forget or erase some content. So, let us see what I mean by these 3 ideas, ok.

(Refer Slide Time: 04:39)

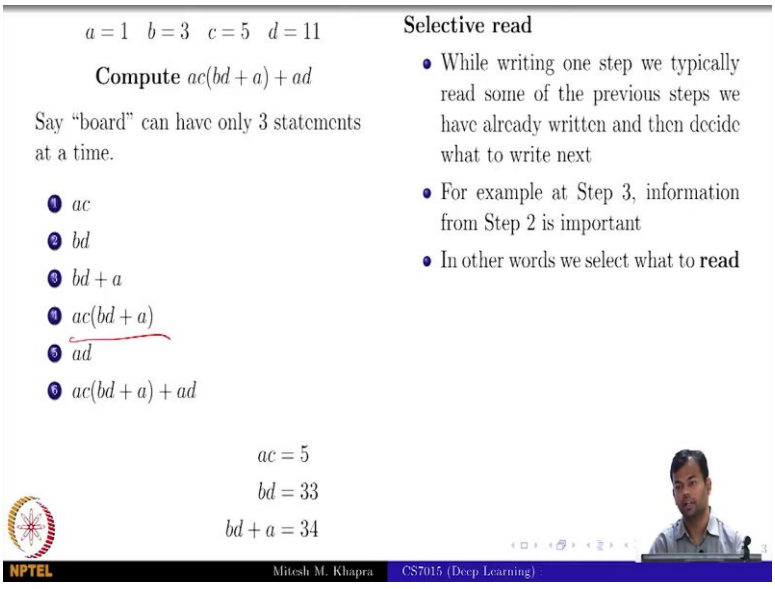


So, first we look at selective write. So, this is my problem this is the derivation that I want to do on the board and I have a very small whiteboard which allows me to write

only 3 steps, ok. That is the situation in which I am operating, ok. So, I am given the values of a b c d and I want to compute this expression ac(bd + a)+ ad, ok. Now, the first thing that I am going to compute is ac, right that make sense because that is the first term that I need. So, I will write ac equal to 5, and the second thing I will write is bd = 33, ok. So, now, why did not I do the following? I could have done this a equal to 1, c equal to 5, then ac = 5, then b is equal to something, then b into d is equal to something.

So, what am I doing here while writing on the board? Why did I write only two steps? Because I know I have a finite size for the whiteboard, right. So, I am only trying to write information which is important, right. I am not writing everything because I know that I am going to run out of memory, right. So, that is why I did not write these intermediate steps that a = to 1, c = 5 and then a c = 5 and so on, right. I juts wrote the results which are important. So, I am selectively writing to the board because I am dealing with a finite size memory, and this is exactly what we do while writing in a note book or a whiteboard or anywhere, right. We just do not write everything one because you are lazy but second is also because we do not have enough space, right. So, that is about selectively writing.

(Refer Slide Time: 06:02)

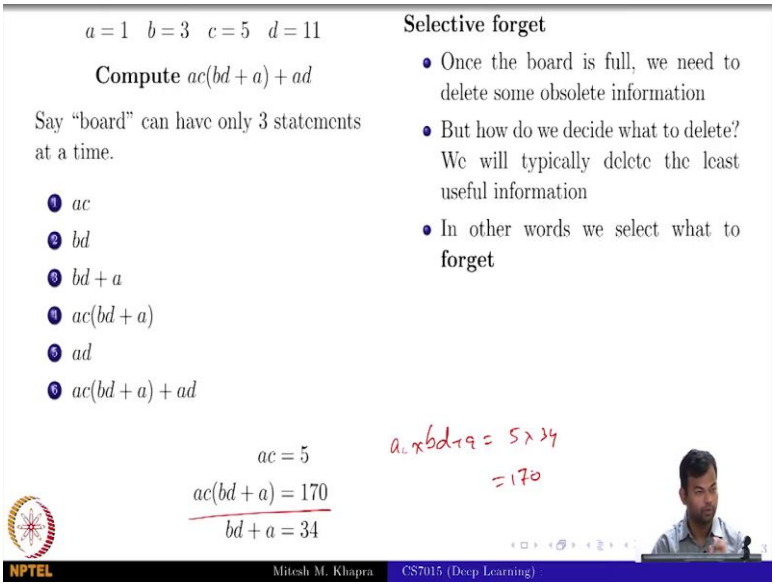


Now, selectively reading I already have something stored on the whiteboard. So, this is the state of the whiteboard at this point. Now, at the next time step I need to read some information, but I do not need to read everything that is on the whiteboard I just need to

read some information. What is the information that I need to read for the next time step? bd, I do not need to read ac, right. So, I am just doing a selective read of the information or the state which is already stored on the whiteboard.

Now, what is happed is I have exhausted my space where I had just 3 steps that could be written on the whiteboard and I have done that. Now, what do I do for the next thing? Now, I need to compute ac (bd + a). I will have to selectively erase, so what will I erase? bd, right.

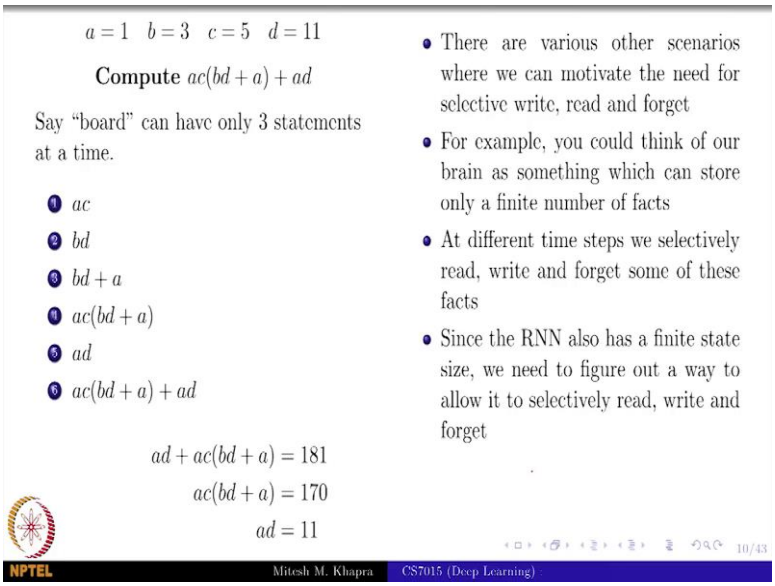
(Refer Slide Time: 06:44)



So now as the whiteboard is full and I will have to selectively delete some information and as this obvious in this trivial example that you can get rid of bd because you have already encoded the information in bd + a. And now the next step which is ac into bd plus a can do on the whiteboard and this is how you keep doing at every time step. You selectively write at every stage.

Here again note that I am not written ac(bd + a) would be something like I do not know what was it 5 x 34 = 170, right. I am not using two steps I am just writing everything in a single step because I do not have enough space, right. So, selectively writing, selectively reading, and selectively forgetting things which are there in a constant memory is something that we do regular, right.

(Refer Slide Time: 07:30)



And then other ways of motivating this; so you could even think of our brain as something which can store only a finite number of facts, right as we keep going or if we will learning more and more things we can only retain a finite number of facts.

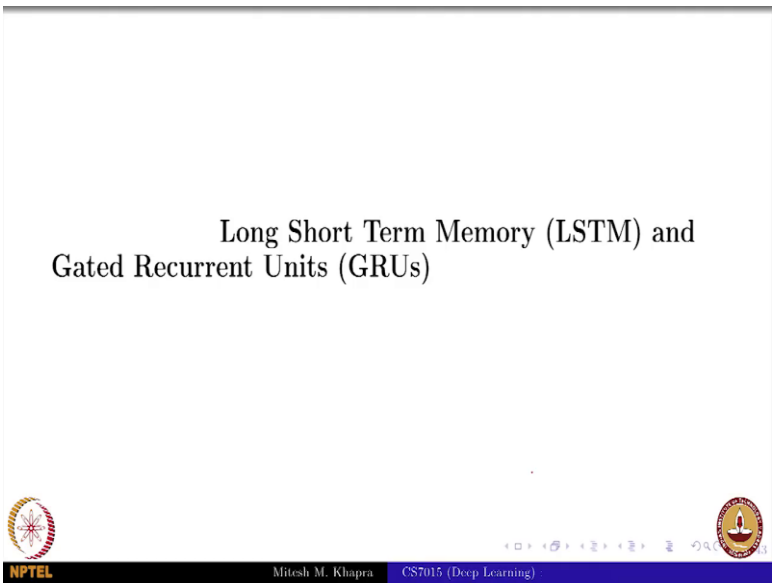
And what happens inadvertently is that you erase some of the steps not consciously of course, you do not have a delete button or anything but you erase some of these things that you forget a lot of things which had happened a year back or so. And also at various times if I ask you what was this which I have done in last class most of you forget to do the selective read but that is what you do, right you always do selective forget but that is what we typically do in our brain also, right. Any time when you are dealing the finite size memory you will always have this 3 operations either they are explicit or implicit, but the intuition is that you end up doing this, ok.

So, now, since the RNN also has a finite state size can we do something like this selective read, write and forget, so that one during forward pass even if the information gets morphed, it gets morphed in a principle manner, right. So, even in the whiteboard example I was morphing the information, I was deleting the information written at time step 1, time step 2, but I was being a bit smart about that. I was retaining some good information and only deleting what was not required. So, can we do this in analogy? So, this analogy really sets it up but now the solution is not going to live up to the expectation, but it will have some it will be something in this direction, ok.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

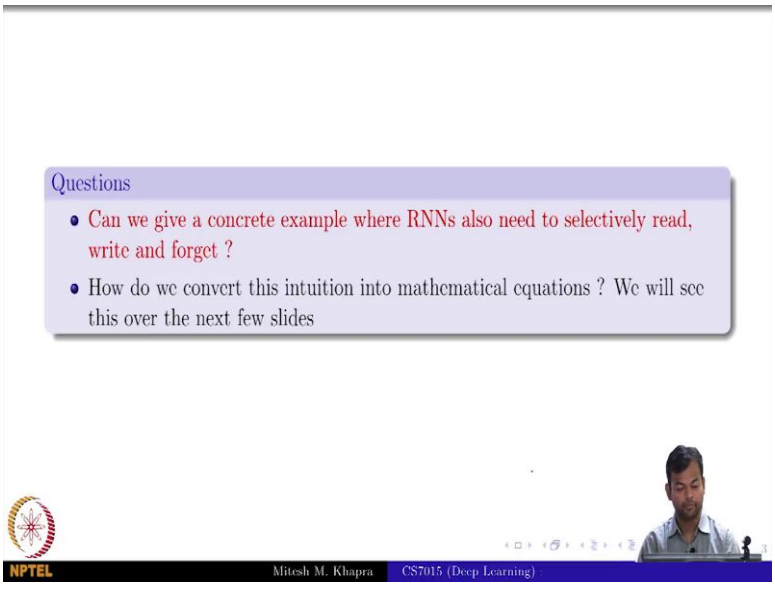
Lecture – 109
Long Short Term Memory (LSTM) and Gated Recurrent Units (GRUs)

(Refer Slide Time: 00:11)



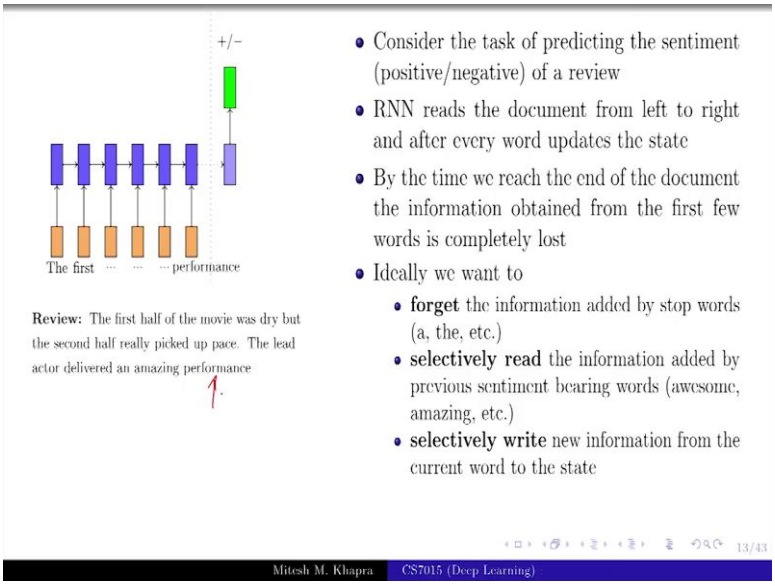
So, with that motivation let us go to the next module, where we will talk about Long Short Term Memory and Gated Recurrent Units ok

(Refer Slide Time: 00:21)



So now, all this was fine in terms of ok. I gave you a derivation on the board and say that, this is not required, but can I give you a concrete example, where RNNs also need to selectively read, write and forget right only then you will be convinced that this kind of morphing is bad in the case of RNNs. So, I will start with that example and then once we agree that, we need selective read, write and forget. How do we convert this into some mathematical equations? Right, because conceptually it is fine, but you have to write some equations. So that the RNN can do some computations, where you have selective read write and forget right. So, that is what we are going to do over the rest of the lecture.

(Refer Slide Time: 00:55)



So, first let me start with the concrete example, where you want to predict the sentiment of a review using an RNN. So, this is the RNN structure, we have done this in the past that you have a sentence, 1 word at a time is your every time step, you will feed this to the RNN and at the last time step, you will make a prediction. And as I said the RNN needs a document from left to right and by the time it reaches the end, the information obtained from the first few words is completely lost right because, it is a long document and you are continuously writing to the same cell state.

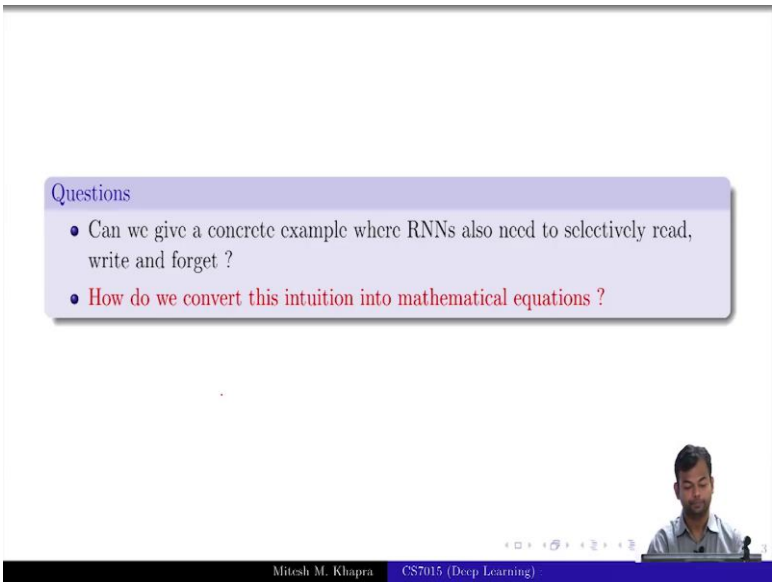
So, you will lose the information that, you had gained at the previous time step. But, ideally we want to do the following, we want to forget the information added by stop

words like a, an, the these do not contribute to the sentiment of the I can ignore these words and still figure out the sentiment of the document.

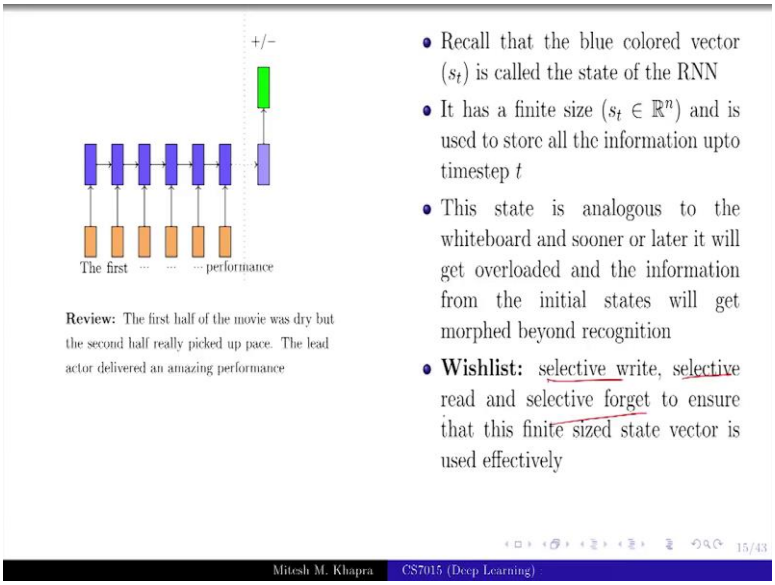
I want to selectively read the information added by previous sentiment bearing words. So, when I have reach the last time step, I should be able to read everything else, which had some sentiments before it and focus on those words just I want to selectively read from these sentiment bearing words and also I want to selectively write the new information. So, I have read the word performance, now I want to selectively write it to the memory, whether I should write it completely or should I only write parts of it or not that is what I need to decide. So, that is fair, this is a typical example, where RNN also when it is dealing with long documents, it needs to understand what is the important information in the document that needs to be retained and then selectively read, write and forget ok.

So, I am spending a lot of time on this analogy, because you need to really understand that this is important and this is where RNN suffer right, if you are using them for very very long documents, if we have document of the size 1000 words, which is not comm, which is not uncommon right because Wikipedia pages have much more than that per document. So, it is going to be very hard to encode the entire document using an RNN not that it is going to become significantly easier with LSTM or GRUS, but to certain extent it will become easier ok

(Refer Slide Time: 02:47)

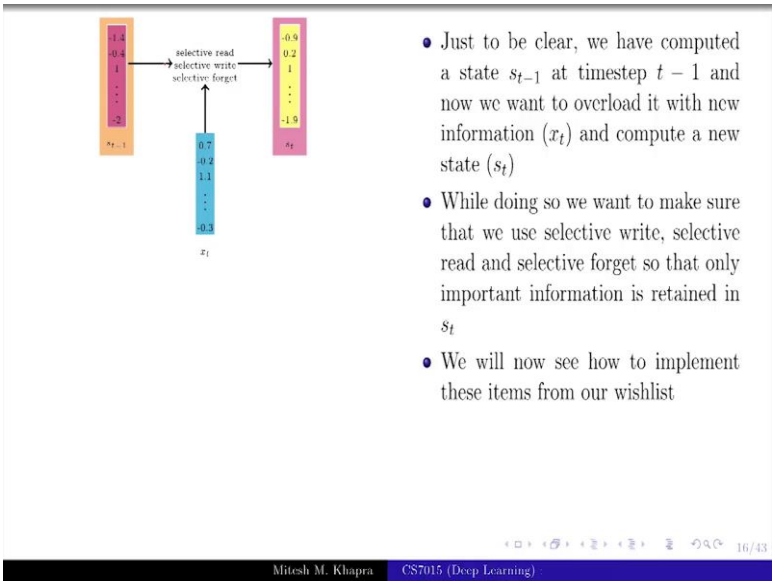


(Refer Slide Time: 02:53)



Now the next part is how do we convert this intuition into some mathematical equations? Right so, let us look at that. So, in this diagram recall that the blue colored vector is called the state of the RNN, it has a finite size, so now I will just call it as s t belongs to some R n and the state is analogous to the whiteboard and sooner or later it will get overloaded with information and we need to take care of this right. So now, our wish list is selectively read write and forget ok. So, let us start with that.

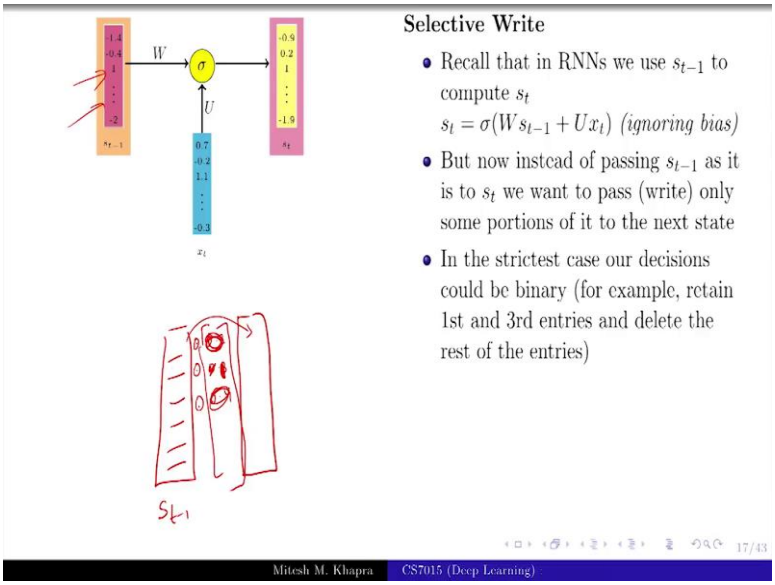
(Refer Slide Time: 03:15)



So, what we want to do is that and this is the problem definition now, that we have computed the state of the RNN, this is a blue colored vector although, it is not blue, but this the blue colored vector from the previous diagram, where the state of the RNN was computed. I know what the state is at time step s t minus 1. Now I want from here to here go from here to here; that means, from s t minus 1. I want to compute the new state of the RNN right. So, I had something written on the whiteboard, I want to write something new, I want to change the state of the whiteboard and this is the new information that has coming to me right the x t is the new information at time step t.

And while doing this I want to make sure that I use selectively write, read and forget. So, these 3 operations have to come in somewhere in the between. So, that I am true or faithful to the analogy, which I have been giving right that is the this is the our problem definition, now going from s t minus 1 to s t and introducing these 3 operations along the way that is what we are interested in doing.

(Refer Slide Time: 04:21)



I will go one by one; we will implement each of these 3 items right. So, we will start with selective write. So, recall that in RNNs this is what happens at every time step t, you take the previous time, step previous cell state, you take the current input. Do you recognize the operation here? How many of you recognize the operation here? Raise your hands ok. So, this is nothing, but the following operation and as usual I have ignored the bias. Ok? Is that fine. So, that is what I am representing it as.

But now so, one way of looking at it is that, when I am computing s_t, I am reading or I am taking the whole of s_{t-1}. So, once I have computed s_{t-1}, I am writing this to my whiteboard and then whole of it would be used to compute the next time step ok, but instead of doing this what I want is, I want to read only selective portions of s t minus 1 or rather I want to write only selective portions of s_{t-1}. Once I have computed s t minus 1, I do not want to write the whole of it, because then the whole of it will be used to compute the next cell state, I do not want that I just want to selectively write some portions of it ok.

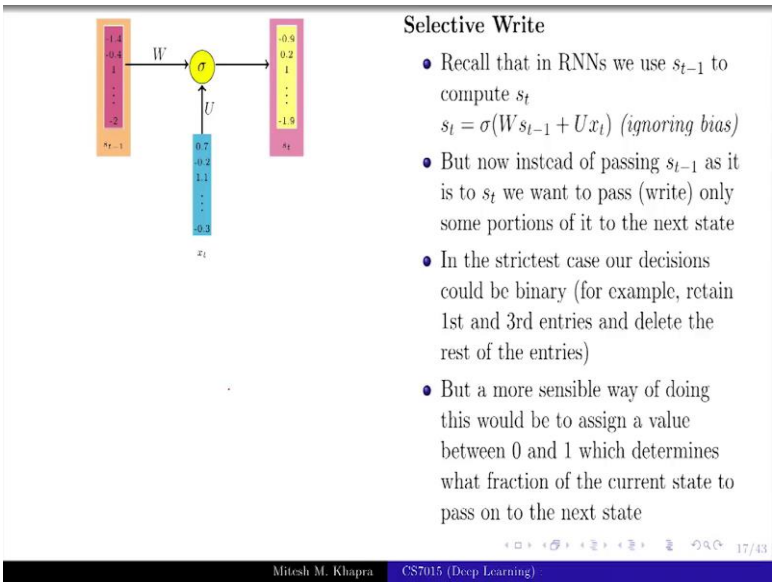
Now, in the strictest case since, I know that s t minus 1 belongs to R n. It is an n dimensional vector in the strictest case, what I could have done is I could have used a binary decision that of all these n entries, I am going to read some entries and ignore the others. So, all the other entries I am going to set to 0 fine, that is the strictest thing, that you could have done. Now for any of these strictest things, what is the soft solution? So, for binary what is the soft solution? Binary 0 to 1. So, what is the soft solution for that? Between?

Student: (Refer Time: 06:05)

0 to 1 so, read some fraction of each of these dimensions right. So, let us try to understand what I am trying to do here ok. So, and the third bullet some of these entries should have gone to 0 right ok.

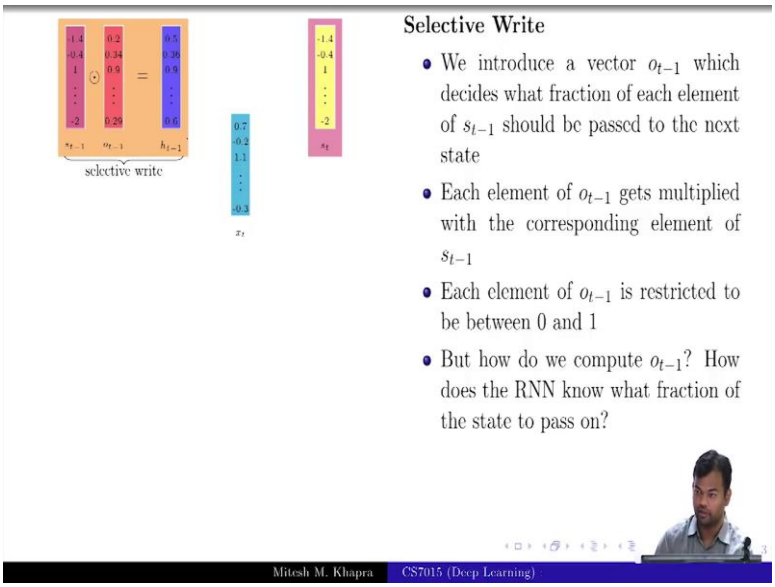
So, instead of doing this, what we want to do is we have this vector, which has n entries this is the cell state at t minus 1. I do not want to write the entire vector onto the final cell state, what I want to do is, I will take some fractions of it is say 0.2 of this 0.3 of this 0.4 of these and then write only that, do you see the operation that I am trying to do right. I want to take some fractions and write only those to the cell and as I said this is the softer version of the hard decision, which would have been 0 for this 1, for this again 0 for this and so on right.

(Refer Slide Time: 07:07)



How to do this? Why to do this? All that is not clear, I am just telling you the intuition, how and why will become clear later is that fine? Ok. So, we want to be able to take s_{t-1} and write only selective portions of it or pass only selective portions of it to s_t.

(Refer Slide Time: 07:23)



So, whenever we compute s_t, we do not want to write the whole of s t minus 1, just want to use selective portions of that. So, what we do is we introduce something known as a gate and. So, this gate is o_{t-1} ok, we take the original cell state s_{t-1}do an element wise product with a gate, which is known as the output gate and then write that product to a

new vector, which is s_{t-1}ok. So, initially this will look confusing, but it will become clear, by the end of this lecture ok. So, is that fine, this is what I am trying to do. Again how to do this is not clear, but this still matches the intuition, which I have been trying to build that I want to write only selective portion of the data, which I already have, is that fine ok. So, each element of o_{t-1} gets multiplied by the corresponding element of s_{t-1} and it decides what fraction is going to be copied. And this o t minus 1 is going to be, between 0 to 1.

But how do I compute o t minus 1, how does the RNN know what fraction of the cell state to get to the next state? How will it do it? We need to learn something whenever you want to learn something, what do we introduce? Everyone,

Student (Refer Time: 08:26)

Parameter sorry, what did you guys say, back propagation. Back propagation will do what? It will work in the air or propagate to what?

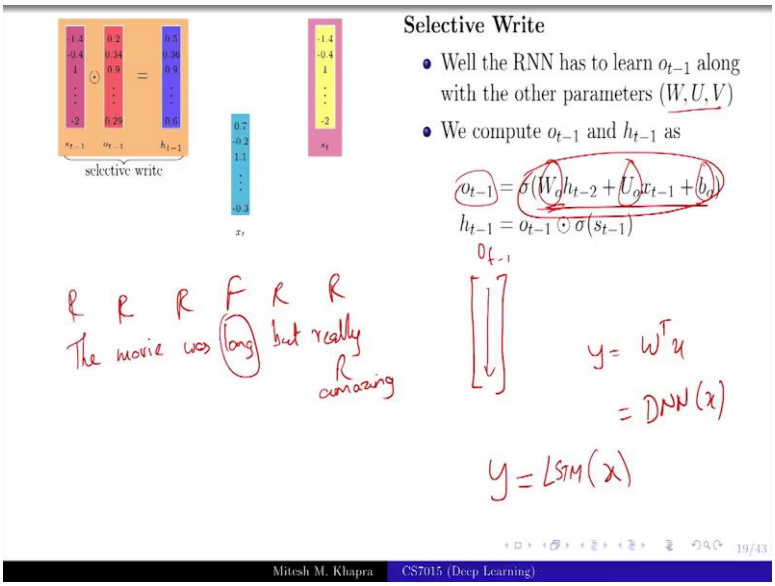
Student: (Refer Time: 08:37).

Whenever you want to do some kind of a learning, I want to learn some function, what do I introduce?

Student: (Refer Time: 08:42) parameter.

Parameter right so, that is what we are going to do, we are now going to introduce a parametric form for o_{t-1} right. And, remember this throughout in machine learning, whenever you want to learn something always introduce a parametric form of that quantity and then learn the parameters of that function, do you get this how many of you get the statement? Ok this is what we have been saying day from right from class 2 or class 3 right.

(Refer Slide Time: 09:13)



Always introduce a parametric function for your input and output and learn the parameters of this function. So, that is exactly, what I am going to do, I am going to say that o_{t-1}, is actually this function. I am just giving some time to digest this. So, this is at time step t - 1. So, it depends on the input at time step t minus 1. It also depends on the output at output means whatever comes out of this right. So, the same operation, what have happened at time step t -2. So, whatever was the output at that state it will also depend on this.

You just take a while to digest, this equation you will see at least 6 more equations of this form in this lecture. So, if you are comfortable with one all of them, would be clear. So, try to connect the whole story, I have s_{t-1}, I do not want to pass it as on pass it on as it is to s_t. So, I am computing some intermediate value, where I will only selectively write some portions of s_{t-1} and selectively write in the strictest case, it should be binary, but that is not, what we are interested in. We introduce fractions if the fraction has to learn binary let it learn, but we will make it fractional; that means, we will make it between 0 to 1.

Hence, the sigmoid function right, remember in one of these lectures, we had said that sigmoids are still used because in RNNs and LSTMs; remember in we had said that sigmoids are bad use stanage (Refer Time: 10:32) or use ReLU, but we had ended with

sigmoids are still used in the case of recurrent neural networks and LSTMs. So, this is where they are used ok, how many should get that connection? Ok good fine.

So, we use sigmoids, because we want the fraction to be between 0 to 1 and we also want some parametrization right and this is the particular form that, we have chosen. There are various equations possible various things, you could have done here. In fact, there are 10 to 15 different variants of LSTMs, I am covering the most popular one, which uses the following equation right. So, it is says that this is how you will compute the output gate and that gate will regulate, how much of the cell state should be passed on from t minus 1 to the next state? Ok. Everyone clear with this ok. So now, if you are clear with this give me an equation for H t minus 1.

Student: (Refer Time: 11:22).

Loudly, everyone. s t minus 1 is that fine right. So, this is the equation that we will have. So, we have done selective writing and these parameters are no special, they will be learned along with the other parameters of the network ok. So, let us spare some thought on that. You got a certain loss at the output ok. Earlier, you just had these parameters W, U, V which were the parameters of RNN, which you are adjusting to learn this loss. Now in addition, you also have the flexibility to adjust these parameters.

So, that if the lost could improve by selectively writing something then, these parameters should be updated accordingly right, may be you are being over aggressive and making o t minus 1 to be all ones; that means, you are passing everything to the next state right. Now it has the chance because, they have introduce parameters, if it helps the overall loss, it better make these fractions more appropriate. So, that only selective information is passed to the next state, how many you get this intuition? So, that is why anytime, you introduce parameters, you have more flexibility in learning, whatever you intend to learn.

There is remember, one clear difference here right and that is where I said that, while I was giving the analogy, I was really setting up things, but here there is one distinction, what is the distinction that is there? Ideally what would, I have wanted. Suppose I take the example of the review ok and the review was say, the movie was long, but really amazing ok. Now which is the word here, which is actually trying to mislead? So, overall sentiment is positive right, everyone agrees with that, but which is the word which is misleading?

Student: Long.

Long right; that means, I need to do what with that word?

Student: (Refer Time: 13:11).

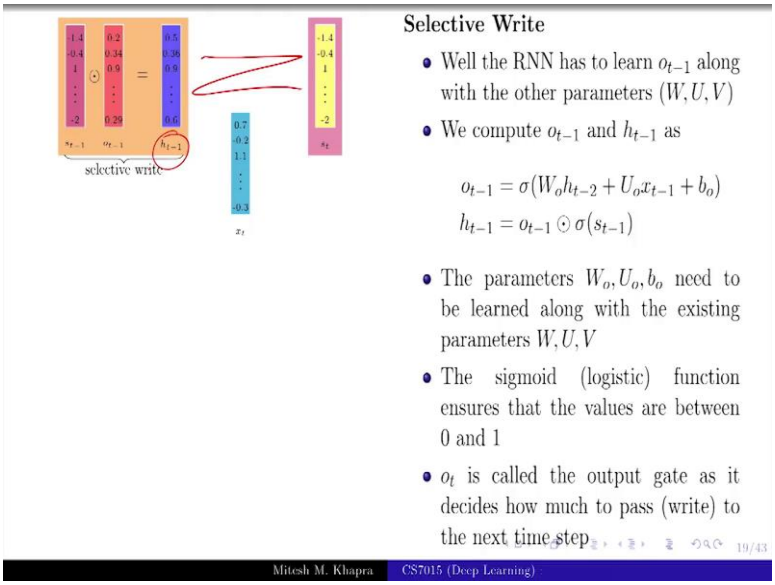
Forget that would, right. Now ideally, I would have wanted someone telling me retain, retain, retain, forget, retain, retain, retain. I would have a label for each of these words and then I could have a loss function which tells me, whether my gates were actually athering to this decisions or not? So, remember my gates are learning some distribution o_{t-1}, which tells me what fraction to retain? And at this particular time step, I would have wanted o t minus 1 to be all 0's ok. I would have wanted to forget, but this kind of not just o t minus 1, this will become more clear, when I do all the other gates also. So, what I am trying to say, is that you should have had some supervision, which tells you which information to retain and which information to forget, but you do not have this supervision right no one is telling, whether these are the important words these are not the important words?

So, that is the difference between the whiteboard analogy, there you knew exactly which step is important and which step is not important, here you do not know that. All you know is, that you have a final loss function, which depends on plus or minus, whether the this prediction is close to positive or close to negative and what is the loss and that loss is what is being back propagated. But the difference now is that you have introduced a model, which can learn to forget some things right. Earlier you did not have a model, which could learn to write or read or forget selectively, now you have introduced a model, this is a better modeling choice right. So, the same as we have had arguments that you could do y is equal to W transpose x or you could do y is equal to deep neural network of x right, you are making different modeling choices here and with the hope that one modeling choice is better than the other choice.

So, just as RNN was one modeling choice now you are using a different modeling choice where again with the help of these gates and all you can definitely write a function of y is a function of the input and that function is going to be LSTM function, which we will see in detail. So this, one part of that function and while doing this you are just making a better modeling choice, which allows you to learn more parameters and along the way, if important do selective write, read and forget is that clear? Right so, you would see the

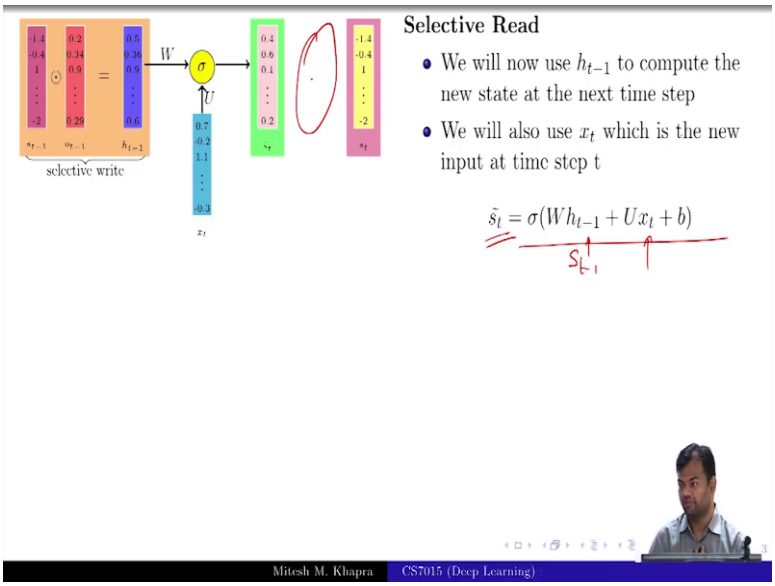
difference, what would have been the ideal case and what is it that you have? The ideal case would have been explicit supervision for what to forget, read and write, you will never have that, but you are still making a modeling choice, which allow you to do that. So, if it required to model while back propagation should be able to learn these parameters. So, get you are able to do that.

(Refer Slide Time: 15:39)



I know I am repeating myself, but it is very important that you understand this situation, how many of you get this now? And as I said these parameters will be learned along with other parameters and o t is called the output gate, because it decides what to output to the next cell state, ok. Still you see that there is a lot of gap here, we have not reached s_t yet we are still at s_{t-1}, we have computed some intermediate value. But we have not reached s t yet and along the way we had 3 things selective write, read and forget, we have only taking care of selective write so, far ok.

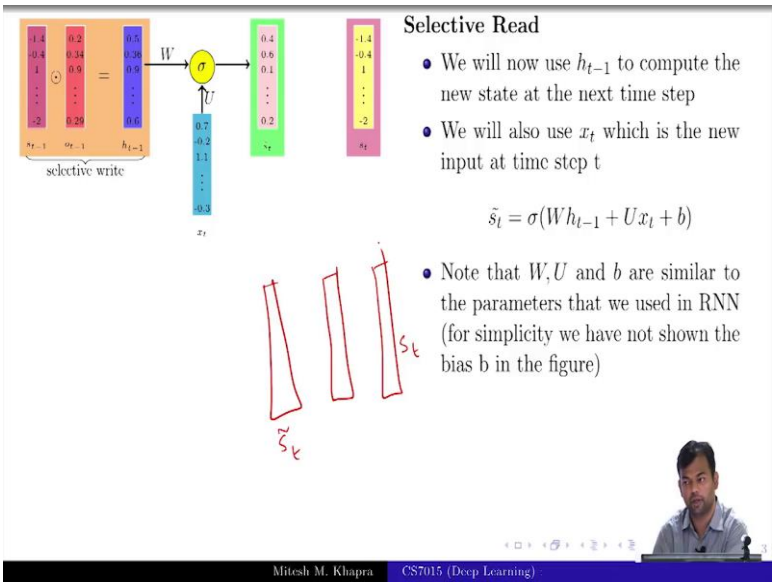
(Refer Slide Time: 16:05)



Now let us look at selective read. So, what this selective read, do you are going to get new information at time step t, which is x t right and now instead of this original cell state, you have used the selectively written cell state because that is what you have written now. So, that is what you should use.

Now, using a combination of these 2, I am going to compute, some intermediate value, ok. And just stare at this equation. This equation form is very similar to the RNN equation form, right. Only thing is that instead of s t minus 1, I am using h t minus 1 and for good reason because, I know that h t minus 1 contains only selectively written values from h_{t-1}. Is that fine and x t is the new input. Still there is some gap here, I have not reached s_t yet I am still at an intermediate value. So, this is the new input, which I have received. Now what should I do with this new input? Selectively read this input I do not want to take all of this input because, may be the input which I have got now is a stop word and I do not want to read all of it right, do you get that?

(Refer Slide Time: 17:15)



So, now it captures all the information from the previous state as well as the current input and we want to selectively read this. So now, what would you do to selectively read? Again the same situation that, you have a $\tilde{s}$ the answer is already here. You have s tilde and you do not want to pass it on as it is to s_t, this is $\tilde{s}$, s_t is somewhere here, which you do not know how to get to, but you know that you do not want to pass on all the input that, you have read, you want to selectively pass it on. So, what will you do now? Again introduce a.

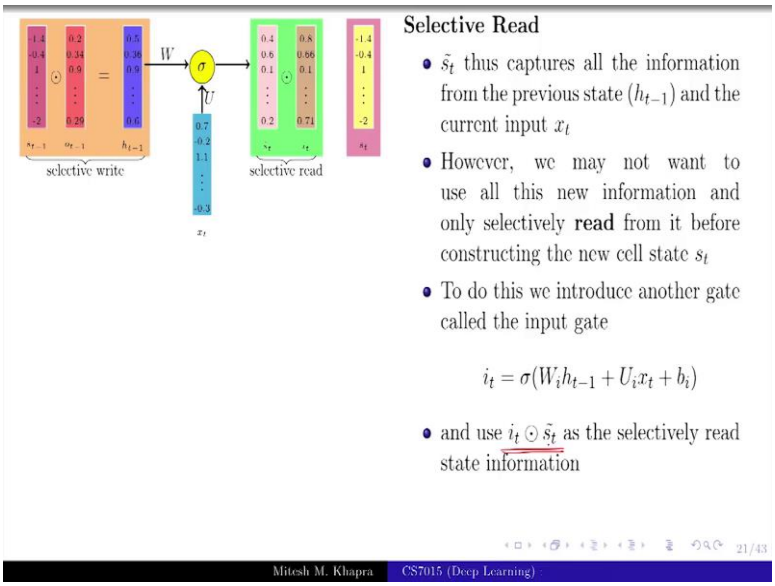
Student: Gate.

Gate and this gate will be called?

Student: Read gate.

Input gate or the read gate right ok.

(Refer Slide Time: 17:55)

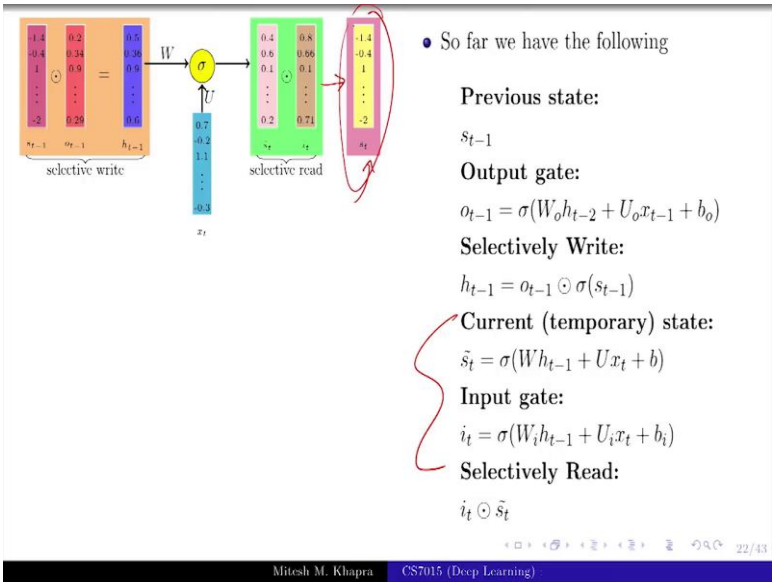


So now, what can you give me an equation for the gate i t is equal to sigma of that is good because, sigmoid is what we need it is going to be a fractional thing. Let me add the easy part. W into.

Student: h_{t-1}.

h_{t-1}, that is telling you what has happened so far and U times x_t. You see the same equation, same form the parameters have changed. So, these we will call as W_i U_i and V_i and they are depending on the input as well as the previous state, previous temporary stay that we had computed ok. So, that is exactly, what your input gate is going to be and now this operation is the selectively reading operation. How many you are fine at this point? Ok and then this product is going to use to be it is will help us to read selectively from this temporary value that, we have constructed or the input that we have taken ok.

(Refer Slide Time: 18:51)



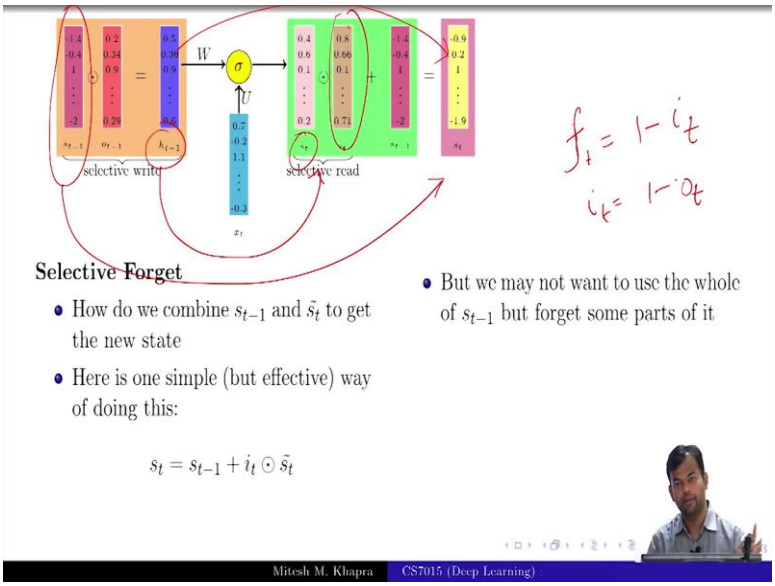
So, far what do we have, we have the following, we have the previous state, which was s t minus 1, then we have an output gate, which was o t minus 1 using these 2, we have done selective write right, we have taken the previous state and the gate and then a selective write. Is that fine? ok. We need to check, if the sigmoid should come here because, sigmoid is already there in the computation of s t minus 1 right. Oh it is not there. So, this already has 1 sigmoid right yeah. So then, again a sigmoid on that is it there? Ok we will figure it out, just check the equation right.

So, there may or may not be the sigmoid, the sigmoid already applied to s t minus 1, but we can figure that out ok. So, this is the selective write portion then, you compute the current temporary state ok, and just look at the similarity between these equations. Then you have an input gate and using these 2, we have done a selective read ok. So, you have taken care of selective write and selective read, but you are still not reached s t, I still do not have an arrow here, I still need to figure out, how to compute the s_t finally, ok. So, what is the operation which is remaining now? Selective.

Student: Forget.

Forget ok. So, what do you think should we forget? We want to find new s_t. So, let us see what we will forget right.

(Refer Slide Time: 20:07)



So, the question now is that you had this s t minus 1 and now you have a temporary state $\tilde{s}_t$ which is here. How do we combine these 2 to get the new cell state? Ok. So, the simplest way would be, that you say that s_t is equal to whatever was there in s_{t-1} plus selectively reading from the current input. Is that fine, this is the one way of doing it ok, but now what am I doing here, what is the problem here? I am reading, I am taking s_{t-1} as it is right. So, what should I do? I should forget some parts of s_{t-1}. So, what should I do for that? Introduce a, what gate?

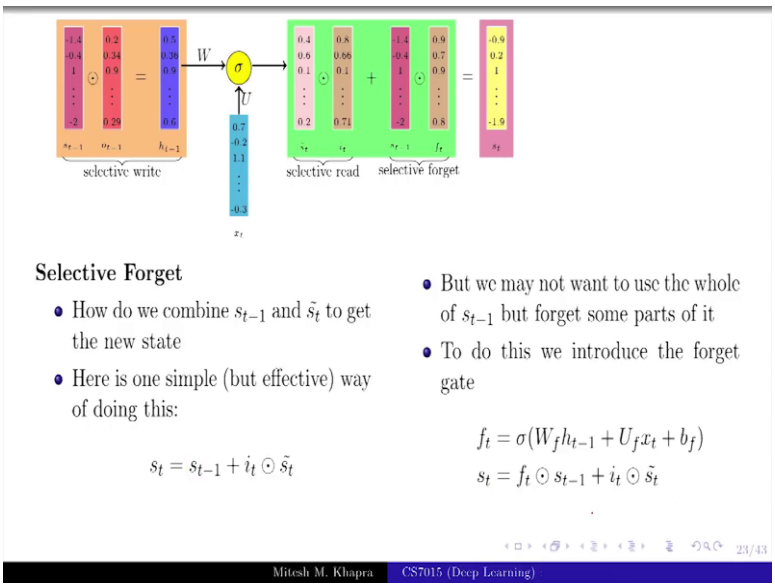
Student: Forget gate.

Forget gate right. So, we may not want to use s_{t-1} as it is, but we are want, to forget. So, there is at this point all of you should get some confusion, if you do not then, I would be worried, if you are getting some confusion good right. You should all get confused at this point. Why are you confused? Because you already did selective write and now again you are doing a selective forget also right, but there is a difference because, the selective write was then used to compute, how to read the information right, but now once you have read the new information, you want to see how to assimilate it back with the old information that you had right. So, that is why you introduce a separate gate. So, think of it as this way that you are keeping these functions separate input, output and forget. So, they can separately learn things ok.

So, whatever you want to selective write let it be a separate function. These h t minus 1 is not going back to s t right. Let us just by use so that, you can compute these temporary states. So, that is what is being passed to the next temporary state. Let i t only decide how much of this input should be read? Ok and then when you want to combine these 2, just use a separate gate and this exact idea, which is confusing all of you, why have a separate write gate and a separate forget gate? Led to something known as gated recurrent units, where they merged these to gates we will get back to that ok.

So, at this point it is fine. I am just telling you the original equations for LSTM and this was the motivation that they had. So, as I said there are at least 15 to 20 different variants of LSTM, which use different equations, they tie some of these weights. So, one thing could be that forget is the same as 1 minus remember. Right? Or output could be same as 1 minus input right, you could have tied these gates instead of learning separate parameters for that.

(Refer Slide Time: 22:57)



So, in the most parameterized form you have a separate parameter for all of these ok. So, we introduce the forget gate again, can you tell me the form for this forget gate f_t is equal to first term.

Student: W_f.

W_f, second term U_f, what will be there in the second term x_t and the first term ok. So, this is what it will look like ok. So, if you remember one of these equations, you will be able to write all of these, not that I am going to ask you to write them in quiz or something, but why take a chance. So, and then once you have completed the forget gate, instead of this equation, can you tell me what is the equation and you are going to use? What is the first term going to be it is s_{t-1} here? What is it going to be now?

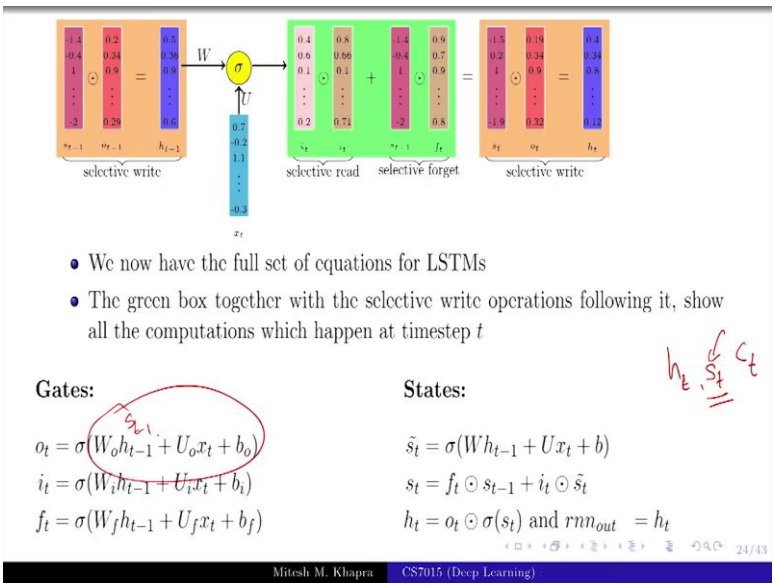
Student: F_t into.

F t into?

Student: S_{t-1}.

S_{t-1} that fine ok.

(Refer Slide Time: 24:47)



So now, we have the full set of equations for LSTM, we have certain gates and certain states, what are the gates? Output gate.

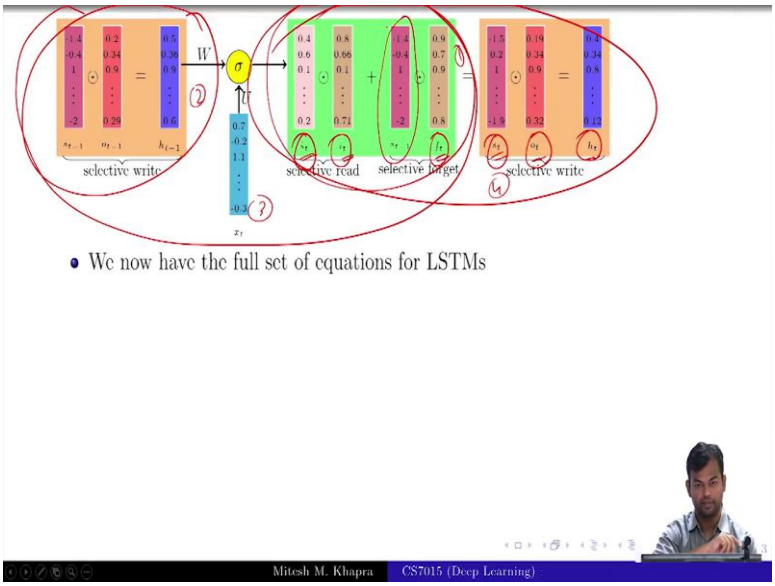
Student: Input gate.

Input gate.

Student: Forget gate.

Forget gate, why do you guys has this momentary amnesia like suddenly you forget everything ok. So, output gate, input gate and forget gate all of these are the same form with different parameters ok. What about the states, which are the states that we have completed? One was s_t the other was h_t and the third one was $\tilde{s}_t$ ok $\tilde{s}_t$ from $\tilde{s}_t$, we get s_t and from s_t, we compute h_t ok.

(Refer Slide Time: 24:37)



So in the diagram, that you see here at the top. Tell me which are the computations, which are happening at one time step. At time step t, which are the computations, which are happening? Is it I will give you the options right, is it this or is it this let us call this 1, let us call this 2 or this 3 or this 4. Which are the computations happening at one time step? And you see the order also here this should be straight forward right why?

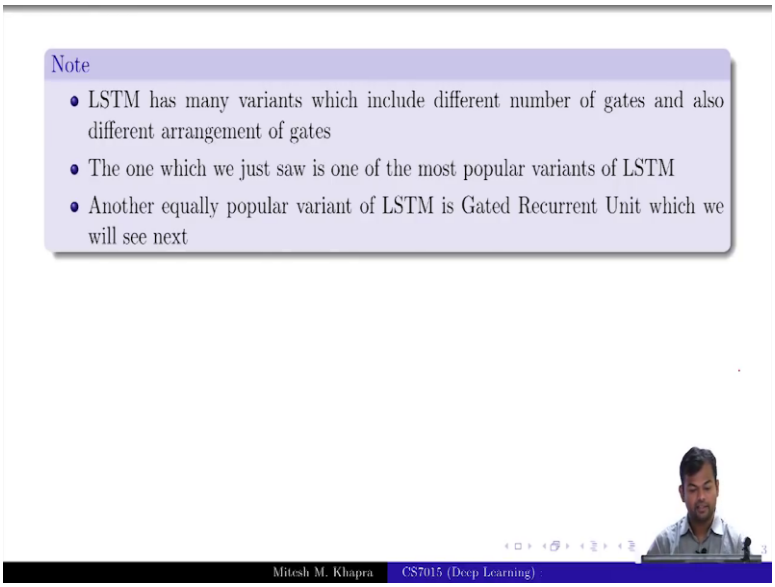
Student: (Refer Time: 25:16).

How many of you say 4? That is the one right because, you start with selective reading right and you can just go by this right these are all indexed by t right is that fine ok. So, these are the computations, which happen at time step t and these are exactly the computations which were written right. So, we have the 3 gates, which you need to compute at every time step and you have the 3 states, which you need to compute at every time step. Is that fine? And this s t minus 1 is not being computed. It is just taken from the previous time step is that fine. So, you have these 6 computations, which happened at every time step and the output final output of an LSTM. So, when you use

tensor flow or something the output of an LSTM, would give you 2 things. It will give you h_t, s_t because, these both the states that are being computed, one is the running state and another one is the current state, which is being computed ok.

And I choose the notation S because, that is what we have been using for RNNs, but in LSTM in all the literature instead of S, you will find it to be ct because it is called the cell state. So, that is why s_t ok. So all these equations wherever, you see an s when you are reading some standard blogs or things like that you will see C instead of S. So, you just do this mapping in your head ok.

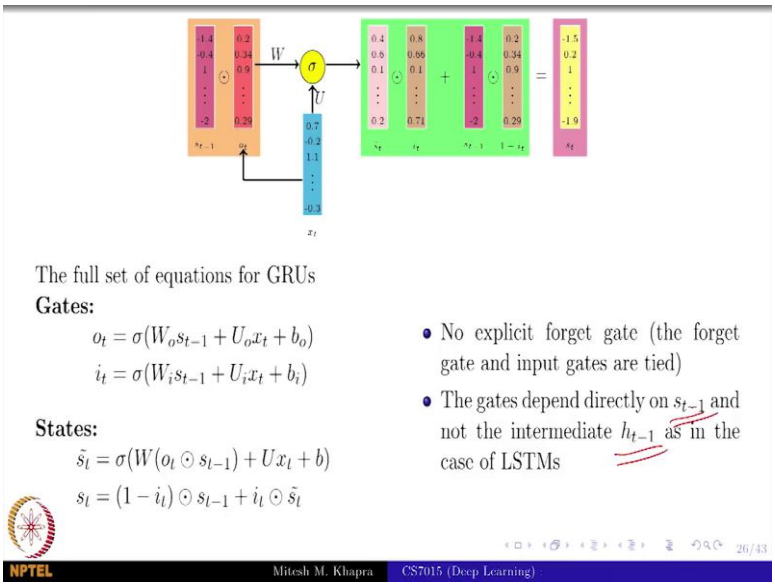
(Refer Slide Time: 26:41)



So, LSTM actually has many variants with include different number of gates and also different arrangement of the gates. So, as I was saying that you could say that input is 1 minus output or input is 1 minus forget or things like that and also why this particular parametric form right why not make W 0 into s_{t-1} instead of h_{t-1} and so on. So, the all points, of things that you could do or all of these are valid these are all valid variants of LSTMs.

So, there is this paper called LSTM, a search space odyssey. So, you can go and look at I think we link it in the in the reading material right ah. So, you can see that there are actually many variants of LSTMs, but this is the most standard and default variant, which you will find in most platforms on tensor flow or pytorch (Refer Time: 27:26) form.

(Refer Slide Time: 27:33)



And there is another very popular variant of LSTMs, which is called gated recurrent units. So, we will just see gated recurrent unit. So, I will just give you the full set of equations for GRUS. So, you have gates, but unlike LSTMs, you have only 2 gates, you have an output gate and you have an input gate, you do not have the forget gate ok.

So, what am I going to do for the forget gate? So, this is what I am going to do, you see the last equation. So, instead of forget gate I am just saying that this is what you are going to selectively input from the current temporary state. So, the rest of you rest of it you take from the previous state right. So, I have just tied the input gate and the forget gate any other changes do you see in this. So, earlier we had s t minus 1, everywhere right now we have s t minus 1 itself is that fine ok. So, the basic idea these equations are many and you could think of your own equations, you could say that I will not really use this input information at all or I will choose to use it differently or what not right there are several things that you could do. At a very abstract level this is what you need to, what is this?

Student: (Refer Time: 28:38).

So, these parameters could then make a difference right, they could adjust it accordingly and so on right. So, that is what I was coming. So, the there are various ways of realizing this right at the abstract level, you need to understand that the original problem was trying to store all the information from time step 1 to time step T capital T right, which is

not feasible, because of this finite size that you have. So, along the way we built this intuition that it should be good to have these operations, which allow you to selectively read write and forget right. How do you mathematically realize these operations? There are various choices for doing that and we saw a few choices for doing that right, there are many others you could have done. But this is largely, what whenever you say that I have used an LSTM, most likely you are using the set of equations which I saw, which we saw on the previous slide and whenever you are using a GRU, these are the set of equations that you will be using ok.

And again remember this, that there is no explicit supervision here, it is just that we have a better modeling choice we are just introduce more parameters. So, that if required these parameters could be adjusted to do a selectively read, write and forget right. So, it is often, it is often valuable, if you are doing some task with RNNs or LSTMs. You should visualize these gates right, you should see that at time step t, if you thought that it should have forgotten everything that it has learnt. So far, because suppose you had this the movie was long, but I really loved it because, the direction was superb and so on.

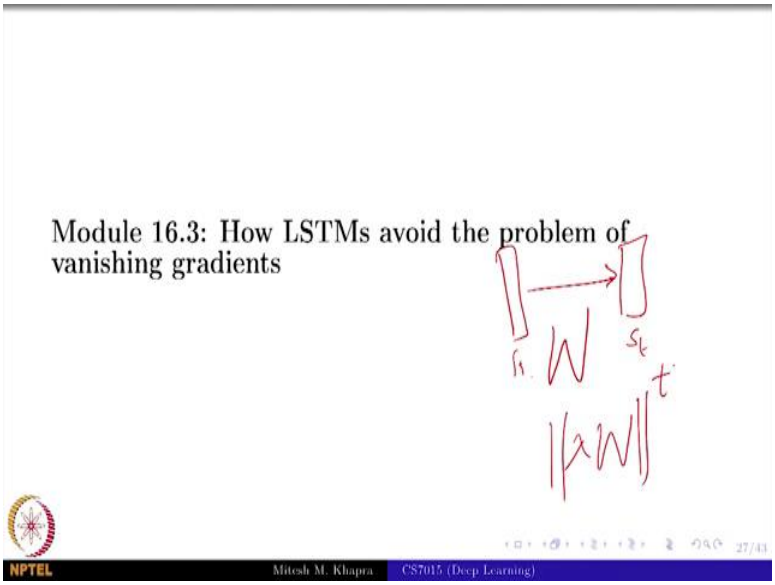
Now this word, but actually changes everything right because, it whatever was written before it does not matter anymore right. So, is it really learning those kind of gates, where everything before, but was forgotten right. So, it would be helpful to visualize these output gates and see what kind of values, they are learning. What kind of things they are remembering forgetting and selectively reading and so on right. So, as I said I will just again summarize the key thing here, is the intuition and then the realization in the form of equations, there are multiple choices, we have seen a few of those right that is what I will end with. And, in particular in GRUS, there is no explicit forget gate and instead of h_{t-1} you use s_{t-1} everywhere.

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 110
How LSTMs Avoid the Problem of vanishing gradients

So, that was LSTM and GRUs.

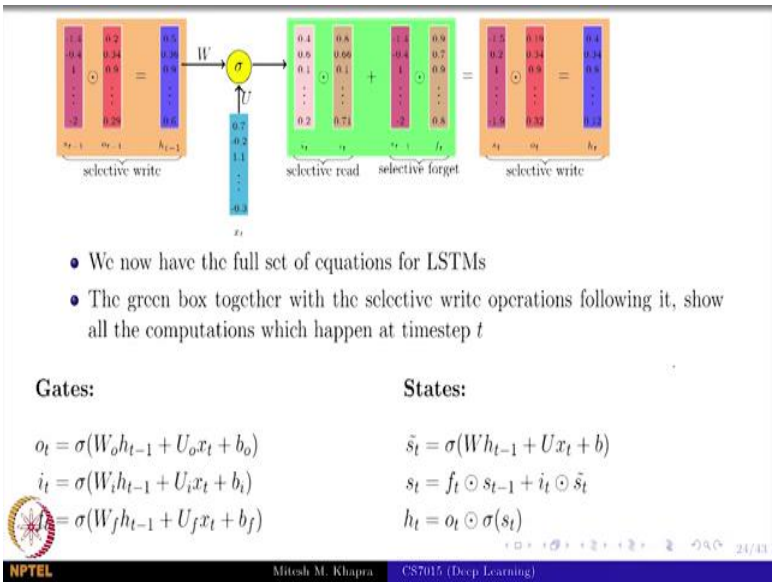
(Refer Slide Time: 00:15)



Now, the issue is that, I have given you a very explanation that why you selectively read write and forget should work, but you have not actually formally proven or even given an intuition for with these sets of equations, how are we sure that the gradients will flow back right. We introduced a bunch of equations, remember in the case of LSTMs sorry in the case of RNNs, the problem was because of the recurrent connections right. Because you had these recurrent connections this W which was the recurrent parameter right, which was connecting cell state S_{t-1} to cell state S_t.

This was repeatedly appearing in your gradients right and that was causing the problem because, when you had this multiplicative factor lambda into W and then if you compute the and this was some raise to t. So then if you compute this magnitude then if the magnitude of W blows up then the whole thing will explode, if the magnitude of W vanishes then the whole thing will vanish right. That is the problem that we had.

(Refer Slide Time: 01:29)

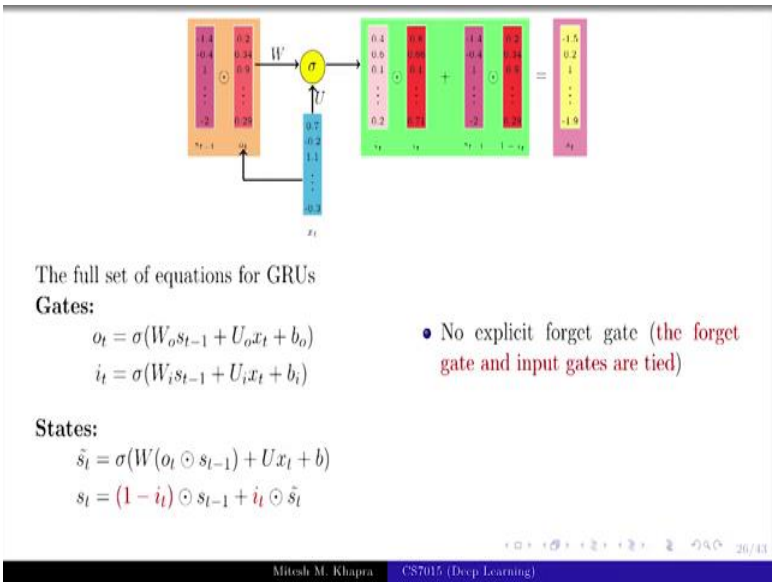


So, this was because of the recurrent connections. Do we have recurrent connections in LSTMs or GRUs for that matter? Do you have recurrent connections? Yes or no?

Student: Yes.

Yes, so then that problem could still occur right, I mean if you had that the crux of the problem for the vanishing gradient was this recurrent connection which is getting multiplied. And hence reading to problem, but we still have recurrent connections the case of LSTMs also and why should things become any easier in this case. How many if you get the question, how many if you can give me the answer selectively. That is a good answer. So, can you think of what is happening here. So first thing that we going to do now so I will go on to the next module.

(Refer Slide Time: 02:05)



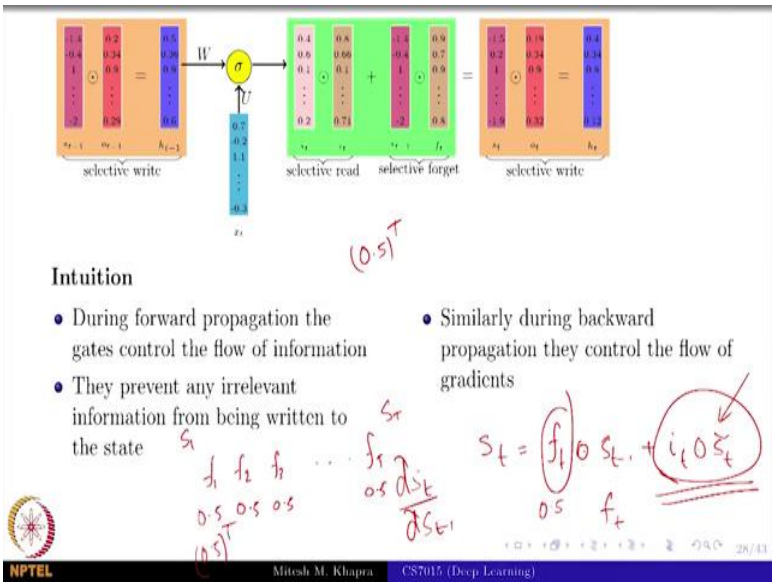
When I going to give you intuition for what is happening and then we will do slightly, in fact, a rigorous proof of why it actually solve the vanishing and exploding gradient problem ok. So, let us look at the intuition first. How LSTMs avoid the problem of vanishing gradients, I am only focusing on vanishing gradients. Exploding gradients are actually easier to deal with. Why?

Student: (Refer Time: 02:26).

What can you do, what are we interested in when we compute a gradient? Direction. So, if the magnitude is very large what can we do, just normalize it and restricted to be a certain magnitude, so that is known as gradient clipping. So, exploding gradients in that sense is still not a big problem, but vanishing gradients is because, if it vanishes you cannot do anything, because you could think of it that you already have a learning rate which is getting multiplied with the vector the gradient. Now in addition to the learning rate which was anyways clipping the norm of the gradient right. So, you are doing an expressive clipping also.

So, it just like a additional learning rate inductions right ok.

(Refer Slide Time: 03:06)



So, here the intuition and then will go to the more rigorous stuff, not in this class probably. So, during forward propagation, the gates control the flow of information right. The gate decides how much of S_{t-1} should be pass to S_t ok. And they prevent any relevant information from being returned to the next state. Similarly during back propagation, the gates will regulate the flow of information. So, what I mean by that is that if at a certain state, you have computed $S_t = f_t (S_{t-1} + i_t \tilde{s}_t)$ right.

So, this gate is actually deciding how much information flows in the positive direction ok. And suppose this gate value was 0.5, so the 0.5 of this information from S_t has been carried on S_{t-1} ok. Now during back propagation what is the derivative of S_t with respect to s_{t-1} going to be? Partial derivative. Is going to be a f_t. Think of S_t and S_{t-1} as single variables like you know n dimension variables, then the just f_t. Of course, you are forgetting that, what kind of a network is this? Ordered network right. So, you cannot read as till de t as a constant.

Where s tilde T also somewhere depends on S t minus ok, but just this assume that, maybe this vanishes and that is the worst case assumption right. Because, I do not want it to vanish, but I am assuming that the second term vanishes, but even then with the first term I will have a gradient which is proportional to the gate. Why is that fine? So, remember that I am not making a easy assumption, I am making a worst case assumption. This is not favourable to me, I am saying that the second term vanishes and i

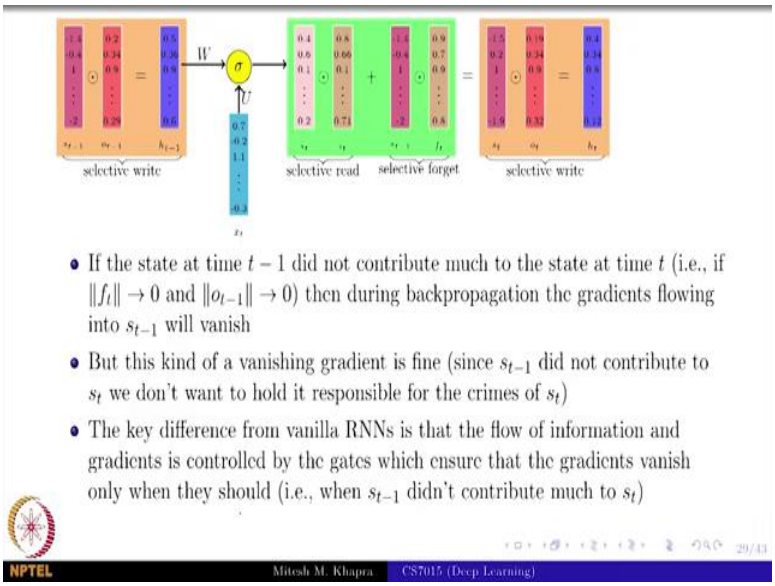
dont want it to vanish, but i am just trying to prove that even in the worst case, by the second term vanishes, you still have this gradient f t from the first term right.

And why is that good, why is that ok, because f t decides how much flew in the forward direction. And it is also deciding how much goes back in the backward direction. So, it is a fair regulator with says that if I passed on only this much information in the forward direction. Then during backward pass also I should only make a responsible by this much, ok. Now let us look at a situation where you had f 1, f 2, f 3 upto f t and all of these gates were 0.5. Now 0.5 seems a reasonable value, but when we have 0.5 raise to t and t is a large value, what is going to happen, this quantity is going to vanish.

So, what is happening is that S1 contribution to ST in the forward direction itself had S1's contribution to ST in the forward direction itself was had already vanished right because, it was continuously getting multiplied by 0.5, 0.5, 0.5, so it is like this Chinese Vespring problems right. So, this guy said something whereas, next guy added noise, the next guy again added noise and so on, till the time it reach the Tth guy this information was completely lost So, in the forward pass if S1 did not contribute to S t in the backward pass should I make it responsible for the crimes of S t, no.

So, what is happening in the backward pass, again the gradients are getting regulated by the same forget gates. So, again in the backward pass will have a situation that, by the time the gradient reach is S1 it would be 0.5 raise to t and that is fine. It is going to vanish, but that is because even in the forward pass it vanished. So let it vanish in the backward pass also. So, this kind of vanishing is ok.

(Refer Slide Time: 06:41)



So, this is just the same thing written in words. So, if the stated time t minus 1 did not contribute much to the state at time t because, f_t was tending to 0 right. Then during backpropagation the gradients flowing into s_{t-1} minus one will also vanish because again during backpropagation the gradients will get multiplied by f_t and they will vanish.

But this kind of vanishing gradient is fine. This is fair because, if we did not contribute in the forward direction why should I help you hold your responsible in the backward direction right. So, that is fair. So, the key difference from RNNs is that the flow of gradients is now controlled by gates, which give the same regulation in the forward pass as well as the backward pass right. So, only if you contributed to something you will be held responsible. If your contribution vanished your responsibility in the backward pass will also vanish right. So, that is the intuition.

(Refer Slide Time: 07:27)

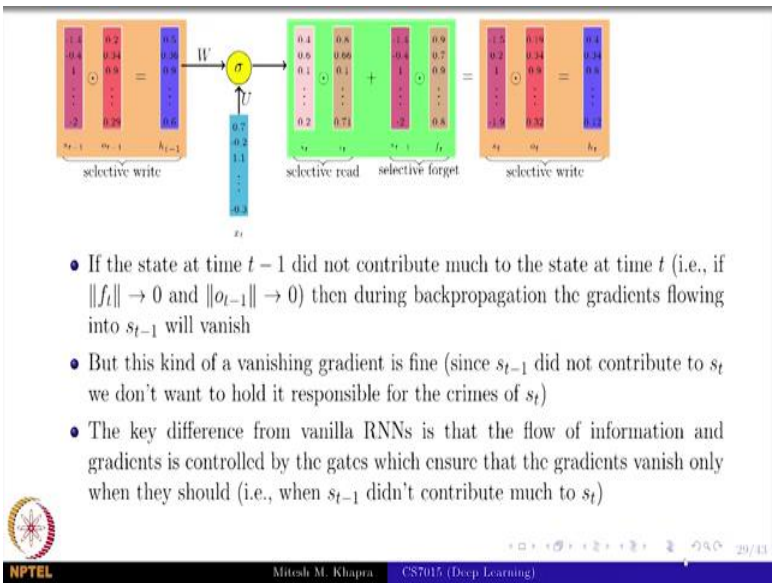


And will next see an proof for this. A proof actually it s as based on the intuition, but I just make it more formal in terms of introducing the notations and so on. So that problem we will do it in the next class ok.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 111
How LSTMs avoid the problem of vanishing gradients (Contd.)

(Refer Slide Time: 00:12)



Ok so, will start from where we left off. So, in the last class we started with this motivation that recurrent neural networks have this problem of Vanishing and Grade Exploding Gradients. And we wanted to arrive with some principle way of avoiding this. So, you have first started with this intuition that in many real life situations like for example, the human brain or the whiteboard. We tend to these to these three operations called selective read, selective write, and selective forget. And they essentially help us in dealing with these finite sized memories right or whether it is a whiteboard which is finite sized or your brain or whatever it is right.

So, can we is it possible to kind of improve RNN's which also suffer from this problem that they have this finite sized memory. And hence if you are trying to capture everything from time step one then by the time you reach time step t where say t is 30 or 40 or so on. Its quite natural that whatever you have learned earlier will get move off to an extent that it just is not recognisable anymore right.

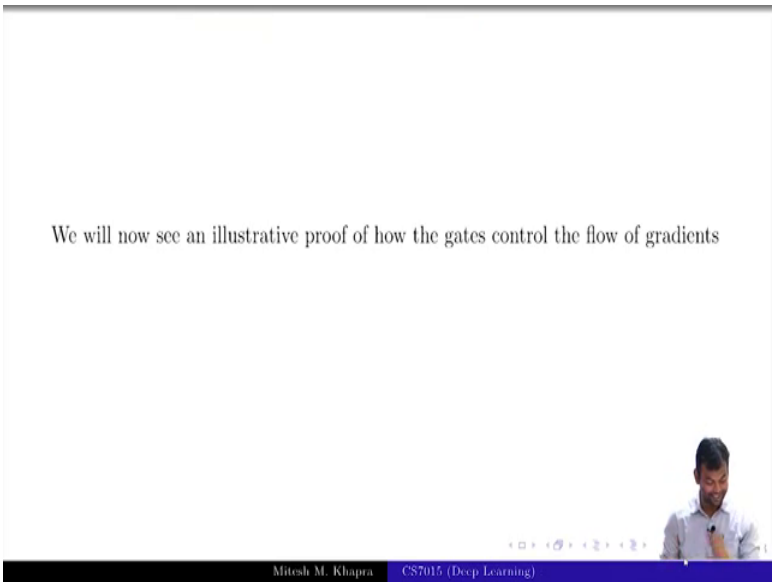
So, you wanted to deal with this problem and with that we motivated selective read write and forget. And then we introduced some equations or converted this into a model and this is the diagram that you see is the model actually that is the LSTM cell, yeah. And it has these three gates output gate, input gate, and forget gate and which perform these three functions of selective read write and forget. So, intuitively all these was fine, but we need to be more technical in terms of you trying to deal with a problem of vanishing and grade exploding gradients.

So, how does it solve that problem all that makes or the story seems fine, but how does this actually relate to the math. So, we saw some intuition for that and the intuition hinsed on this observation that. During forward pass the gates control how much of information passes from one state to another. And in particular if you have the situation that from one time step to another say the forget gate tells you that keep forgetting point 5 of the previous state. Then by the time you reach say the 100 state you would have forgotten 0.5 raise to 100 of the first state.

So; that means, even during forward pass the information from state 1 vanishes. So, if it vanishes during backward path that is also fine, because state 1 did not contribute to state 100. And that was the intuition that all this hinsed on. Now we are not going to do much different from this intuition. We just going to see the corresponding equations for these intuitions and just make a more I would not call it rigorous but more mathematical proof on why LSTM solve the problem of vanishing gradients.

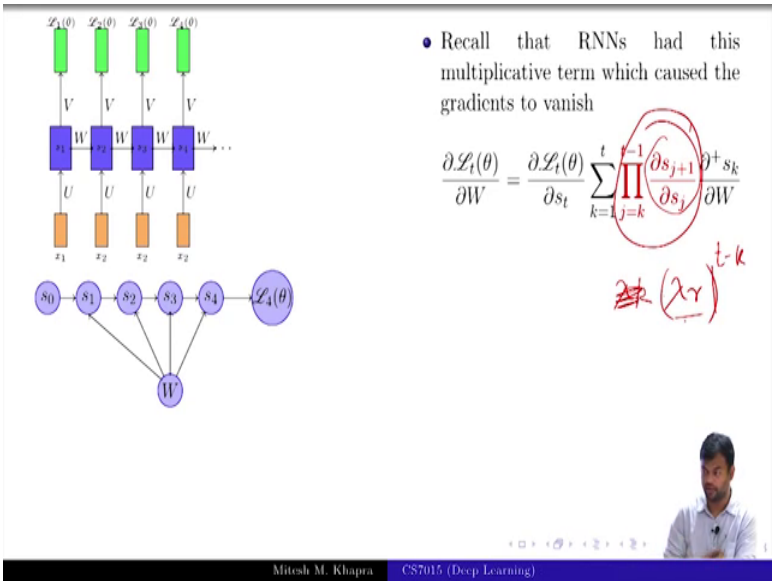
And we are also sure that they actually do not solve the problem of exploding gradients and then we will see a simple trick of dealing with exploding gradient. That is what we will do in the remainder of this particular lecture and then will move on to the next lecture in this lecture.

(Refer Slide Time: 03:02)



So, we will now see an illustrative proof of how the gates control the flow of gradients right.

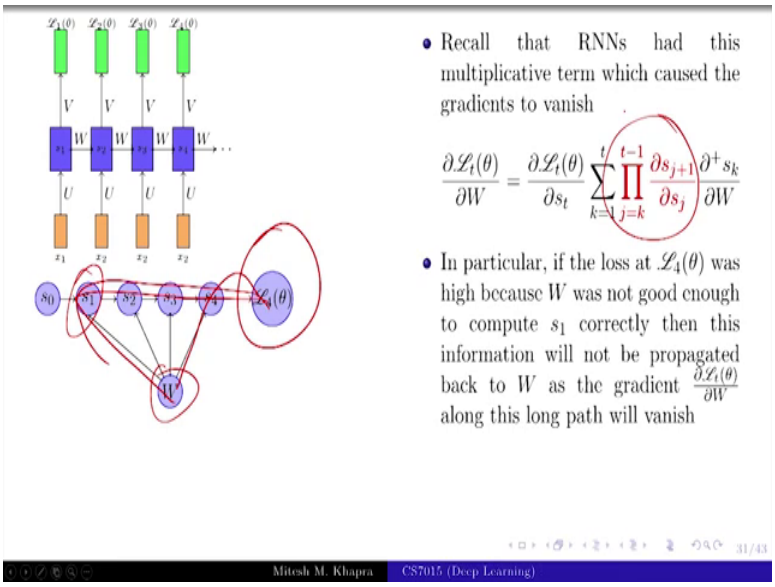
(Refer Slide Time: 03:11)



So, we call that this is the control this is the flow diagram or the dependency diagram that you had for RNN's. And in particular because you are dealing with an ordered network we add this explicit and implicit derivatives and finally, you came up with this multiplicative form. And this term here is actually a matrix because it is a derivative of a vector with respect to a vector.

And then this same matrix was getting multiple times and then we did this proof it showed that this term is actually lambda gamma ok. It is actually proportional to this term right and as if lambda into gamma is greater than 1, then this will explode. If it is less than 1 then it will vanish given sufficient times that is.

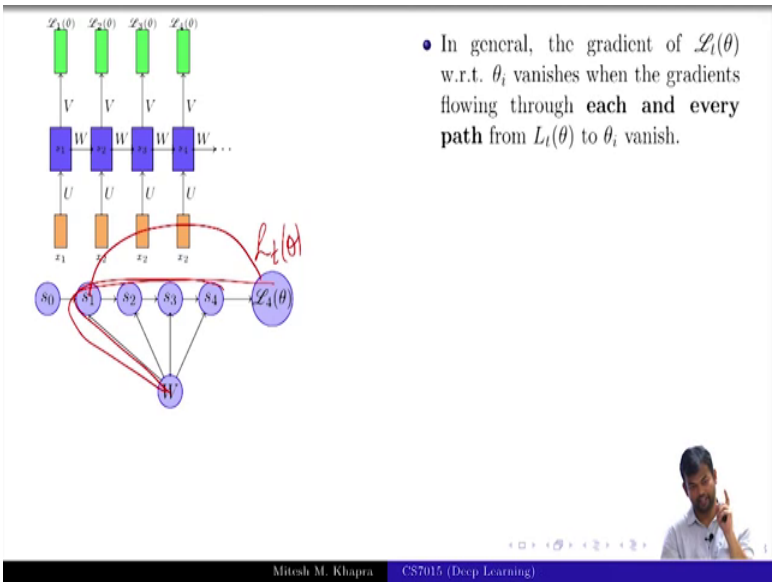
(Refer Slide Time: 03:56)



Now, in particular what is happening here is the following that you have this loss at time step t you have the time step is 4. Now, what if this loss or this error occurred, because W was not good enough to compute a good value for S1 right. So, W was at a certain configuration based on that you computed S1. And that S1 was not good enough which eventually led to the error at time step 4 all of you if you can imagine this situation that you mean you not being not be able to do something well at S1. Now this needs to be told to W so that it can improve right. And that information has to come through S1, that information is already going from here, but this information is about how badly it performed in computing S4.

This is not how badly it can perform in computing S1. So, that information has to travel to W all the way through S1 and that was not happening because this path do not look at the bullets this path was actually vanishing. And that is what this multiplicative term says that as the number of times that increased that time that path would vanish ok. So, that is the actual problem that we are trying to deal.

(Refer Slide Time: 05:05)



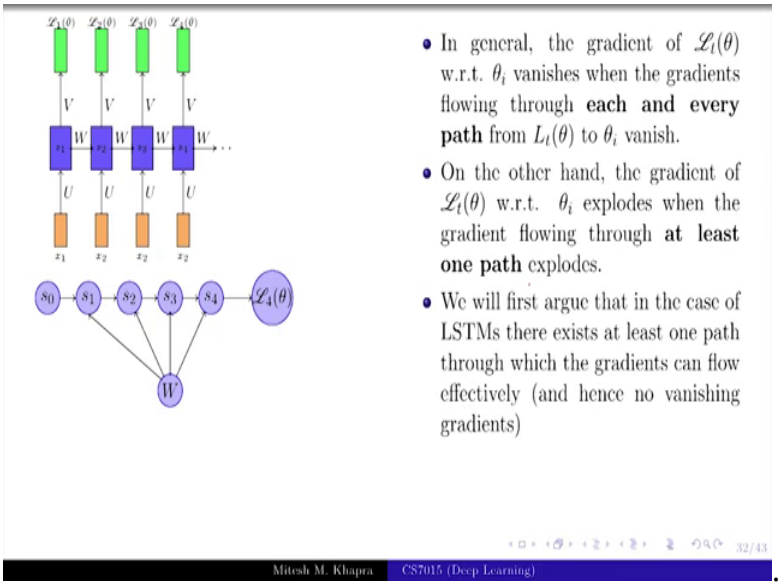
So, now what is the general situation here right, the general principle is that the gradient of L theta at particular time step say here we are considering L4 so I will just call it Lt with respect to any parameter theta i. The parameters at W u v b and c with respect to any parameter it would vanish if all the paths leading to that parameter if it vanishes. So, with respect to this particular path so that is the only path which leads to W through S1. If there were multiple paths if there was say one such direct path right if we had you some other kind of connection which gave us this direct path then it would still have been fine.

But there was only one path leading to W through S1 at the gradient vanishes along that path then the gradient will vanish ok. If there were multiple paths then only if the gradient varnishes across all the paths then the gradient would vanish. Is it fine? What is the corresponding rule for exploding gradients? If there are multiple paths the gradient would explode if.

Student: (Refer Time: 06:06).

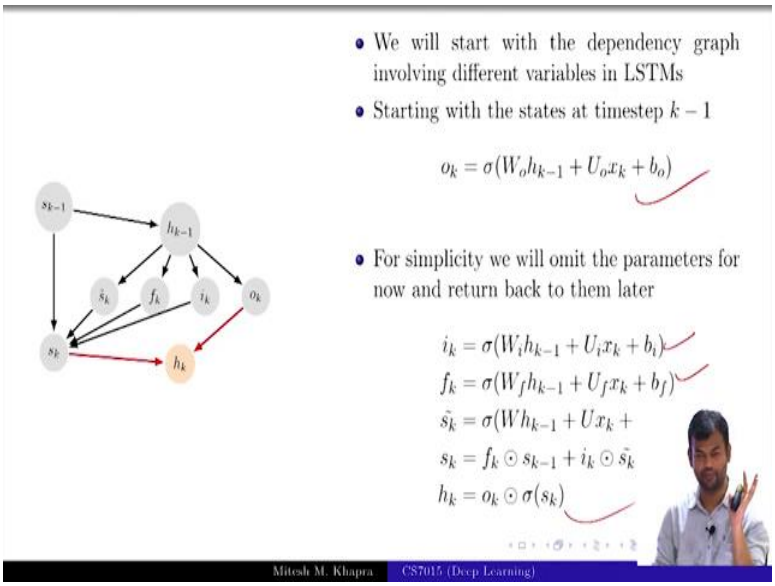
If it vanishes through any o[ne]- if it explodes though any one of the paths ok.

(Refer Slide Time: 06:11)



So, these are the two things that we need to consider ok. So, to prove that in the case of L st m this does not happen. For the first case will have to show that there are at least one path through which it does not vanish and for the second case because we are going to show that it explodes we just have to show that there is at least one path through which it can explode ok. So, these are the two things that we need to prove and the first thing that we are going to focus on; is the vanishing gradient problem.

(Refer Slide Time: 06:38)



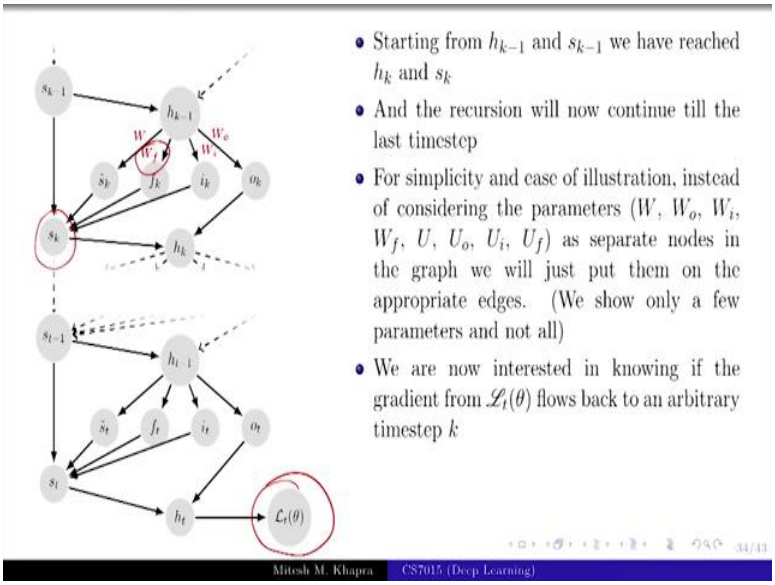
So, will start with the dependency graph for LSTM's; that means, I want to draw something similar for LSTM's involving all the different elements in LSTM. So, what are these different elements the two rhyming things one being gates states ok. So, gates and states that the two things that we care about. So, let us look at all these. So, starting with states at time step k - 1, at time step k -1 you have this two states s_k- 1 and h_{k-1} ok. Using h_{k-1} you are going to compute the output gate at time step k. And it is also depends on these parameters W_o, U_o and b_o right which is obvious from the equation.

Just to make sure that this diagram remains tractable, I am going to get rid of the parameters and I will come back to them later. So, right now will just focus on the states and the gates ok and then you have these other intermediate states and the other gates right. So, you had f_k you had i_k. So, add these three gates the temporary state and then what else what are the other two things at time step k. So, we saw this diagram about all the computations which happen at time step k right. How many computations happen? Three states and three gates right.

So, you seen the three gates and this one temporary state. So, which are the other two things? There is no selective forget with you guys is early everything forget. Hint look at the grey cells and change the time step. What will you get away are you all I mean we did LSTM's two days right I mean are you all with that or should I we need to revise something; mean I do not need to revise it, but we going to is it fine ok. So, sk and the other thing h_k remember that s_k also depends on h_k just stare at this for 30 seconds and make sure that you are with it right. All the equations are there these are the see 6 equations that or the 6 computations which happen at time step k.

There are three gates and three states and the dependency graph is obvious from these equations; except for the fact that I have ignored the parameters. How many if you are comfortable with the equations and the graph corresponding graph please raise your hands high. So, I think it should be right we have these six equations and we have this dependency graph.

(Refer Slide Time: 08:50)



Now, starting so, what happened in the graph is, we started from s_{k-1} and h_{k-1} and we reached s_k and h_k which were the outputs at the next state. Now what will happen from here? We were looking at recurrent neural networks recursion is the answer. What will happen now?

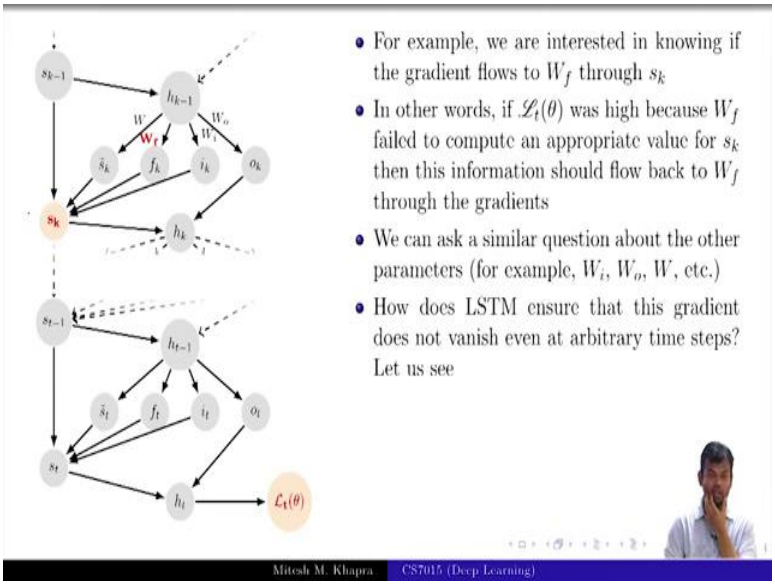
The same graph will keep recusing right for the next time step and up to the last time step right, does that makes sense ok? This looks much more complicated than the dependency graph that we had for RNN's right by just because there are so, many we in RNN we just had this one state and no gates so here but we have these three states and three gates that is why this so many paths ok. Now for simplicity what I will do is, I will not draw separate nodes for the parameters all the in the case of the RNN dependency graph I had drawn them separately. What I am going to do is, I am just going to put the parameters on the corresponding edges right.

So, f_k actually depends on W_f, it also depends on U_f, and it also depends on that bias. But I am just going to take a small set of parameter I am only going to focus on the W's not the U's and the biases ok. There is only for illustration for no other reason right and whatever arguments or proof that we are going to see it holds for all the parameters, but we just need to prove it with respect to one parameter and the same story repeats for everything ok. So, this is the dependency graph and these are the parameters. Now what I

am interested in knowing is that, there was some loss at time step t and maybe that loss happened because W_f was not good enough to compute s_k.

Of course W_f computes f_k and then f_k helps in computing s_k, but maybe W_f was not I am just short it short circuiting it and saying that W_f was not good enough to compute s_k right. And that is why I want the gradient to reach to W_f through this s_k that is what I want do ok.

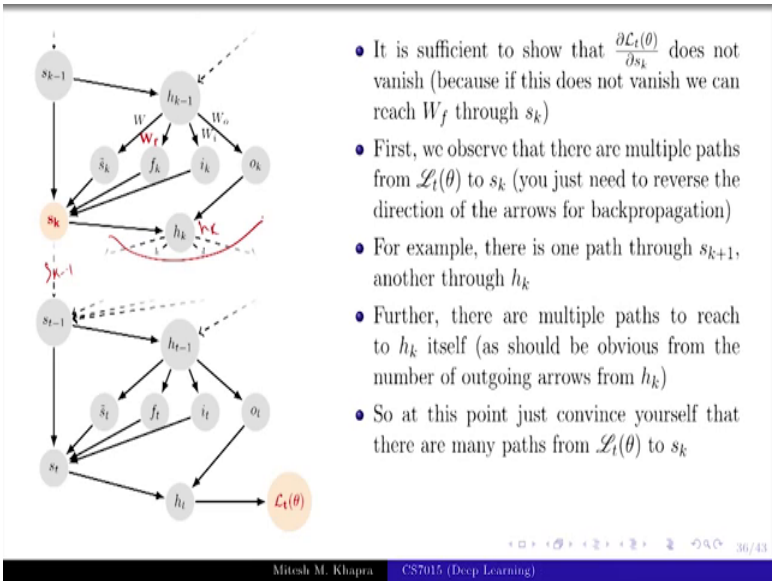
(Refer Slide Time: 10:42)



And this exactly what I said I am interested in knowing that if this loss can reach W_f through s_k right. So, all the three highlighted things that what I am interested in, I am interested in the path to W_f through s_k. Of course, there are many other paths to W_f, but they do not account for the problem in s_k; is that fine everyone is clear the setup ok?

Now and we can ask similar questions about all the other parameters the W's the us the the input gate parameters the output gate parameters and so on right. There is nothing so special about W_f the same question holds for all these other parameters also ok. Now how does LSTM ensure that this does not vanish? So let us see that.

(Refer Slide Time: 11:20)



As I argued earlier it is sufficient to show that this gradient does not vanish ok. If I can show that this gradient does not vanish, then I am pity sure there is only there is no recursive connection here because it just a single connection. So, there is no recursive connection here. So if I can show that the gradient reaches up to this point, then after that I can be sure that it is going to reach Wf everyone buys that set up right that is what I need to show?

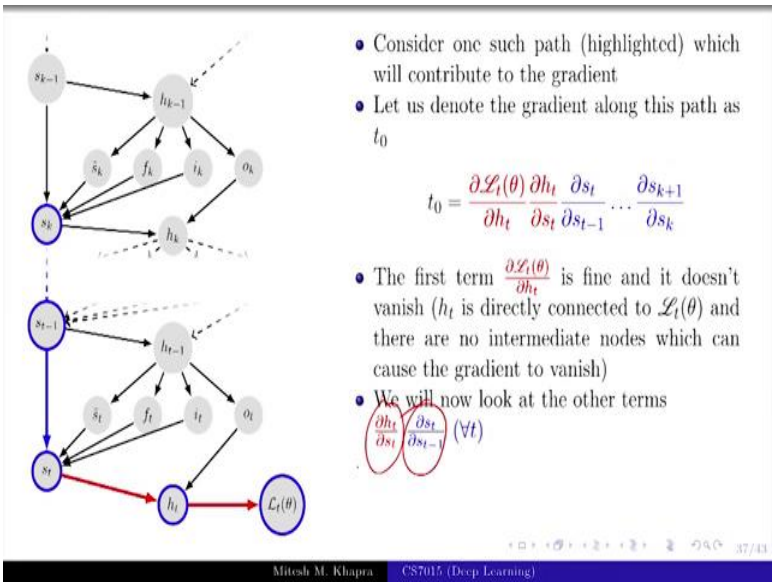
So, to prove that the gradient reaches W_f I just need to show that it reaches s_k that is the only thing that I need to show. And the first thing I am going to observe is that there are multiple paths to reach to sk which are these paths? One through s_{k+1}, because s_k contributes to s_{k+1} the other through.

Student: H_k.

H_k which is visible, but now also notice that how many paths are there to reach h_k itself? 4. Not four actually that is going to be combinatorial because there four outgoing edges from here, but then again there will be four next stage and four next stage and so on right. So, let us not count the number of paths, but let us just convince ourselves that there are many paths to reach to s_k from $L_t(\theta)$. Everyone is convinced about that we are not counting the exact number of paths that is not very hard to do. But all we are saying is that we know that there is one path through s_{k+1}, one path through h_k and h_k itself seems to have many incoming path during back propagation.

So, there are many paths which are reaching from $L_t(\theta)$ to sk. Everyone is convinced about that anyone who has a problem with that? Now to show that the gradient does not vanish what do I need to show of all the paths the set there exist at least one path through which the gradient can flow that is what I need to show ok. Even if I vanishes across all the other paths I am still fine with it ok.

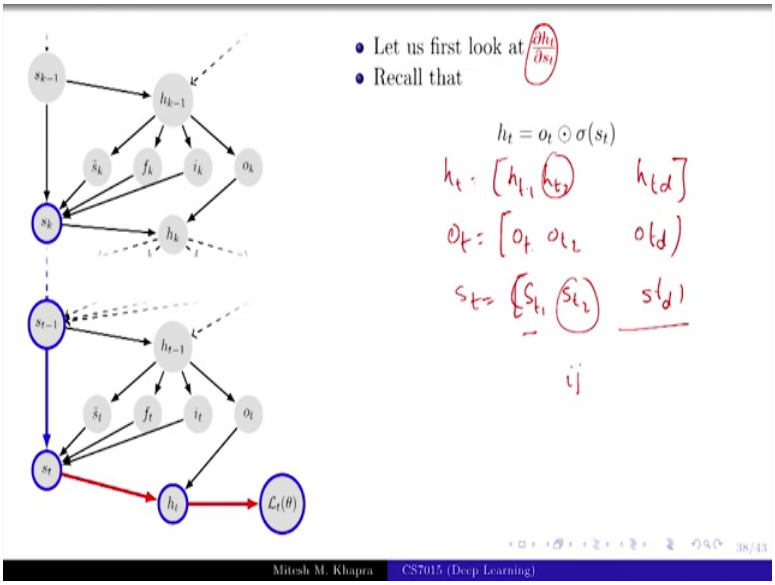
(Refer Slide Time: 13:05)



So, now consider one such path which is this highlighted path that is a valid path to reach to sk. Now let us denote the gradient along this path to be t naught and the total gradient is going to be a sum of many such paths right. So, I am calling this path as t naught and this is what the gradient look like ok. So, this is simple just this red path the next red path and then the series of problematic multiplications right you have this recursive multiplications again. So, everyone agrees that red is good, the red path there is no recursion the gradient will flow right we just need to focus on the blue path everyone is convinced about that right ok.

So, that is good the first term is fine as I said because it directly connected to L t there is no recursive or no other intermediate nodes. So, the gradient will just flow through that there is not a problem there and now we look at the other terms which is first is dh_t / s_t and the other is this ok.

(Refer Slide Time: 14:03)



So, let us look at $\partial h_t / s_t$ what is this going to be? Tensor, vector, matrix scalar at this point in the course I want a unanimous answer.

Student: Matrix

Matrix right and recall that in particular the equation was of this form ok. So, what is the derivative going to look like even without computing can you tell me something profound about it; it will be a dash matrix. Big matrix, how many if you say diagonal matrix? How many if you do not think it is a diagonal matrix please raise your hands total sum is never one. So, remember that ht is equal to h_{t1}, h_{t2} up to h_{td}. And you have o_t equal to o_{t1} o_{t2} o_{td} and s_t equal to s_{t1} s_{t2} s_{td}. So, h_{t2} depends only on o_{t2} and s_{t2} right it does not depend on in particular does not depend on any of the other st's.

So, we have already seen this before in such cases whats the i j^{th} entry of this matrix of the gradient matrix? Derivative of h_{ti} with respect to s_{tj} which of these terms are going to be zero wherever?

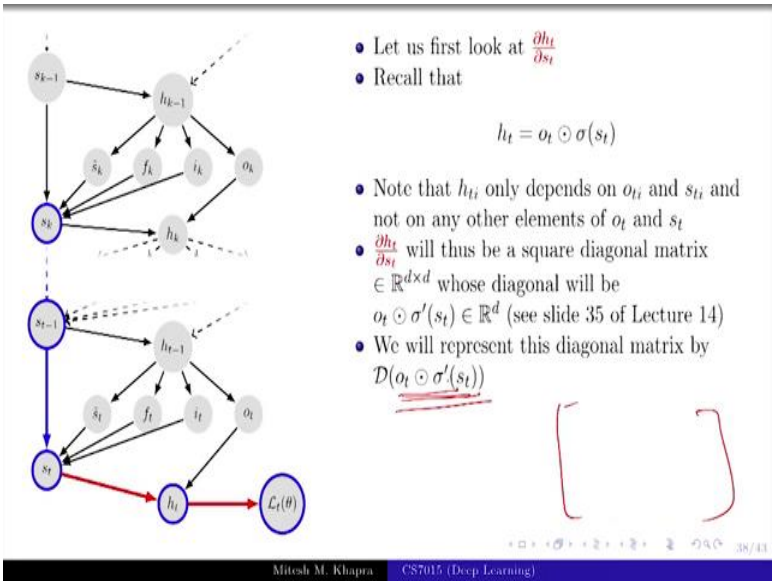
Student: I not equal to 0.

I is not equal to 0; that means, it results in a.

Student: Diagonal matrix

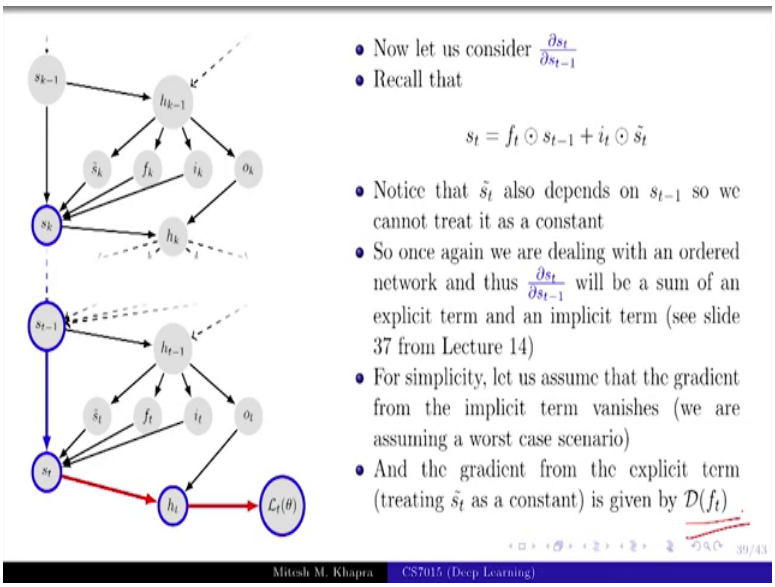
Diagonal matrix.

(Refer Slide Time: 15:31)



So, that is exactly what is written here and the diagonal elements are going to be this. Is that fine? Everyone with this? Ok. So, now, this diagonal matrix which contains this on the diagonal I am going to represent it by the following notation. Is that ok? Fine. So, this is a diagonal matrix where every element is, I mean this is actually a vector right everyone agrees this is a vector. So, this diagonal is this vector is along the diagonal of this matrix. How many if you get this notation? If you do not get this you will not understand anything else.

(Refer Slide Time: 16:07)



Now let us consider $\partial s_t / \partial s_{t-1}$. Ok. This is what s_t is equal to? So, what is the derivative of $\partial s_t / \partial s_{t-1}$? Ft right ft right what else, why no, why are you rebelling, what the I mean s_t only right. If it is. Can you treat this as a constant, no why? Because this is a dash network.

Student: (Refer Time: 16:34).

So in an ordered network the derivative will have.

Two terms which are those?

Student: Explicit.

Explicit and implicit In the explicit term what you assume? The other terms to be a constant right fine. So, s_t I mean S tilde t also depends on s_{t-1}. So, we cannot treated as a constant so once again this derivative is going to contain an explicit term and an implicit term. Now I am going to make a worst case assumption. I making this assumption that actually the implicit term vanishes. Notice that this not favourable to me I am trying to prove that the gradient does not vanish the gradient is a sum of two terms I am saying it let the worst case be that one of these terms vanishes ok.

So, this is not a favourable assumption this is a unfavourable assumption which I am making. So, let us fine. So, I making the assumption that the implicit term vanishes. So, what is the explicit term actually?

Student: F_t.

F_t and what kind of a matrix is that?

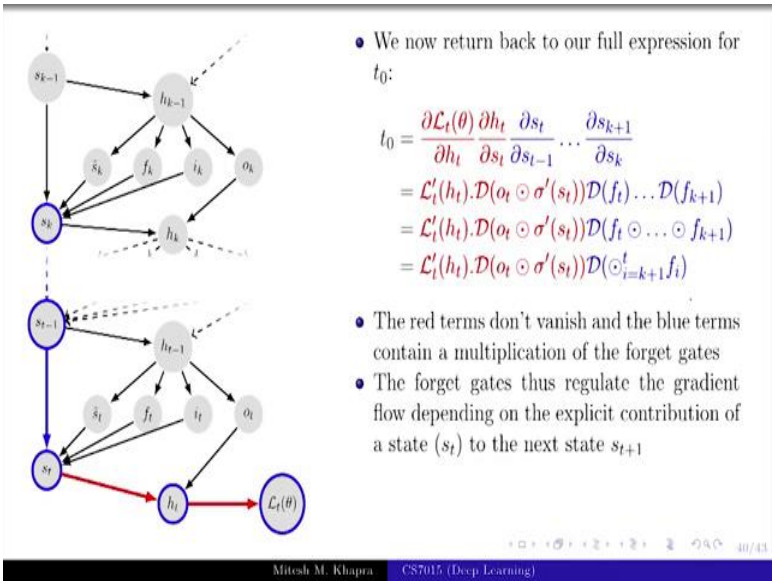
Student: Diagonal matrix.

If you agree that it is a matrix first of all. It is a diagonal matrix again and what is the diagonal?

Student: F_t.

F_t right. So, I am going to represent it as D of ft. Is that fine?

(Refer Slide Time: 17:46)



So, remember that the original equation, had three terms all of these the last blue once for all identical. So, this is not problematic because this is a directly the last layer. This we have already derived a form this is sum diagonal. And now for each of these we have a form; do you get that these are the three paths that we have done so far. So, let me just substitute them, this is what it looks like ok. Now this is a product of diagonal matrices what will the product look like?

Student: Diagonal matrix.

A diagonal matrix and each element would be each element on the diagonal would be a.

Student: Product

A product of all those things right. So, is it fair if I write it as this right, which I can write it as this ok. Now just stare at this equation and tie it back to the intuition that we developed something about the gates regulating the flow of information. You have a multiplicative term here right. Whenever there is a multiplicative term we have a problem, because remember these gates are between 0 to 1.

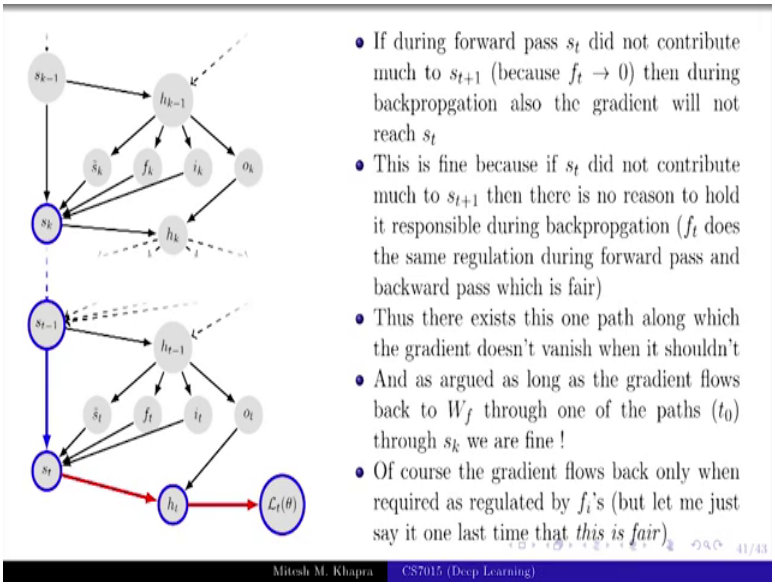
So, there is a chance of vanishing. Everyone sees that? You are multiplying t terms all of which are between 0 to 1. So, there is a chance of vanishing. But I am going to end this proof by saying that the gradient does not vanish. So, by what am I going to do now? Ok

make the statement the gradient could vanish, but this kind of vanishing is fair. What do you mean by that? Now when will the gradient vanish?

Student: Product

At this product of the forget gates vanishes, but if the product of the forget gate vanishes; that means, what would have happened during the forward pass that information was not carried all the way back, all the way front two times step t right do you see that ok. So, that is the main reason here right. So, the red term does not vanish, the red term time zone vanish the blue term can vanish, but it will vanish only if during the forward pass also this multiplicative term at cause the information to vanish by the time you are reach the time step t. How many if you get this? And this exactly what I meant earlier by saying that,

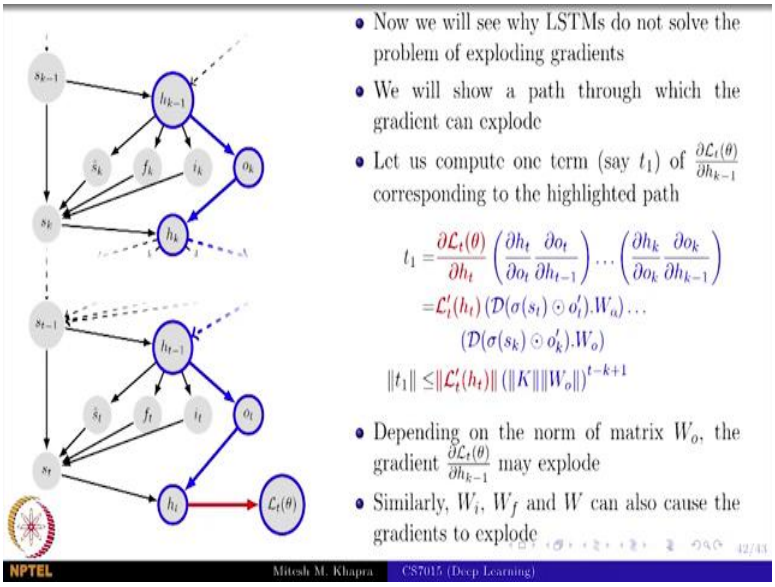
(Refer Slide Time: 19:45)



if during the forward pass s_t did not contribute much to s_{t+1}. Because the forget gate was tending to 0. Then during backward pass there is no need to pass this information back to s_t right because during forward pass you did not contribute. So, during backward pass why should I hold you responsible right? And this is absolutely fine to do this and this is exactly what the equation tells us that they gauze the gradient will vanish only if things vanished in the forward pass ok. And the gates are doing the same regulation in the forward pass as they will do in the backward pass so everything is fair. Is that ok?

And does there exist one path along which the gradients will not vanish when they do not need to vanish. So, if during forward pass all the gates were on; that means, the information from state one was actually carried all the way up to state t then during backward pass what will happen? The information will go all the way back right is that fine. So, the gradients flow back only when required as regulated by the forget gates and this is fair because if you are regulating the same thing in the forward as well as the backward direction then you are not doing anything wrong ok.

(Refer Slide Time: 20:51)



Now that is a proof for LSTM solve the vanishing gradient problems or in other words the gradients vanish only when required and not unnecessarily or arbitrary as is to happen in the case RNN's. Now we will show there exist one path along which the gradients can explode right. So, let us show that path. So, consider this path. Ok. A this path is again also active so I if we consider the path to h_k there is going to be active for all the gates and all the states right. So, in whatever gates or states you are considering this paths would be there.

And this is what this path looks like you have the derivative with respect to the last layer and then you have these guys ok these pairs h_t by o_t, o_t by $h_{t-1}$and so on. Everyone fine with this so far? What is the derivative of h_t with respect to o_t? We do not remember the equations so I will just tell you directly. So, based on whatever we have done so, far just trust me that this is what each of the terms in the bracket looks like. We can go back and

check this. This is just comes directly from the equations. Now what is happening here does this look very similar to the situation that we had with RNN's?

We had a diagonal matrix and a weight matrix and a repeated multiplication of these right? And again the diagonal matrix is bounded the weight matrix is bounded. So, now, the repeated multiplication could explode is that fine. So, it does not solve the problem of exploding gradients. But it solves the problem of vanishing gradients, but now still this is bad for us right whether the gradients explode or vanish our training is going to get messed up. So, how do we deal with this for exploding gradients what will you do?

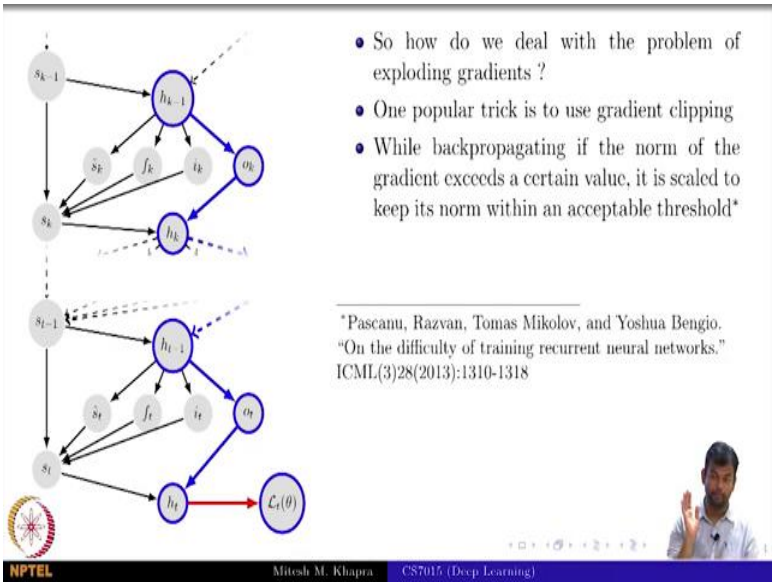
Student: (Refer Time: 22:27) clipping.

We will do?

Student: Clipping.

Clipping right.

(Refer Slide Time: 22:32)



So in practice the way of dealing with this is gradient clipping. If the norm of the gradient exceeds the certain value, then we are going to just clip it to a certain threshold. And this is fine because we care about the gradients only for the direction and not for the magnitude. Anyways when we introduce a learning rate, we are doing some kind of scale

down for the gradient magnitude. So, this is just being more explicit and being careful that if the gradients are non manageable in terms of their magnitude,

then we just going to keep them to some manageable value while being faithful to the direction. And the direction is what? Matters. So that is why exploding gradients is easy, but in the case of vanishing gradients you do not have direction also because the entire gradient becomes 0, so there is no direction there right. So, that is why vanishing gradients is more serious than exploding gradients. And as long as LSTM solve that they are fine with it is that fine ok. So, that is the end of this lecture.